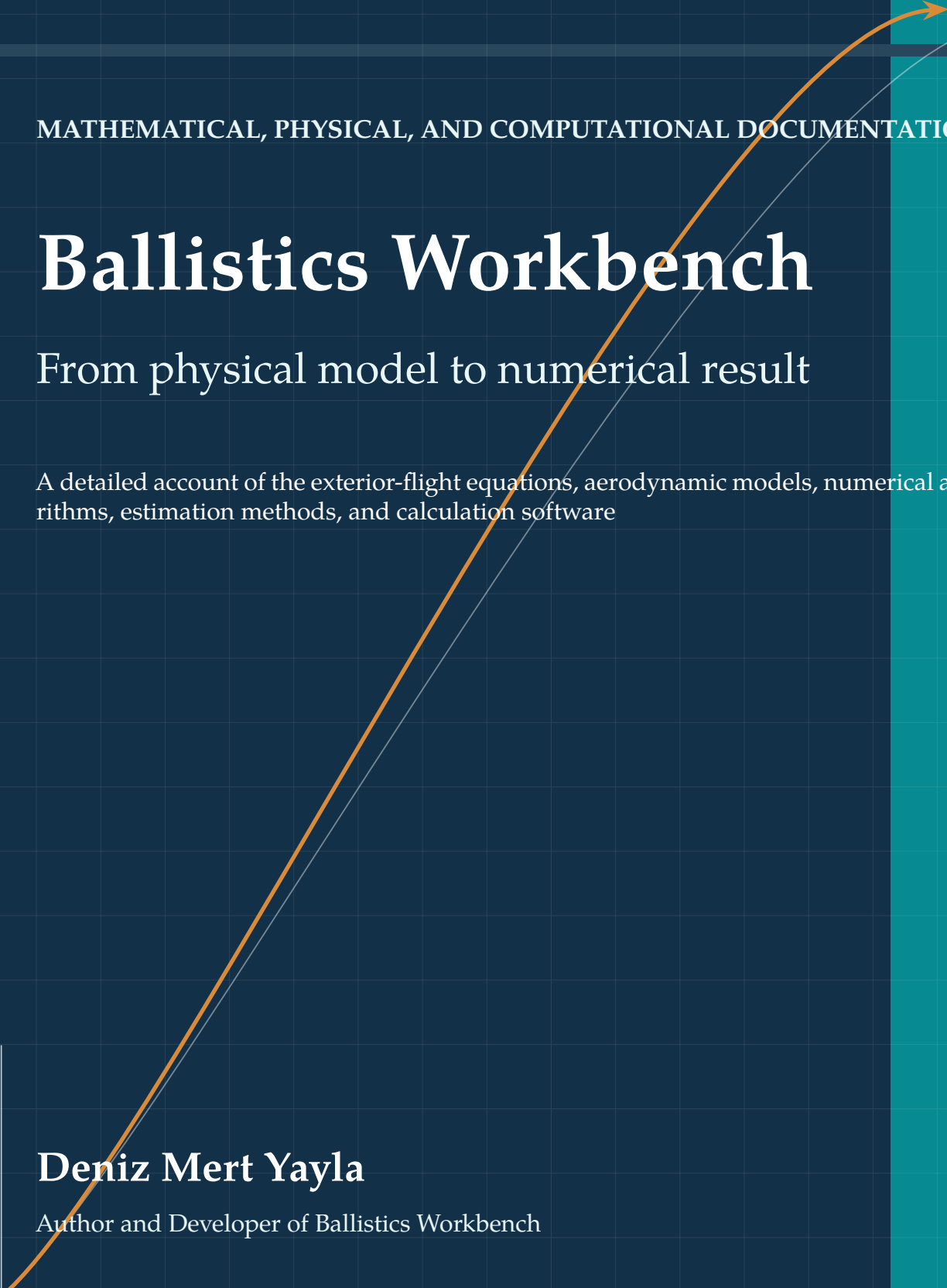


A thick orange arrow starts at the bottom left, curves upwards and to the right, and ends with an arrowhead pointing towards the top right corner of the cover.

MATHEMATICAL, PHYSICAL, AND COMPUTATIONAL DOCUMENTATION

Ballistics Workbench

From physical model to numerical result

A detailed account of the exterior-flight equations, aerodynamic models, numerical algorithms, estimation methods, and calculation software

Deniz Mert Yayla

Author and Developer of Ballistics Workbench

August 2026

Preface

I wrote BALLISTICS WORKBENCH as an exterior-ballistics workbench: a program in which a physical scenario can be stated explicitly, solved numerically, inspected at exact ranges, and compared across different projectile and drag representations. This document explains the program from the inside out. I begin with the physical quantities that the calculation represents, derive the equations that govern their evolution, and then show how I turn those equations into bounded, repeatable computations in the C++ core.

The center of the program is not its window, charts, or data files. It is a three-dimensional point-mass initial-value problem. Position and ground-relative velocity form a six-component state. At each trial time inside the integrator, the core evaluates the atmosphere and wind at the current location, constructs the projectile's velocity relative to the air, evaluates its aerodynamic model, adds gravity and optional Coriolis acceleration, and returns a state derivative. An adaptive Dormand–Prince method advances that state in time. Dense interpolation then places output at exact downrange coordinates and locates physical or geometric events between accepted integration points.

Several calculations surround that deterministic trajectory solve. Zeroing is a root-finding problem that repeatedly solves a trajectory while changing bore elevation. Maximum point-blank range is another nested search. Ballistic-coefficient calibration repeatedly solves trajectories inside a weighted nonlinear least-squares procedure. First-order uncertainty propagation estimates local derivatives by paired perturbations. Monte Carlo analysis samples a correlated input distribution and resolves the complete zeroed scenario for every accepted draw. Buckshot pattern analysis is separate from exterior flight: it fits an empirical D90 growth model and converts it into target and pellet-count probabilities.

I have written the explanation for a reader who is comfortable with algebra, calculus, differential equations, probability, and ordinary program logic, but who need not already know exterior ballistics or C++. The mathematical chapters therefore define symbols and build the model progressively. The implementation chapters emphasize the sequence of calculations and the relationships between physical models, solvers, interpolation, root finding, analysis passes, and stochastic wrappers. Where it clarifies the idea, a short excerpt of the actual C++ core stands in place of pseudocode, explained in terms of ordinary data flow and control flow rather than language mechanics.

The numerical values throughout the text come from the current implementation and the calculation records that accompany it. Every formula, coefficient, and bound is stated so that a result can be traced from the physical scenario to the reported quantity.

Purpose of this document

The main purpose is to make the calculation intelligible. A reader should be able to follow a projectile from its initial state, through atmosphere and drag evaluation, through the adaptive integrator, to sight-relative events and derived observables. The same reader should then be able to understand why calibration and uncertainty require many repeated trajectory solutions and how the program distinguishes a numerical result from a model-domain statement.

Scope and organization

This document is an account of the exterior-flight calculation at the center of BALLISTICS WORKBENCH: the mathematics of a projectile's motion through the atmosphere, the physical models that supply the forces acting on it, the numerical methods that solve the resulting equations, and the estimation and uncertainty procedures built on top of that solve. It is written to be read on its own terms, as a study of the models and algorithms, rather than as a manual for a particular piece of software.

The subject is deliberately bounded. The model treats the projectile as a translating point mass acted on by aerodynamic drag, gravity, and an optional Coriolis term. It does not describe internal ballistics, barrel dynamics, or recoil; it does not integrate the projectile's attitude, so it excludes six-degree-of-freedom aerodynamics, coning, and aerodynamic jump; and it does not represent terrain, obstacles, or terminal interaction with a target. Where a quantity such as spin drift or a buckshot pattern falls outside the point-mass trajectory, it is computed by a separate empirical relation and reported as such. Keeping these boundaries explicit is part of the model's discipline, and the text is careful throughout to distinguish a numerical result from a statement about the domain in which the underlying model is valid.

What the calculation produces

For each selected load, BALLISTICS WORKBENCH produces a time-resolved trajectory and a range-indexed presentation of that trajectory. The directly integrated state consists only of position and ground-relative velocity. Other quantities are evaluated from that state: air-relative velocity, airspeed, Mach number, Reynolds number where applicable, drag coefficient or reference drag coefficient, kinetic energy, linear momentum, time of flight, geometric drop, sight-relative path, angular holdover, and lateral wind displacement. The program can add an empirical spin-drift displacement for a rifle projectile when the required geometry and stability conditions are present.

The trajectory-analysis pass localizes near and far sight-line zero, maximum ordinate, selected Mach crossings at 1.2, 1.0, and 0.8, supersonic range, and ground intersection. A separate maximum-point-blank-range calculation finds a bore angle whose greatest sight-relative ordinate reaches the upper boundary of a circular vital zone, then locates the later crossing of the lower boundary.

Estimation and uncertainty use the same deterministic physics rather than alternate approximations. Ballistic-coefficient calibration predicts each observed velocity by running the trajectory solver with trial coefficients. First-order uncertainty pairs slightly higher and lower inputs around a nominal zeroed solution. Monte Carlo uncertainty draws correlated normal perturbations and repeats the complete scenario, including zeroing. These layers differ in their statistical interpretation, but all of them call the same trajectory engine to integrate the same state equations.

The buckshot pattern module addresses a different question. It does not cause the exterior-flight solver to treat a nine-pellet load as one projectile with nine times the mass. The exterior trajectory remains a per-pellet or per-projectile solve. Pattern analysis instead fits the radial spread of impacts and then derives probability distributions for one pellet and for the pellet count on a defined target region.

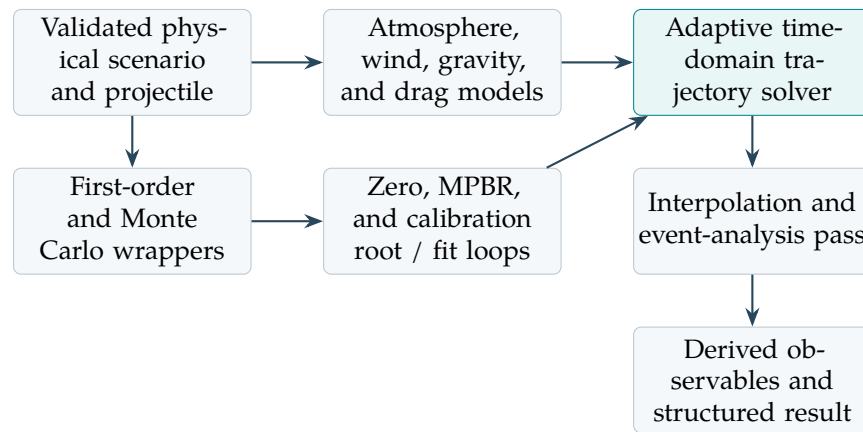


Figure 1.: The deterministic trajectory solver is the central numerical operation. Root finders, fitters, and uncertainty methods call it repeatedly; interpolation and analysis interpret its accepted state history.

How the calculation layers relate

A *physical model* maps the current state and scenario to quantities such as density, sound speed, wind, drag coefficient, and acceleration. A *solver* advances differential equations under that model. A *root finder* varies one input until a scalar geometric error vanishes. *Interpolation* evaluates a continuous approximation between states already accepted by the solver. A *derived analysis pass* searches the retained trajectory for crossings or extrema without changing the flight itself. A *stochastic wrapper* repeats the deterministic process under sampled inputs. Keeping these roles distinct is essential: a zeroing error is not an integration error, a graph sample is not an event root, and a Monte Carlo percentile is not a new aerodynamic law.

Organization

Parts I through V develop the deterministic exterior-flight calculation. Part VI explains calibration and uncertainty. Part VII treats the separate buckshot pattern model. Part VIII follows the actual C++ execution path and shows how the mathematical layers cooperate. Part IX gives a compact account of the desktop workflow. Part X explains how I check the implementation and where the model stops. The appendices collect symbols, constants, bounds, built-in load definitions, status meanings, source landmarks, and references.

Contents

Preface	2
Scope and organization	3
I. Mathematical and physical foundations	16
1. What the flight state represents	17
1.1. Local shot coordinates	17
1.2. The six-component state	18
1.3. Ground-relative and air-relative motion	18
1.3.1. Worked vector example	19
1.4. Initial velocity from bore elevation and azimuth	19
1.5. Scalar speed is not the forward component	20
2. Newtonian point-mass flight	21
2.1. From force to a first-order differential system	21
2.2. Deriving the drag vector	21
2.3. Dimensions and similarity parameters	22
2.4. A direct drag-acceleration calculation	23
2.5. Point-mass assumptions	23
3. Observables derived from the state	24
3.1. Kinetic energy and linear momentum	24
3.1.1. Worked muzzle calculation	24
3.2. Per-projectile values and payload totals	24
3.3. Geometric drop	25
3.4. Path and angular holdover	25
3.5. Lateral displacement, wind, Coriolis, and spin	25
3.6. Time, Mach, Reynolds number, and aerodynamic diagnostics	26
3.7. A compact dependency map	27
II. Atmosphere and environmental physics	28
4. Thermodynamic properties of the air	29
4.1. Input state and thermodynamic units	29
4.2. Water-vapor pressure	29
4.3. Moist-air density	30
4.3.1. Worked firing-point atmosphere	31
4.4. Speed of sound: the Cramer calculation	31
4.4.1. Why sound speed matters downstream	32
4.5. Dynamic viscosity and Sutherland's relation	32
4.6. Atmospheric model identity and comparison	32
5. Pressure, altitude, and changing atmosphere	34
5.1. Station pressure versus altimeter setting	34
5.2. Density altitude	35
5.3. Altitude-dependent flight atmosphere	35

6. Wind, gravity, and Earth rotation	37
6.1. Homogeneous wind	37
6.2. Piecewise-linear wind layers	37
6.2.1. Interpolation example	37
6.3. Constant and local gravity	38
6.4. Coriolis acceleration in a rotating frame	38
6.4.1. Following one orientation	39
6.5. Combined derivative evaluation	39
 III. Aerodynamic drag models	 40
7. Reference drag and ballistic coefficient	41
7.1. Direct C-D and reference BC answer different questions	41
7.2. Density scaling in the reference formulation	41
7.3. Constant and velocity-banded BC	42
7.4. G1 piecewise retardation	43
7.5. G7 tabulated reference curve	44
7.6. Reference-BC validity and interpretation	46
8. Direct Mach–C-D and spherical drag	47
8.1. Custom Mach–C-D data	47
8.1.1. Interpolation and clamping example	47
8.2. Sphere geometry and mass	47
8.3. Reynolds-dependent base sphere correlation	48
8.4. Compressibility correction	48
8.5. Declared sphere domain and behavior outside it	50
8.6. Observed drag validity	50
8.7. Comparing the four aerodynamic families	50
9. Compressible flow around the projectile	53
9.1. Streamlined supersonic flow	53
9.2. Blunt-body flow and the price of a sphere	54
 IV. Numerical trajectory solution	 56
10. The trajectory as an initial-value problem	57
10.1. Compact ODE form	57
10.2. Why time is the production independent variable	57
10.3. The validation-only range solver	58
11. Dormand–Prince embedded Runge–Kutta 5(4)	59
11.1. What a Runge–Kutta stage means	59
11.2. The implemented tableau	59
11.3. Component-wise error normalization	60
11.4. Accepting and rejecting a step	61
11.5. Configured time-step and work bounds	61
11.6. The adaptive loop in the core	62
12. Dense output and numerical termination	64
12.1. Cubic Hermite position	64
12.2. Finding an exact downrange coordinate	64

12.3.	The internal range grid	65
12.4.	Finite-state detection	65
12.5.	Termination conditions and their order	65
12.5.1.	Requested distance	65
12.5.2.	Horizontal reversal	66
12.5.3.	Ground impact	66
12.5.4.	Minimum forward speed	66
12.5.5.	Maximum time and maximum steps	66
12.5.6.	Non-finite state	66
12.6.	Partial trajectories	67
12.7.	Analytical constant-drag comparison	67
12.8.	Convergence evidence for production settings	67
V.	Sight geometry and trajectory analysis	69
13.	Bore line, sight line, and zeroing	70
13.1.	Three different geometric objects	70
13.2.	The zero error function	70
13.2.1.	A sign-changing illustration	71
13.3.	Bracketing the root	71
13.4.	Bisection convergence	72
13.5.	Near zero, far zero, and configured zero	73
13.6.	Worked native zero	73
14.	Maximum point-blank range	74
14.1.	Vital-zone radius	74
14.2.	The optimal-bore condition	74
14.3.	Bracketing and refinement	74
14.4.	Complete and horizon-limited results	75
14.5.	Configured zero versus MPBR zero	75
15.	Trajectory events and exact geometric analysis	76
15.1.	Range-domain Hermite path interpolation	76
15.2.	Near and far sight-line zero	76
15.3.	Maximum ordinate	77
15.4.	Mach crossings and supersonic range	77
15.5.	Ground intersection	77
15.6.	Worked event set	78
16.	Spin stability and empirical spin drift	79
16.1.	Required geometry and units	79
16.2.	Implemented Miller stability equation	79
16.3.	Status thresholds	80
16.4.	Time-of-flight drift relation	80
16.5.	Relationship to the lateral trajectory	80
17.	An end-to-end deterministic example	81
17.1.	Scenario and projectile	81
17.2.	Representative states	81
17.3.	Analysis after integration	82

VI. Estimation and uncertainty	83
18. Ballistic-coefficient calibration	84
18.1. Observations and predictions	84
18.2. Constant and velocity-banded fits	84
18.3. Logarithmic parameterization	85
18.4. Numerical Jacobian	85
18.5. Gauss–Newton model and Levenberg–Marquardt damping	85
18.6. Trial-step acceptance	86
18.7. Residual diagnostics	87
18.8. Covariance approximation and coefficient intervals	88
18.9. Current fitted built-ins	88
19. First-order uncertainty propagation	89
19.1. Local linearization	89
19.2. Central finite differences	89
19.3. Drag perturbation semantics	90
19.4. Fixed bore versus re-zeroing	90
19.5. Output construction and availability	91
19.6. Worked variance contribution	91
20. Monte Carlo uncertainty	93
20.1. Input random vector	93
20.2. Cholesky construction	93
20.2.1. Two-variable example	94
20.3. Deterministic pseudorandom normals	94
20.4. Transforming samples into physical inputs	94
20.5. Complete scenario solution per draw	95
20.6. Empirical quantiles	96
20.7. Split-sample quantile diagnostic	96
20.8. First-order and Monte Carlo compared	97
VII. Buckshot pattern analysis	98
21. Empirical D90 pattern model	99
21.1. What D90 means	99
21.2. Through-origin linear growth	99
21.3. Calibration and holdout residuals	100
21.4. Slope uncertainty and predicted diameter interval	101
22. From D90 to target probability	102
22.1. Isotropic Gaussian impact model	102
22.1.1. Worked D90 conversion	102
22.2. Rectangular target probability	103
22.3. Circular target probability	103
22.4. Diameter interval to probability interval	104
23. Pellet-count probabilities and assumptions	105
23.1. Binomial model	105
23.2. Stable logarithmic calculation	105
23.3. What independence assumes	106
23.4. Complete model pipeline	106

VIII. How the calculations are implemented	108
24. Calculation-oriented core design	109
24.1. Core data as physical concepts	109
24.2. Five computational roles	109
24.2.1. Physical model	109
24.2.2. Differential-equation solver	110
24.2.3. Root-finding operation	110
24.2.4. Interpolation and derived analysis	110
24.2.5. Estimator or stochastic wrapper	110
25. The deterministic execution path	112
25.1. 1. Validate and normalize the request	112
25.2. 2. Construct atmosphere and environment	112
25.3. 3. Assemble the load set	112
25.4. 4. Select drag behavior	113
25.5. 5. Build the trial launch state	113
25.6. 6. Evaluate the derivative at a provisional state	113
25.7. 7. Advance adaptively	114
25.8. 8. Retain exact-range states	114
25.9. 9. Solve sight zero by repetition	114
25.10. 10. Solve native MPBR separately	114
25.11. 11. Analyze events and drag validity	115
25.12. 12. Select public output samples	115
25.13. 13. Run optional uncertainty	115
25.14. 14. Run optional buckshot pattern analysis	115
25.15. 15. Assemble the structured result	116
26. Repeated solves, bounds, and failure containment	117
26.1. Validation before evaluation	117
26.2. Avoiding invalid divisions	117
26.3. Bracketing rather than unbounded iteration	117
26.4. Finite states and partial trajectories	118
26.5. Determinism	118
26.6. Bounding computational work	118
26.7. Numerical diagnostics as part of the result	119
27. Source-to-equation landmarks	120
IX. Desktop workflow	122
28. From interface input to native result	123
28.1. Desktop calculation workflow	123
28.2. Calculation boundary	123
28.3. Debouncing, cancellation, and stale results	123
28.4. Custom loads, profiles, and export	124
X. Verification and model limitations	125
29. How I verify the calculations	126
29.1. Mathematical identity and analytical cases	126

29.2.	Adaptive convergence	126
29.3.	Independent trajectory implementations	126
29.4.	Atmospheric properties	127
29.5.	Reference drag and manufacturer tables	127
29.6.	Calibration recovery and uncertainty behavior	127
29.7.	Buckshot pattern checks	128
29.8.	Protocol robustness	128
29.9.	Current summary	128
30.	Scientific and numerical limitations	130
30.1.	Translational point-mass boundary	130
30.2.	Local geometry and Earth model	130
30.3.	Atmosphere and wind	130
30.4.	Aerodynamic data	131
30.5.	Numerical solution	131
30.6.	Calibration	131
30.7.	Uncertainty	131
30.8.	Buckshot probability	132
30.9.	Derived totals and interpretation	132
30.10.	Software boundary	132
XI.	Appendices	133
A.	Symbols, frames, and units	134
A.1.	Coordinate and state symbols	134
A.2.	Dimensional and sign audit conventions	134
A.3.	Aerodynamic and projectile symbols	135
A.4.	Atmospheric symbols	135
A.5.	Sight and event symbols	136
A.6.	Estimation and uncertainty symbols	137
B.	Constants, defaults, and configured bounds	138
B.1.	Physical and atmospheric constants	138
B.2.	Adaptive trajectory defaults	138
B.3.	Nested numerical procedures	139
B.4.	Encoded atmosphere and drag domains	140
C.	Built-in projectile definitions	142
C.1.	Reference-BC projectiles	142
C.2.	Derived 00-buck sphere	142
C.2.1.	Initial pellet observables	143
C.3.	Custom projectile invariants	143
D.	Status meanings and partial results	144
D.1.	Trajectory termination	144
D.2.	Sight solutions and trajectory events	144
D.3.	Drag and spin-drift validity	145
D.4.	Estimation, uncertainty, and pattern status	146
E.	Algorithm reference	147
E.1.	Derivative, solver, and zeroing	147
E.2.	Calibration and Monte Carlo	147

E.3.	Buckshot pattern fit	148
F.	End-to-end native calculation example	149
F.1.	Scenario and projectile	149
F.2.	Atmospheric state	149
F.3.	Zero solution and initial state	150
F.4.	Adaptive integration record	150
F.5.	Drop, path, and holdover	151
F.6.	Localized trajectory events	151
F.7.	MPBR as a separate optimized geometry	151
F.8.	First-order uncertainty at the output range	152
F.9.	What the example demonstrates	152
F.10.	Reproducibility boundary	153
G.	Dimensional and limiting checks	154
G.1.	Direct-drag dimensions	154
G.2.	Vacuum and no-wind limits	154
G.3.	Sight geometry limits	154
G.4.	Uncertainty limits	155
G.5.	Pattern-probability limits	155
H.	Calculation source map	156
H.1.	Core public model	156
H.2.	Deterministic flight sources	157
H.3.	Repeated-solve and analysis sources	158
H.4.	Native request and result path	159
I.	Reference sources	161
I.1.	Bibliography	161
I.2.	How the references enter the program	162
J.	Closing model statement	163

List of Figures

1.	The deterministic trajectory solver is the central numerical operation. Root finders, fitters, and uncertainty methods call it repeatedly; interpolation and analysis interpret its accepted state history.	4
1.1.	Local coordinates and representative vectors. The drawing is schematic; drag is exactly antiparallel to air-relative velocity, not necessarily to the ground-relative arrow shown here.	17
2.1.	Drag-vector construction. Wind changes v_a , so drag need not be exactly opposite ground-relative velocity.	22
4.1.	Magnus saturation vapor pressure over the accepted temperature range. The near-exponential rise means that at 40 °C saturated air holds more than ten times the moisture it does at 0 °C.	30
4.2.	Sutherland dynamic viscosity across the accepted temperature range. Unlike a liquid, air becomes more viscous as it warms because faster molecular motion transfers more momentum across the flow. The change is gentle, near 29 % from −40 °C to 50 °C.	33
5.1.	Air density and sound speed against height under the standard dry lapse used by the altitude-dependent model. Density falls about 46 % over the first 6 km; sound speed falls about 7 %, tracking the temperature drop.	35
7.1.	The G1 standard-projectile drag, expressed as an equivalent $C_D(M)$ from the retardation table. Compared with G7 in Figure 7.2, the peak is broader, higher, and sits near $M \approx 1.35$ rather than just above Mach 1. This is the signature of a blunt flat-base reference shape, and it is why a G1 coefficient and a G7 coefficient of equal numerical value describe different deceleration histories.	44
7.2.	Shape of the G7 reference curve. Drag is nearly flat below $M \approx 0.8$, rises steeply through the transonic region to a sharp peak just above Mach 1, then decays through the supersonic range. The production interpolation uses the full source table, not only the points drawn here.	45
8.1.	The smooth-sphere base coefficient of Equation (8.4). At low Reynolds number it follows the Stokes $24/Re$ law; through the intermediate range it settles near 0.4; then near $Re \approx 3 \times 10^5$ the boundary layer turns turbulent and drag collapses to roughly 0.08 before recovering. A pellet passes through this range as it slows, so its drag is not a single constant.	49
8.2.	G1 and G7 standard-projectile drag on common axes. The blunt G1 shape carries markedly more supersonic drag than the boat-tailed G7 shape, and the two peaks fall at different Mach numbers. A ballistic coefficient only has meaning together with the family whose curve it scales.	51
9.1.	Reduced-order flow around the spitzer boat-tail near $M \approx 2.4$, drawn on its real silhouette. An attached oblique shock (amber) leaves the nose at angle β , steeper than the Mach angle μ . The boat-tail shoulder sets off an expansion fan (green), and the base leaves a narrow separated wake. The streamlined shape keeps both the shock and the wake small, which is why its supersonic C_D is low.	53

9.2. Mach angle and attached oblique shock angle against Mach for a nose deflection near that of the spitzer profile. The shock always stands steeper than the Mach wave, and for this deflection it can no longer stay attached below $M \approx 1.6$	54
9.3. Reduced-order flow around the sphere above Mach 1. With no sharp nose, the shock detaches and stands off as a bow shock, the flow stagnates at high pressure on the front face, and separation occurs early, leaving a wide wake. Both effects are large, which is the physical reason a sphere sheds speed so quickly.	54
9.4. Static pressure and density jump across a normal shock as functions of upstream Mach number. The pressure rise is steep; the density rise saturates toward the strong-shock limit of $(\gamma + 1)/(\gamma - 1) = 6$. A blunt body drives close to this compression on its front face at every point of a supersonic flight.	55
11.1. Adaptive step control. Rejection changes only the trial step; acceptance changes the state and enables dense interpolation across the interval.	63
12.1. Fourth-order convergence of fixed-step Runge–Kutta on the gravity-free constant-drag case. The measured terminal-speed error tracks the reference line of slope four across five step halvings, confirming the expected $O(h^4)$ behavior of the method independently of the adaptive controller.	68
13.1. Sight line, bore line, and a representative zeroed trajectory. Zeros are intersections of trajectory and sight line, not intersections with the horizontal axis.	70
13.2. Computed sight-relative path for the built-in A-MAX load at a 250 m zero, from the production solver. The near zero (19.1 m), maximum ordinate (119.8 mm at 138.1 m), and far zero (250 m) are the values returned by the event-analysis pass of Chapter 15, not estimates read off the curve.	71
14.1. MPBR geometry. The optimized trajectory touches the upper path limit and later crosses the lower limit.	75
15.1. Mach decay for the A-MAX load and the three tracked crossings. Supersonic range is the first decelerating Mach-1 crossing, here 1,028 m. The projectile is transonic across a long stretch of its flight, which is where a reference curve changes most rapidly.	78
17.1. Ground speed and geometric drop for the worked A-MAX fixture over the full 500 m horizon. Drop is drawn with the axis reversed so the trajectory falls downward as it does in flight; it is briefly negative where the zeroed bore angle lifts the projectile above the muzzle plane before gravity dominates.	82
19.1. First-order speed uncertainty for the A-MAX fixture, computed by the same central-difference perturbations the engine uses. The combined curve is the quadrature sum of the two contributions; its early dip and later rise show that no single input dominates the whole trajectory.	92
21.1. Through-origin D90 growth model with its slope-uncertainty band (values illustrative). The fit has a single parameter, so the whole family of admissible lines pivots about the origin. Calibration observations at longer ranges therefore constrain the slope more tightly than observations near the muzzle.	100
22.1. Single-pellet probability of hitting a fixed 0.10 m-radius circle as range grows and the pattern widens, for slope $\alpha = 0.02$ rad. The isotropic Gaussian scale σ grows linearly with range, so the probability follows the Rayleigh law $1 - \exp(-R_T^2/2\sigma^2)$	103

- 23.1. Binomial pellet-count distribution for the worked example ($N = 9, p = 0.2257$). Shaded amber bars are the outcomes counted by $P(K \geq 3) \approx 0.33$. The spread of this distribution, not just its mean, is what a minimum-hit requirement depends on. 106
- 25.1. Per-load calculation sequence at the native response boundary. Blue blocks contain repeated calls to the same deterministic trajectory solver. 116
- 28.1. Concise desktop calculation path. The surrounding application transports and presents the calculation; the C++ core evaluates it. 124

List of Tables

3.1. Which base quantities determine common outputs.	27
7.1. G1 threshold table used by <code>g1_retardation</code> . The active row is the first row for which $V \geq V_j$	43
7.2. Complete G7 Mach- C_D knots used by the core, arranged as two pairs per row. . . .	45
8.1. Drag-model semantics in Ballistics Workbench.	51
12.1. Production trajectory termination statuses.	66
17.1. Selected native results from the current protocol fixture.	81
18.1. Current deterministic coefficient-recovery results.	88
19.1. First-order finite-difference step selection.	90
20.1. Interpretive differences between the two uncertainty methods.	97
24.1. Important calculation data structures and their meaning.	110
27.1. Calculation features and implementation landmarks.	120
29.1. Current atmosphere-property comparisons.	127
29.2. Current calculation verification summary.	128
A.1. Coordinate, state, and kinematic notation.	134
A.2. Aerodynamic and projectile notation.	135
A.3. Atmospheric notation.	136
A.4. Sight-geometry and event notation.	136
A.5. Estimation, uncertainty, and pattern notation.	137
B.1. Physical and atmospheric constants encoded by the current model.	138
B.2. Production adaptive-solver defaults.	139
B.3. Bounds and stopping scales for repeated-solve procedures.	139
B.4. Selected declared or enforced model domains.	140
C.1. Built-in reference-ballistic-coefficient loads.	142
D.1. Trajectory termination states.	144
D.2. Zeroing, MPBR, and event-analysis statuses.	144
D.3. Aerodynamic-domain and spin-drift statuses.	145
D.4. Statuses for repeated-solve and empirical analyses.	146
E.1. Where each core algorithm is shown as real source.	147
F.1. Selected exact-range values from the native A-MAX fixture.	150
H.1. Public calculation types in <code>src/Ballistics.Core/include/ballistics.hpp</code>	156
H.2. Deterministic physical-model and solver landmarks.	157
H.3. Analysis, estimation, and uncertainty landmarks.	158
H.4. Calculation-oriented native boundary.	159

Part I.

Mathematical and physical foundations

1. What the flight state represents

Physical interpretation

Ballistics Workbench evolves one translating projectile in a local right-handed Cartesian frame. The six integrated numbers are position and ground-relative velocity. Wind does not alter that definition of velocity; it creates a second, air-relative velocity used by aerodynamic models.

1.1. Local shot coordinates

I use a local frame attached to the firing point. Its axes are

x : downrange along the local shot direction,
 y : vertical, positive upward,
 z : lateral, positive to the shooter's right.

The origin is the muzzle. The position vector is

$$r(t) = [x(t) \ y(t) \ z(t)]^T, \quad (1.1)$$

and its time derivative is the ground-relative velocity

$$v(t) = \dot{r}(t) = [v_x(t) \ v_y(t) \ v_z(t)]^T. \quad (1.2)$$

Because $\hat{x} \times \hat{y} = \hat{z}$, the basis is right-handed. This convention fixes every later lateral sign. A positive integrated z displacement lies to the right; a positive v_y points upward; a projectile falling below the horizontal bore-origin plane has negative y .

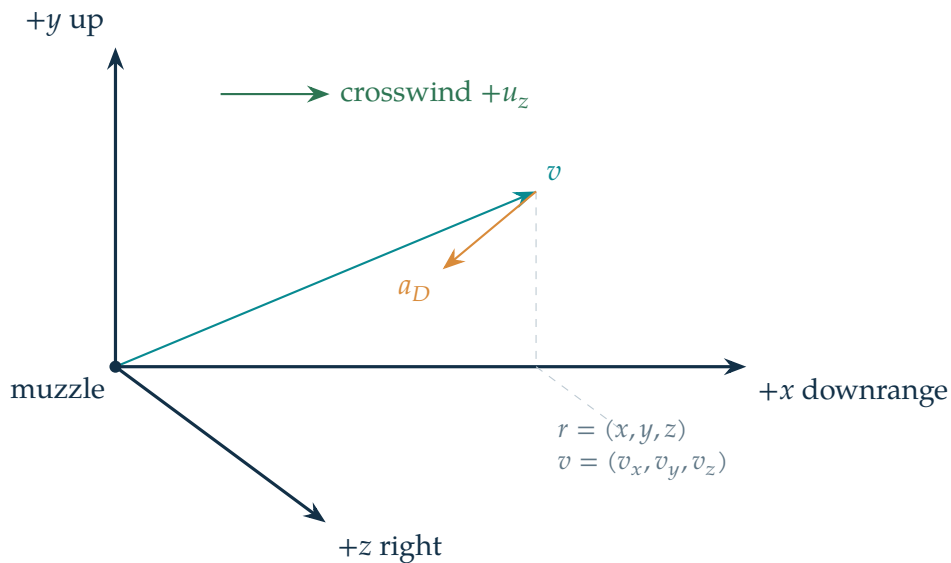


Figure 1.1.: Local coordinates and representative vectors. The drawing is schematic; drag is exactly antiparallel to air-relative velocity, not necessarily to the ground-relative arrow shown here.

The axes are local rather than geocentric. Over the modeled path I treat them as a fixed tangent frame. Optional latitude and shot azimuth are used to express the rotating Earth's angular velocity in an east-north-up frame and then transform Coriolis acceleration back into (x, y, z) . The position itself is not propagated on an ellipsoid, and the ground is not curved beneath the shot.

1.2. The six-component state

The numerical state is

$$s(t) = [x(t) \ y(t) \ z(t) \ v_x(t) \ v_y(t) \ v_z(t)]^T \in \mathbb{R}^6. \quad (1.3)$$

The first three components have dimensions of length and the last three dimensions of length per time. This split matters to adaptive error control: position components use an absolute tolerance in metres, while velocity components use a separate absolute tolerance in metres per second. A single unscaled Euclidean error across all six components would add unlike dimensions and let the numerical magnitude of one unit system dominate the other.

No acceleration, Mach number, energy, or drop variable is integrated as an independent state. The derivative function recomputes acceleration from the current state and environment. Diagnostics and observables are derived when a state is retained. This avoids accumulating a second numerical history for quantities that are exact algebraic functions of r and v .

The production source represents Equation (1.3) as a fixed six-element numerical array in `src/Ballistics.Core/src/ballistics.cpp`. The layout is explicitly

$$\{x, y, z, v_x, v_y, v_z\}.$$

That ordering is used consistently by the derivative routine, Dormand–Prince stages, error normalization, finite-value checks, and Hermite interpolation.

1.3. Ground-relative and air-relative motion

Four related quantities must remain distinct:

Ground-relative velocity v .

The time derivative of position in the local frame. It determines where the projectile goes.

Ground speed v_g .

The scalar magnitude $v_g = \|v\|$.

Air-relative velocity v_a .

Projectile velocity relative to the moving air mass. It determines aerodynamic force direction and magnitude.

Airspeed v_a .

The scalar magnitude $v_a = \|v_a\|$, used for Mach number and Reynolds number.

Let the physical wind velocity relative to the ground be u . Relative velocity follows from subtracting reference-frame velocities:

$$v_a = v - u. \quad (1.4)$$

The program's input convention describes wind with a positive headwind component w_h blowing toward the shooter and a positive crosswind component w_c blowing from the shooter's left to right. In shot coordinates, the air mass therefore moves as

$$u = [-w_h \ 0 \ w_c]^T. \quad (1.5)$$

Substitution into Equation (1.4) gives the exact transformation in the core:

$$v_a = \begin{bmatrix} v_x + w_h \\ v_y \\ v_z - w_c \end{bmatrix}. \quad (1.6)$$

The plus sign on headwind is physically important. If the projectile moves at 800 m/s downrange through a 10 m/s wind moving toward the muzzle, the closing speed is near 810 m/s, not 790 m/s. The minus sign on crosswind is equally important. With $v_z = 0$ and $w_c > 0$, the air-relative z component is negative. Drag opposes v_a , so its lateral component is positive, accelerating the projectile to the right with the moving air.

Physical interpretation

A headwind can change drag immediately even though the ground-relative velocity has not yet changed. The atmosphere is moving toward the projectile, so the relative flow is faster. A crosswind does not appear as a separate sideways force term; the sideways acceleration is the lateral component of the same drag vector that acts opposite v_a .

1.3.1. Worked vector example

Take

$$v = (800, 0, 0) \text{ m/s}, \quad w_h = 10 \text{ m/s}, \quad w_c = 5 \text{ m/s}.$$

Then

$$\begin{aligned} v_a &= (810, 0, -5) \text{ m/s}, \\ v_g &= 800 \text{ m/s}, \\ v_a &= \sqrt{810^2 + (-5)^2} = 810.015 \text{ m/s}. \end{aligned}$$

The unit vector along the relative flow is approximately

$$\frac{v_a}{v_a} = (0.999981, 0, -0.006173).$$

If the scalar drag acceleration at this condition were a_D , its vector would be

$$a_D = -a_D \frac{v_a}{v_a} \approx (-0.999981a_D, 0, +0.006173a_D).$$

The small positive z component is the rightward response to the left-to-right crosswind.

1.4. Initial velocity from bore elevation and azimuth

Let v_0 be the configured muzzle velocity after its multiplicative scenario adjustments. Let θ_b be the launch or bore elevation above the local horizontal, and let ψ_b be the launch azimuth within the shot frame. The initial position and velocity are

$$r(0) = (0, 0, 0), \quad (1.7)$$

$$v(0) = v_0 \begin{bmatrix} \cos \theta_b \cos \psi_b \\ \sin \theta_b \\ \cos \theta_b \sin \psi_b \end{bmatrix}. \quad (1.8)$$

The factors follow from resolving a unit vector: elevation first divides speed into vertical and horizontal parts; azimuth then divides the horizontal part between downrange and lateral components. The squared magnitude is

$$v_0^2 (\cos^2 \theta_b \cos^2 \psi_b + \sin^2 \theta_b + \cos^2 \theta_b \sin^2 \psi_b) = v_0^2,$$

so the construction preserves the requested launch speed exactly apart from floating-point rounding.

Normal desktop zeroing varies θ_b while leaving $\psi_b = 0$. The solver supports a nonzero local launch azimuth, but Earth-rotation shot azimuth is a different quantity: it locates the downrange axis relative to north and east for the Coriolis transform. Confusing these two angles would mix firearm alignment with geographic orientation.

1.5. Scalar speed is not the forward component

The scalar ground speed is

$$v_g = \sqrt{v_x^2 + v_y^2 + v_z^2}. \quad (1.9)$$

It is generally larger than $|v_x|$ when the projectile has vertical or lateral motion. The production solver uses v_x for downrange monotonicity and termination, because requested output is indexed by x . It uses v_g for kinetic energy and momentum magnitude. It uses v_a for aerodynamic laws. These are separate numerical decisions arising from separate physical definitions.

The minimum-forward-speed termination also tests v_x , not v_g . A state can retain appreciable vertical speed while ceasing to make useful downrange progress. Horizontal reversal is detected when $v_x \leq 0$ and is localized within the accepted time step before the ordinary minimum-forward-speed condition is considered.

2. Newtonian point-mass flight

Physical interpretation

The trajectory is the solution of Newton's second law for a translating mass. Position differentiates to velocity; velocity differentiates to the sum of aerodynamic, gravitational, and optional Coriolis accelerations.

2.1. From force to a first-order differential system

For a projectile of constant mass m , Newton's second law is

$$m \frac{dv}{dt} = \sum F. \quad (2.1)$$

Dividing by m gives an acceleration equation. Together with the kinematic identity $\dot{r} = v$, the model becomes

$$\frac{dr}{dt} = v, \quad (2.2a)$$

$$\frac{dv}{dt} = a_D + a_g + a_C. \quad (2.2b)$$

Here a_D is aerodynamic drag, a_g is gravity, and a_C is the optional Coriolis acceleration. In the local frame, constant gravity is

$$a_g = (0, -g, 0), \quad g = 9.80665 \text{ m/s}^2. \quad (2.3)$$

Writing all six scalar equations shows exactly what the derivative routine returns:

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{z} = v_z, \quad (2.4)$$

$$\dot{v}_x = a_{D,x} + a_{C,x}, \quad \dot{v}_y = a_{D,y} - g + a_{C,y}, \quad \dot{v}_z = a_{D,z} + a_{C,z}. \quad (2.5)$$

With local gravity enabled, g in the middle equation is replaced by a latitude- and height-dependent value. With Coriolis disabled, all components of a_C are zero. Validation configurations can disable drag or gravity to create analytical comparison problems; normal application calculations include both.

2.2. Deriving the drag vector

The familiar scalar drag magnitude for a reference area A is

$$F_D = \frac{1}{2} \rho C_D A v_a^2, \quad (2.6)$$

where ρ is air density, C_D is a dimensionless drag coefficient, and v_a is airspeed. This equation gives only a nonnegative magnitude. Direction comes from the physical statement that drag opposes relative flow. The unit vector in the flow direction is v_a/v_a , so the force vector is

$$F_D = -F_D \frac{v_a}{v_a} = -\frac{1}{2} \rho C_D A v_a v_a. \quad (2.7)$$

Dividing by projectile mass produces

$$a_D = -\frac{\rho C_D A}{2m} v_a v_a. \quad (2.8)$$

The apparent reduction from v_a^2 to v_a is only notational: the vector v_a contributes the other factor of speed. Its magnitude is indeed

$$\|a_D\| = \frac{\rho C_D A}{2m} v_a^2.$$

When v_a is numerically zero, the implementation returns a zero drag scale instead of dividing by zero.

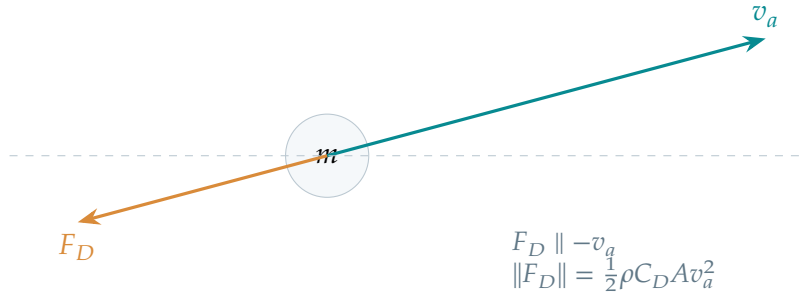


Figure 2.1.: Drag-vector construction. Wind changes v_a , so drag need not be exactly opposite ground-relative velocity.

2.3. Dimensions and similarity parameters

The dimensions in Equation (2.8) provide a useful check:

$$\left[\frac{\rho A}{m} v_a v_a \right] = \frac{(\text{kg/m}^3)(\text{m}^2)}{\text{kg}} (\text{m/s})(\text{m/s}) = \text{m/s}^2.$$

The plus signs here indicate multiplication of displayed dimensional factors. C_D is dimensionless. Reference area is normally the frontal area

$$A = \frac{\pi d^2}{4} \quad (2.9)$$

for diameter d . The tabulated Mach- C_D model uses its explicit reference diameter; the sphere model uses sphere diameter. G1 and G7 reference-BC models instead use the reference projectile's one-inch area and an equivalent reference mass constructed from BC, as developed in Chapter 7.

Two nondimensional flow parameters select aerodynamic behavior:

$$M = \frac{v_a}{a}, \quad (2.10)$$

$$Re = \frac{\rho v_a d}{\mu}, \quad (2.11)$$

where a is local sound speed and μ is dynamic viscosity. Mach number expresses speed relative to acoustic propagation and therefore marks compressibility and transonic behavior. Reynolds number compares inertial and viscous effects around a body. A Mach-only table assumes Reynolds dependence has either been absorbed into the supplied data or is negligible for its intended use. The sphere correlation explicitly depends on both.

2.4. A direct drag-acceleration calculation

The custom Mach- C_D fixture contains a 10 g projectile with reference diameter $d = 7.82$ mm and a knot $C_D = 0.36$ at $M = 1.2$. Under the fixture atmosphere, $\rho = 1.2210893898$ kg/m³ and $a = 340.8756270$ m/s, so the knot speed is

$$v_a = 1.2a = 409.0507525 \text{ m/s.}$$

The frontal area is

$$A = \frac{\pi(0.00782)^2}{4} = 4.8030 \times 10^{-5} \text{ m}^2.$$

Equation (2.8) gives the scalar magnitude

$$a_D = \frac{(1.2210893898)(0.36)(4.8030 \times 10^{-5})}{2(0.010)} (409.0507525)^2 \\ \approx 162.3 \text{ m/s}^2.$$

This large instantaneous value is physically plausible for a small supersonic projectile: it is about sixteen and a half times standard gravitational acceleration, but it acts primarily opposite the high forward speed and falls as the projectile slows and the drag coefficient changes.

Computational implementation

The derivative routine does not integrate a precomputed drag table in distance. At every Runge-Kutta stage it recomputes local atmosphere, constructs v_a , evaluates M and possibly Re , obtains the active drag coefficient or reference retardation, computes the scalar a_D , and multiplies by $-v_a/v_a$.

2.5. Point-mass assumptions

Equations (2.2) track translational motion only. The projectile has mass and an aerodynamic description but no integrated attitude, angular velocity, yaw, pitch, roll, or moments of inertia. The model therefore omits coning motion, yaw-dependent drag, aerodynamic jump, pitch damping, dynamic stability, and changes in projected area caused by attitude. A fitted ballistic coefficient can absorb some average behavior present in the fitting data, but that does not turn the state into a rotational model.

The empirical spin-drift calculation is deliberately layered on top of the point-mass trajectory. It uses time of flight and a Miller stability estimate to produce a separate lateral correction. It neither feeds back into the integrated state nor changes drag. That separation is mathematically and computationally explicit.

3. Observables derived from the state

Physical interpretation

Most displayed quantities are not additional dynamic variables. They are algebraic or geometric interpretations of an accepted position-velocity state, evaluated with a clearly chosen reference velocity and line.

3.1. Kinetic energy and linear momentum

The program computes kinetic energy from ground speed:

$$E_k = \frac{1}{2}mv_g^2, \quad (3.1)$$

and reports the magnitude of linear momentum

$$p = mv_g. \quad (3.2)$$

These are ground-frame mechanical quantities. Replacing v_g with airspeed would make them change merely because the air mass changed reference velocity, even before the projectile's ground motion responded. Airspeed correctly governs drag; ground speed correctly governs the projectile's translational kinetic state in the local ground frame.

The vector momentum is mv , but the result table reports its magnitude. Similarly, energy carries no direction. Both values use the single-projectile mass stored in the projectile definition.

3.1.1. Worked muzzle calculation

The built-in Hornady A-MAX 168 gr load has

$$m = 0.01088621688 \text{ kg}, \quad v_0 = 822.96 \text{ m/s}.$$

Therefore

$$\begin{aligned} E_{k,0} &= \frac{1}{2}(0.01088621688)(822.96)^2 = 3,686.417 \text{ J}, \\ p_0 &= (0.01088621688)(822.96) = 8.958921 \text{ kg m/s}. \end{aligned}$$

These values agree with the native protocol fixture to the displayed precision. At 100 m in that same calm, zeroed example, the integrated ground speed is 761.0101 m/s, giving 3,152.302 J and 8.284521 kg m/s.

3.2. Per-projectile values and payload totals

A load contains a projectile mass and a payload count. The payload mass helper is

$$m_{\text{payload}} = m \max(1, N), \quad (3.3)$$

where N is pellet count. That total is useful for presentation, but the trajectory solver always receives the physical definition of one projectile. For the built-in nine-pellet 00-buck load, each

nominal lead sphere has mass about 3.49667 g; the load total is nine times that mass only when a payload total is displayed.

If a presentation layer reports total kinetic energy or total momentum by multiplying a per-pellet result by N , it is performing a derived total under the assumption that all pellets share the same modeled speed. The core has not created one object with mass Nm , because doing so would change the area-to-mass ratio, Reynolds number interpretation, and drag deceleration incorrectly.

3.3. Geometric drop

The integrated vertical coordinate y is positive upward from the muzzle-origin plane. The retained trajectory field named drop is

$$d = -y. \quad (3.4)$$

Thus a state below the muzzle-origin horizontal plane has positive drop. A zeroed trajectory can initially rise above that plane because the bore elevation is positive; in that interval $y > 0$ and the stored drop is negative. Drop is not automatically sight-relative and it is not displacement below the line of sight.

This convention explains an otherwise surprising fixture value. At 50 m, the zeroed A-MAX trajectory has $y = 0.0395726$ m and therefore reports $d = -0.0395726$ m. With a 40 mm sight height and horizontal sight line, its path is still slightly negative:

$$q = y - h_s = 0.0395726 - 0.0400 = -0.0004274 \text{ m}.$$

The projectile is above the bore-origin plane but remains about 0.43 mm below the sight line.

3.4. Path and angular holdover

Let the line of sight have height h_s over the muzzle at $x = 0$ and elevation θ_s . Its vertical coordinate is

$$y_s(x) = h_s + x \tan \theta_s. \quad (3.5)$$

The program defines sight-relative path as

$$q(x) = y(x) - h_s - x \tan \theta_s. \quad (3.6)$$

Positive path lies above the line of sight; negative path lies below it. The angular correction required to move from the sight line toward a point displaced by q at horizontal distance x is

$$\alpha_h = \text{atan2}(-q, x), \quad x > 0. \quad (3.7)$$

At the muzzle the implementation returns zero rather than evaluate an undefined viewing angle. For small angles and $x > 0$, $\alpha_h \approx -q/x$, but the two-argument arctangent preserves sign and behaves correctly for larger slopes.

Path and holdover are geometry derived from the integrated trajectory. Changing sight height or sight-line elevation does not change drag or gravity; it changes the reference line against which y is interpreted. Changing bore elevation does change the initial velocity and therefore the trajectory itself.

3.5. Lateral displacement, wind, Coriolis, and spin

The integrated lateral coordinate z receives any lateral acceleration present in Equation (2.5): the lateral component of air-relative drag, optional Coriolis acceleration, and any nonzero initial v_z .

The retained trajectory member and protocol field call this value `windDriftM`, but mathematically it is the total integrated point-mass lateral displacement

$$z(t) = \int_0^t v_z(\tau) \, d\tau. \quad (3.8)$$

In the ordinary zero-azimuth, Coriolis-off case, wind is indeed its only lateral source. When Coriolis is enabled, the present result structure does not separately decompose z into a wind-only part and a Coriolis-only part. Such a decomposition would require companion solves or extra force-accounting state and is not claimed by the current implementation.

Spin drift is different. It is computed after the trajectory sample from time of flight and gyroscopic stability. The protocol returns it in a separate `spinDriftM` field, or null when the empirical calculation is unavailable. It is not added to z by the C++ core. A presentation that wants a combined lateral correction must preserve this distinction and state the chosen combination explicitly.

Model limitation

The labels “wind drift,” “spin drift,” and “Coriolis displacement” name different physical ideas, but the current integrated lateral state stores wind and Coriolis effects together. Only spin drift is separately reported. The program therefore supports an unambiguous total point-mass lateral coordinate and a separate empirical spin term, not a three-way diagnostic force decomposition.

3.6. Time, Mach, Reynolds number, and aerodynamic diagnostics

Time of flight is the independent integration coordinate accumulated to each retained state. Mach and Reynolds number are recomputed from the local airspeed and local atmosphere:

$$M = \frac{\|v - u\|}{a}, \quad Re = \frac{\rho \|v - u\| d}{\mu}.$$

For sphere and tabulated- C_D projectiles, the sample carries physical C_D and Re . For G1 and G7 projectiles, it carries the active reference-curve C_D and Mach, while Reynolds is not asserted. The diagnostic C_D for a reference-BC model belongs to the standard projectile curve; it is not a directly measured coefficient of the actual bullet.

Derived diagnostics are evaluated at the state and atmosphere associated with the sample’s position. In an altitude-dependent atmosphere, two samples at different heights can therefore have different density, sound speed, viscosity, Mach, and Reynolds number even if their airspeeds were equal.

3.7. A compact dependency map

Table 3.1.: Which base quantities determine common outputs.

Output	Direct dependence
Ground speed	v
Airspeed	v and local wind u
Kinetic energy	m and $\ v\ $
Momentum	m and $\ v\ $
magnitude	
Mach	$\ v - u\ $ and local sound speed a
Reynolds number	ρ , μ , reference diameter, and $\ v - u\ $
Drop	$-y$
Sight-relative path	y , x , sight height, and sight-line elevation
Holdover	path and downrange distance
Integrated lateral displacement	z , containing every lateral term integrated in the point-mass state
Empirical spin drift	time of flight, twist, projectile geometry, muzzle velocity, and firing-point density

This map also explains why a result cannot be reconstructed reliably from a reduced subset of displayed columns. For example, airspeed cannot be recovered from scalar ground speed and scalar wind components alone without the full velocity vector, and sight-relative path cannot be recovered from drop without knowing how bore and sight geometry were defined.

The dependencies fall into three computational layers. The primary layer is the integrated state (r, v) and time. The local-environment layer evaluates wind and atmospheric properties at that state. The presentation layer then forms energy, momentum, Mach, Reynolds number, drop, path, and angular correction. Keeping these layers explicit prevents a reporting convention from feeding back into the differential equation by accident.

Reference geometry deserves particular care because it can change an observable without changing the flight state. A different sight height changes path and holdover, but leaves r , v , energy, and Mach untouched. A different wind field changes air-relative velocity inside the drag model and therefore changes the state itself. The distinction separates a change of coordinates from a change of physics.

Part II.

Atmosphere and environmental physics

4. Thermodynamic properties of the air

Physical interpretation

Drag depends on density; Mach-dependent drag depends on sound speed; Reynolds-dependent drag also depends on viscosity. Ballistics Workbench computes these as separate properties of temperature, station pressure, and humidity rather than treating “the atmosphere” as one correction factor.

4.1. Input state and thermodynamic units

The firing-point atmosphere is specified by dry-bulb temperature T_C in degrees Celsius, station pressure P in hectopascals, relative humidity H in percent, and the two signed horizontal wind components. Thermodynamic calculations first use

$$T = T_C + 273.15 \quad (4.1)$$

for absolute temperature and

$$P_{\text{Pa}} = 100P_{\text{hPa}} \quad (4.2)$$

for pressure in pascals.

The accepted input ranges in `Atmosphere::create` are $-60 \leq T_C \leq 60$ degrees Celsius, $500 \leq P \leq 1100$ hPa, and $0 \leq H \leq 100$ percent. Each wind component must have magnitude no greater than 100 m/s. Acceptance is not the same as source qualification. The program separately reports narrower declared domains for the density and sound-speed formulations.

4.2. Water-vapor pressure

Relative humidity is the ratio between actual vapor pressure and the saturation vapor pressure at the same temperature. For the density calculation, the program evaluates the Magnus-type relation

$$e_s(T_C) = 610.94 \exp\left(\frac{17.625T_C}{T_C + 243.04}\right) \text{ Pa}, \quad (4.3)$$

then constructs

$$e = \text{clamp}\left(\frac{H}{100}e_s, 0, 0.99P_{\text{Pa}}\right). \quad (4.4)$$

The upper bound prevents the dry-air partial pressure from becoming zero or negative under an extreme combination of otherwise accepted inputs. Dalton’s law then gives

$$P_d = P_{\text{Pa}} - e, \quad (4.5)$$

where P_d is dry-air partial pressure.

The saturation pressure of Equation (4.3) grows sharply with temperature, as Figure 4.1 shows. This is why humidity has a small effect on cold, dry air and a larger one on warm air: at fixed relative humidity the actual vapor pressure e scales with e_s , and warm air can hold far more moisture.

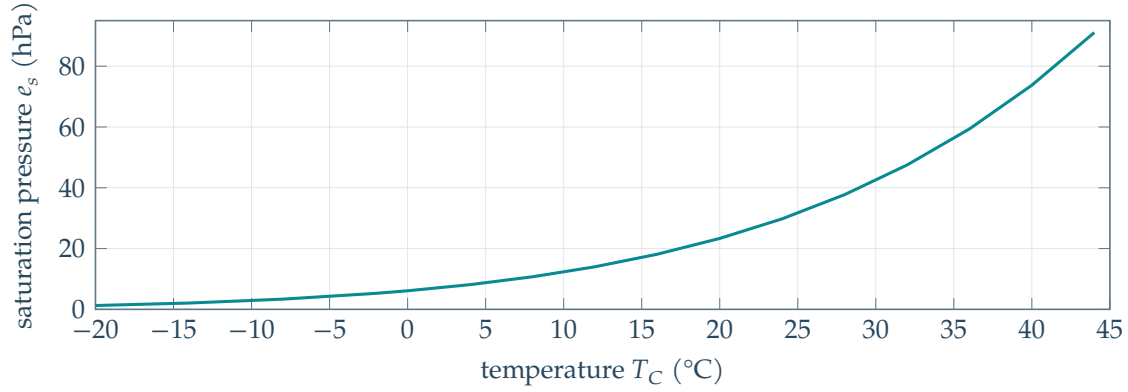


Figure 4.1.: Magnus saturation vapor pressure over the accepted temperature range. The near-exponential rise means that at 40 °C saturated air holds more than ten times the moisture it does at 0 °C.

The sound-speed implementation follows Cramer’s formulation [1] and uses its own published saturation-pressure expression,

$$e_{s,C}(T) = \exp\left(1.2811805 \times 10^{-5}T^2 - 1.9509874 \times 10^{-2}T + 34.04926034 - \frac{6353.6311}{T}\right), \quad (4.6)$$

with T in kelvin. The two expressions serve two separately implemented correlations; I do not silently replace one with the other.

4.3. Moist-air density

Treating the mixture as ideal dry air plus ideal water vapor, each partial density follows $\rho = P/(RT)$. The total is

$$\rho = \frac{P_{Pa} - e}{R_d T} + \frac{e}{R_v T}, \quad (4.7)$$

with

$$R_d = 287.058 \text{ J/(kg K)}, \quad R_v = 461.495 \text{ J/(kg K)}.$$

Water vapor has a larger specific gas constant because its molecular mass is lower than the mean molecular mass of dry air. At fixed total pressure and temperature, replacing part of P_d with e therefore decreases density: $1/R_v < 1/R_d$. Humid air is slightly less dense than dry air under otherwise identical conditions.

Density enters drag acceleration linearly. From Equation (2.8), holding every other factor fixed,

$$\frac{\partial a_D}{\partial \rho} = \frac{a_D}{\rho}. \quad (4.8)$$

A one-percent density increase produces a one-percent increase in instantaneous physical drag magnitude. Over a complete trajectory the output change is not generally one percent, because the increased deceleration changes all later speeds, times, Mach numbers, and drag coefficients.

4.3.1. Worked firing-point atmosphere

The standard protocol fixture uses $T_C = 15^\circ\text{C}$, $P = 1,013.25\text{ hPa}$, and $H = 50$ percent. The native implementation returns

$$\begin{aligned}\rho &= 1.2210893898\text{ kg/m}^3, \\ a &= 340.8756270\text{ m/s}, \\ \mu &= 1.7892858604 \times 10^{-5}\text{ Pa s}.\end{aligned}$$

At zero humidity and the same temperature and pressure, Equation (4.7) gives approximately $1.2249781262\text{ kg/m}^3$. In this example, raising relative humidity from zero to 50 percent lowers the modeled density by about 0.003889 kg/m^3 , or 0.318 percent.

Physical interpretation

Humidity affects more than density. It also changes the Cramer water-vapor mole fraction and therefore the modeled sound speed. Consequently, it can change both the magnitude of drag through ρ and the point at which the projectile meets a Mach-dependent feature in a drag curve.

4.4. Speed of sound: the Cramer calculation

Mach number is defined by local airspeed divided by local sound speed. A constant textbook value for a would move the transonic features of G1, G7, tabulated, and sphere drag as temperature and humidity change. The program instead evaluates the Cramer 1993 polynomial with a fixed carbon-dioxide mole fraction

$$x_c = 400 \times 10^{-6}. \quad (4.9)$$

Cramer's enhancement factor is

$$f = 1.00062 + 3.14 \times 10^{-8} P_{\text{Pa}} + 5.6 \times 10^{-7} T_C^2, \quad (4.10)$$

and the water-vapor mole fraction used by the implementation is

$$x_w = \text{clamp}\left(\frac{H}{100} \frac{f e_{s,C}}{P_{\text{Pa}}}, 0, 0.99\right). \quad (4.11)$$

With temperature in degrees Celsius in the polynomial terms, the calculated speed is

$$\begin{aligned}a &= 331.5024 + 0.603055 T_C - 0.000528 T_C^2 \\ &\quad + (51.471935 + 0.1495874 T_C - 0.000782 T_C^2) x_w \\ &\quad + (-1.82 \times 10^{-7} + 3.73 \times 10^{-8} T_C - 2.93 \times 10^{-10} T_C^2) P_{\text{Pa}} \\ &\quad + (-85.20931 - 0.228525 T_C + 5.91 \times 10^{-5} T_C^2) x_c \\ &\quad - 2.835149 x_w^2 - 2.15 \times 10^{-13} P_{\text{Pa}}^2 + 29.179762 x_c^2 \\ &\quad + 0.000486 x_w P_{\text{Pa}} x_c.\end{aligned} \quad (4.12)$$

The output is in metres per second. The terms represent a fitted thermodynamic correction for temperature, water vapor, pressure, and carbon-dioxide composition. They should be read as one correlation, not as independent forces or additive physical mechanisms that remain valid when separated.

The code declares the sound-speed result source-qualified when $0 \leq T_C \leq 30$ degrees Celsius and $75000 \leq P_{\text{Pa}} \leq 102000\text{ Pa}$. The atmosphere constructor accepts a wider domain, so a calculation may continue outside this interval. In that case the response marks the sound-speed model as extrapolated rather than silently presenting the result as equally supported.

4.4.1. Why sound speed matters downstream

At every derivative evaluation,

$$M = \frac{v_a}{a(T_C, P, H)}.$$

G7 obtains its reference coefficient by interpolating a tabulated function of M . G1 converts Mach back to a standard reference speed before evaluating its piecewise retardation law. A custom Mach- C_D curve is indexed directly by M . The sphere model uses M in its compressibility correction. A change in a can therefore change the selected branch or interpolation interval even if v_a and ρ were artificially held fixed.

For the fixture atmosphere, the A-MAX muzzle airspeed is 822.96 m/s, so

$$M_0 = \frac{822.96}{340.875627} = 2.414253.$$

The exact native response reports the same Mach value. At 100 m, airspeed has decreased to 761.0101 m/s and Mach to 2.232515, while the homogeneous-atmosphere value of a is unchanged.

4.5. Dynamic viscosity and Sutherland's relation

Dynamic viscosity measures the proportionality between shear stress and velocity gradient in a Newtonian fluid. It enters the sphere and tabulated-drag diagnostics through Reynolds number. The program evaluates

$$\mu = \mu_0 \left(\frac{T}{T_0} \right)^{3/2} \frac{T_0 + S}{T + S}, \quad (4.13)$$

using

$$\mu_0 = 1.716 \times 10^{-5} \text{ Pa s}, \quad T_0 = 273.15 \text{ K}, \quad S = 110.333 \text{ K}.$$

Across the application's accepted temperature range, viscosity increases with temperature, as Figure 4.2 shows. Because

$$Re = \frac{\rho v_a d}{\mu},$$

higher μ lowers Reynolds number if density, speed, and diameter are unchanged. In an actual atmosphere change, density and viscosity vary together, so the net Reynolds response is determined by both.

The source-domain flag for viscosity covers the full accepted temperature interval from -60 to 60 degrees Celsius. The checked NACA Report 1135 comparison [3] spans that same interval. The implementation's constant $S = 110.333 \text{ K}$ and reference viscosity are documented in the response model identity `sutherland_110_333_k`.

4.6. Atmospheric model identity and comparison

The response reports three distinct model names:

```
density : ideal_moist_air_mixture,
sound speed : cramer_1993_400_ppm_co2,
viscosity : sutherland_110_333_k.
```

This identity matters because agreement with a reference dataset does not mean the production formula is the same reference formula. For example, the density comparison uses CIPM-2007

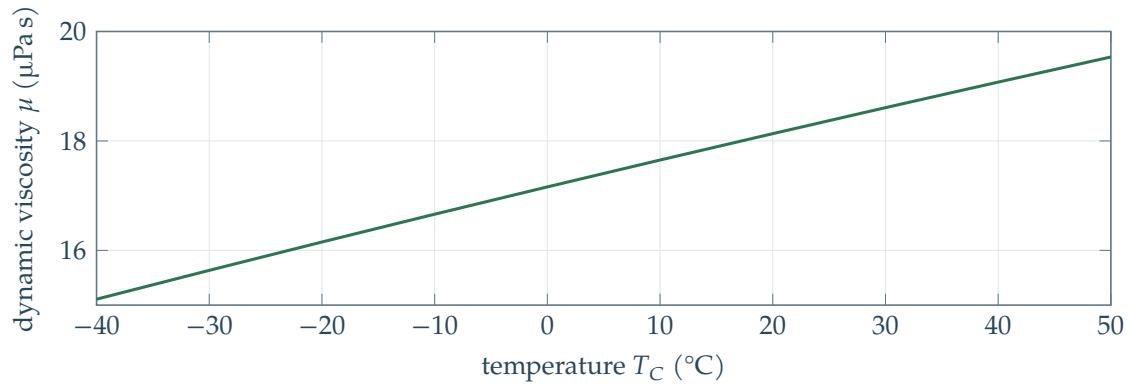


Figure 4.2.: Sutherland dynamic viscosity across the accepted temperature range. Unlike a liquid, air becomes more viscous as it warms because faster molecular motion transfers more momentum across the flow. The change is gentle, near 29 % from -40°C to 50°C .

values [2] to check the simpler ideal-mixture implementation over a stated domain. The current checked report contains nine density cases with a maximum relative residual of 4.5202×10^{-4} , four sound-speed cases with maximum relative residual 3.5061×10^{-4} , and eight viscosity cases with maximum relative residual 3.6071×10^{-4} . Each comparison tolerance is 10^{-3} .

Model limitation

The density model omits the compressibility factor and composition detail of CIPM-2007. Cramer sound speed fixes the carbon-dioxide mole fraction at 400 ppm. Relative humidity is treated as a supplied scalar, not as a transported moisture profile. The atmosphere results are therefore explicit engineering models, not a complete meteorological state equation.

5. Pressure, altitude, and changing atmosphere

Physical interpretation

Station pressure is the pressure physically used in density and sound-speed calculations. Pressure altitude, altimeter setting, and density altitude are conversions or summaries. When altitude-dependent flight is enabled, temperature and pressure are reevaluated from the projectile's vertical position at every derivative stage.

5.1. Station pressure versus altimeter setting

Station pressure is the ambient pressure at the firing location. It belongs directly in Equations (4.7), (4.10), and (4.12). An altimeter setting is a pressure reduced to a reference datum using a standard-atmosphere convention. Substituting an altimeter setting directly as station pressure can overstate density at an elevated site.

The shared conversion constants are

$$\begin{aligned} T_{\text{ref}} &= 288.15 \text{ K}, & P_{\text{ref}} &= 1,013.25 \text{ hPa}, \\ L &= 0.0065 \text{ K/m}, & g_0 &= 9.80665 \text{ m/s}^2, \\ M_d &= 0.0289644 \text{ kg/mol}, & R_u &= 8.31447 \text{ J/(mol K)}. \end{aligned}$$

The pressure exponent is

$$n = \frac{g_0 M_d}{R_u L}. \quad (5.1)$$

With geometric altitude h clamped to $[-1000, 11000]$ m, the standard pressure function is

$$P(h) = P_{\text{ref}} \left(1 - \frac{Lh}{T_{\text{ref}}} \right)^n. \quad (5.2)$$

Its inverse provides pressure altitude:

$$h_p = \frac{T_{\text{ref}}}{L} \left[1 - \left(\frac{P}{P_{\text{ref}}} \right)^{1/n} \right]. \quad (5.3)$$

Pressure is clamped to the values corresponding to the supported altitude interval before the inverse is evaluated.

For altimeter-setting conversion, the code uses a closely related conventional reference temperature $T_A = 288.0 \text{ K}$ and exponent $n_A = 5.2561$:

$$P_{\text{station}} = P_{\text{alt}} \left(\frac{T_A - Lh}{T_A} \right)^{n_A}, \quad (5.4)$$

$$P_{\text{alt}} = P_{\text{station}} \left(\frac{T_A - Lh}{T_A} \right)^{-n_A}. \quad (5.5)$$

These helpers convert representations before the C++ core constructs an `Atmosphere`. Once the atmosphere is constructed, the pressure member is station pressure.

5.2. Density altitude

Density altitude is the standard-atmosphere altitude whose dry-air density matches a calculated density. The shared reference density is

$$\rho_{\text{ref}} = \frac{100P_{\text{ref}}}{R_d T_{\text{ref}}}. \quad (5.6)$$

Combining the standard lapse law with the pressure law gives $\rho/\rho_{\text{ref}} = (1 - Lh/T_{\text{ref}})^{n-1}$ and therefore

$$h_\rho = \frac{T_{\text{ref}}}{L} \left[1 - \left(\frac{\max(0, \rho)}{\rho_{\text{ref}}} \right)^{1/(n-1)} \right]. \quad (5.7)$$

Density altitude is a compact interpretation, not an independent input to drag. The core still uses the actual density, sound speed, and viscosity computed from the supplied temperature, pressure, and humidity. Two atmospheres with the same density altitude can have different sound speeds and therefore different Mach-dependent drag.

5.3. Altitude-dependent flight atmosphere

When altitude dependence is off, every state uses the firing-point temperature, pressure, and humidity, while wind may still vary by layers. When it is on, the projectile's local vertical coordinate y changes temperature according to

$$T_{\text{local}}(y) = T_0 - Ly, \quad (5.8)$$

where T_0 is the supplied firing-point absolute temperature. Pressure is

$$P(y) = P_0 \left(\frac{T_{\text{local}}(y)}{T_0} \right)^n. \quad (5.9)$$

The implementation then calls the ordinary atmosphere constructor with local temperature and pressure, the firing-point relative humidity, and the local wind. Density, Cramer sound speed, and Sutherland viscosity are therefore all reevaluated.

Figure 5.1 shows how the two drag-relevant properties vary with height under the standard lapse. A projectile that climbs several hundred metres over a long shot moves into thinner air with a lower sound speed, which lifts its Mach number even as airspeed falls. Both effects reach the drag model through the reevaluated atmosphere.

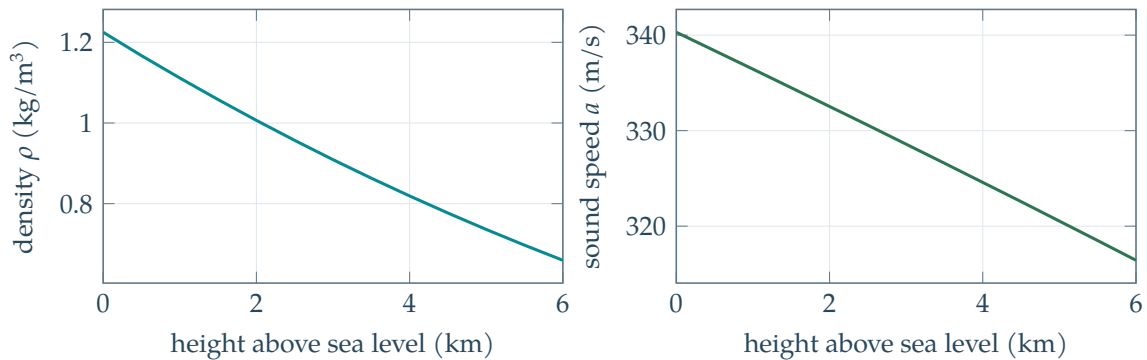


Figure 5.1.: Air density and sound speed against height under the standard dry lapse used by the altitude-dependent model. Density falls about 46 % over the first 6 km; sound speed falls about 7 %, tracking the temperature drop.

The vertical coordinate in Equations (5.8) and (5.9) is height relative to the firing point, not absolute geometric altitude. Firing-point altitude enters the optional local-gravity correction, but the lapse calculation is anchored directly to the supplied firing-point atmosphere. This makes the local profile continuous at $y = 0$ even when the firing-point measurements do not exactly lie on the standard atmosphere.

At an intermediate Dormand–Prince stage, the provisional y value can differ from both endpoints. The derivative routine evaluates the atmosphere at that provisional state, so the integrator responds continuously to changing height. The model is not merely corrected once per output row.

Computational implementation

The sequence inside `Environment::atmosphere_at` is: validate all configured wind layers; apply the lapse law if enabled; select and interpolate the first active wind layer; then call `Atmosphere::create`. Every Runge–Kutta stage therefore receives a self-consistent tuple (ρ, a, μ, w_h, w_c) for its trial position.

Declared limitation. The lapse-rate option is a dry standard relation anchored at the firing point. It excludes measured inversions, arbitrary pressure profiles, height-varying humidity, and weather grids; $T_0 - Ly \leq 0$ K is rejected as nonphysical.

6. Wind, gravity, and Earth rotation

Physical interpretation

Wind changes aerodynamic relative velocity; gravity supplies a vertical acceleration; Coriolis acceleration appears only when the local frame is treated as rotating with the Earth. These effects enter the derivative at different mathematical points and must not be conflated.

6.1. Homogeneous wind

A homogeneous atmosphere stores one headwind and one crosswind value. Equation (1.6) applies those values everywhere. The wind does not directly change $\dot{r}$; position still differentiates to ground-relative velocity. It changes drag by changing v_a , and the accumulated drag response changes v and then position.

Consider a projectile initially parallel to $+x$. A headwind raises $v_{a,x}$ and generally raises drag magnitude, shortening distance traveled during a given time and increasing time to a given distance. A tailwind has $w_h < 0$, reducing closing speed. A positive crosswind makes $v_{a,z} < 0$, so the drag vector has $a_{D,z} > 0$ and gradually carries the projectile rightward. As v_z approaches the wind's $+z$ velocity, the magnitude of the relative lateral component decreases; the projectile does not experience a constant independent lateral acceleration.

6.2. Piecewise-linear wind layers

A wind layer defines an axis, interval, endpoint headwinds, and endpoint crosswinds. The independent coordinate is

$$\xi = \begin{cases} y, & \text{height layer,} \\ x, & \text{downrange layer.} \end{cases} \quad (6.1)$$

For a layer spanning ξ_0 to ξ_1 with endpoint values w_0 and w_1 , the interpolated component is

$$\lambda = \text{clamp}\left(\frac{\xi - \xi_0}{\xi_1 - \xi_0}, 0, 1\right), \quad w(\xi) = w_0 + \lambda(w_1 - w_0). \quad (6.2)$$

The same λ is used independently for headwind and crosswind endpoints.

The implementation scans layers in their supplied order and selects the first layer whose inclusive interval contains the current coordinate. If none contains it, the firing-point homogeneous wind is used. This means layers do not implicitly persist beyond their endpoints, and overlapping layers are order-sensitive: the first containing layer wins. Each layer must have $\xi_1 > \xi_0$ and finite wind components within ± 100 m/s.

6.2.1. Interpolation example

Suppose a downrange layer spans 100 m to 300 m, with headwind increasing from 2 m/s to 10 m/s and crosswind decreasing from 6 m/s to 2 m/s. At $x = 175$ m,

$$\lambda = \frac{175 - 100}{300 - 100} = 0.375.$$

The local components are

$$w_h = 2 + 0.375(10 - 2) = 5 \text{ m/s}, \quad w_c = 6 + 0.375(2 - 6) = 4.5 \text{ m/s}.$$

Those values are used to construct v_a at every Runge–Kutta stage falling at that downrange coordinate.

6.3. Constant and local gravity

The default is the conventional constant $g_0 = 9.80665 \text{ m/s}^2$. With local gravity enabled, the program first evaluates Somigliana normal gravity on the reference ellipsoid. For latitude ϕ ,

$$g_e(\phi) = 9.7803253359 \frac{1 + 0.00193185265241 \sin^2 \phi}{\sqrt{1 - 0.00669437999013 \sin^2 \phi}}. \quad (6.3)$$

A first-order free-air correction is then applied at total height $h_0 + y$:

$$g(\phi, h_0 + y) = g_e(\phi) - 3.086 \times 10^{-6} (h_0 + y). \quad (6.4)$$

h_0 is firing-point geometric altitude and y is the integrated height above the muzzle. Latitude must lie between -90 and 90 degrees.

Latitude matters because the rotating, oblate Earth has different normal gravity at equator and pole. Height matters because gravitational acceleration decreases with distance from the Earth. Over an ordinary small-arms trajectory the correction is small, but the implementation preserves its sign and location dependence explicitly.

The gravity vector in shot coordinates remains vertical:

$$a_g = (0, -g(\phi, h_0 + y), 0).$$

Local gravity does not tilt toward geocentric vertical or follow a curved Earth surface. It is a scalar correction within the fixed local frame.

6.4. Coriolis acceleration in a rotating frame

In a frame rotating with angular velocity Ω , the Coriolis acceleration is

$$a_C = -2\Omega \times v. \quad (6.5)$$

To evaluate it, the program transforms shot-frame velocity into local east-north-up components. Let A be shot azimuth measured from north toward east, and let ϕ be latitude. The transformation is

$$v_E = v_x \sin A + v_z \cos A, \quad (6.6)$$

$$v_N = v_x \cos A - v_z \sin A, \quad (6.7)$$

$$v_U = v_y. \quad (6.8)$$

At $A = 0$, downrange is north and rightward is east. At $A = 90$ degrees, downrange is east and a positive shot-frame z points south, which is consistent with the rotation above.

In local east-north-up coordinates, the Earth's angular velocity is

$$\Omega_{ENU} = [0 \quad \Omega \cos \phi \quad \Omega \sin \phi]^T, \quad \Omega = 7.2921150 \times 10^{-5} \text{ rad/s}. \quad (6.9)$$

Expanding Equation (6.5) gives the components used by the core:

$$a_E = 2\Omega(\sin \phi v_N - \cos \phi v_U), \quad (6.10)$$

$$a_N = -2\Omega \sin \phi v_E, \quad (6.11)$$

$$a_U = 2\Omega \cos \phi v_E. \quad (6.12)$$

Finally, the inverse horizontal rotation returns to shot coordinates:

$$a_{C,x} = a_E \sin A + a_N \cos A, \quad (6.13)$$

$$a_{C,y} = a_U, \quad (6.14)$$

$$a_{C,z} = a_E \cos A - a_N \sin A. \quad (6.15)$$

6.4.1. Following one orientation

For a due-north shot, $A = 0$, a projectile initially has $v_N \approx v_x$, $v_E \approx 0$, and $v_U \approx 0$. Equation (6.10) gives $a_E \approx 2\Omega \sin \phi v_x$. The inverse transform makes $a_{C,z} = a_E$, so at positive northern latitude the initial deflection is to the shooter's right. For a due-east shot, $v_E \approx v_x$; the dominant terms are southward a_N and upward a_U . The shot azimuth therefore affects both lateral and vertical components.

Computational implementation

Coriolis is evaluated from the current ground-relative velocity at every derivative stage. The height argument is presently unused by `coriolis_acceleration`; it is accepted by the interface for environmental consistency. Earth rotation does not alter air-relative velocity directly. Wind affects v_a for drag, while Coriolis uses v in the rotating-ground frame.

6.5. Combined derivative evaluation

With all options active, one call to the derivative routine follows this chain:

1. Read r and v from the six-component trial state.
2. Evaluate local temperature, pressure, humidity-derived properties, and wind at r .
3. Construct $v_a = (v_x + w_h, v_y, v_z - w_c)$ and $v_a = \|v_a\|$.
4. Evaluate the selected aerodynamic retardation a_D .
5. Form $a_D = -a_D v_a / v_a$, with a zero-safe scale.
6. Evaluate either constant or local g .
7. Evaluate a_C from ground velocity, latitude, and shot azimuth if enabled.
8. Return $(v, a_D + (0, -g, 0) + a_C)$.

This ordering is repeated seven times for a normal Dormand–Prince trial step, at different provisional states. Environmental gradients and nonlinear drag therefore participate in the embedded Runge–Kutta estimate rather than being frozen over the entire step.

Model limitation

The environment uses one firing-point thermodynamic state, an optional standard lapse, and at most sixteen linear wind layers. It excludes turbulence, gust histories, vertical wind, terrain flow, gridded meteorology, Earth curvature, and a geodesically rotating frame. Coriolis and local gravity remain local corrections in this geometry.

Part III.

Aerodynamic drag models

7. Reference drag and ballistic coefficient

Physical interpretation

G1 and G7 do not treat the entered ballistic coefficient as a physical drag coefficient. They scale the deceleration of a named standard projectile. The reference curve supplies the speed dependence; BC supplies the effective mass-to-area scaling.

7.1. Direct C_D and reference BC answer different questions

Equation (2.8) is a direct physical- C_D model when C_D belongs to the actual body and A is its stated reference area. A Mach- C_D table and the sphere correlation use this interpretation. Their acceleration magnitude is

$$a_D = \frac{\rho C_D A}{2m} v_a^2. \quad (7.1)$$

A G1 or G7 ballistic coefficient has different semantics. It compares the projectile's deceleration with a standard projectile described by a reference drag function. The numerical BC is meaningful only together with the named family. A value of 0.3 under G1 is not interchangeable with 0.3 under G7, because the two reference projectiles have different $C_D(M)$ curves.

In the conventional reference formulation used by the program, the standard frontal area is the area of a one-inch-diameter circle,

$$A_1 = \frac{\pi}{4} (1 \text{ in})^2, \quad (7.2)$$

and the equivalent reference mass is

$$m_{\text{ref}} = BC (1 \text{ lb}). \quad (7.3)$$

The instantaneous reference-BC acceleration is therefore

$$a_D = \frac{\rho C_{D,\text{ref}}(M) A_1}{2BC (1 \text{ lb})} v_a^2. \quad (7.4)$$

Although Equation (7.3) resembles a mass, it is a computational normalization. The actual projectile mass still determines energy and momentum, and it remains stored independently in the projectile definition.

Physical interpretation

A larger BC gives less deceleration because it appears in the denominator of Equation (7.4). A BC is “effective” whenever it summarizes the projectile's observed velocity loss relative to the chosen standard curve. It is conditional on the reference family, data, atmosphere, velocity span, and any unmodeled behavior absorbed by the fit.

7.2. Density scaling in the reference formulation

Both reference families are evaluated at local Mach but with their tabulated or fitted standard-projectile shape. Current density enters Equation (7.4) directly. For G1, the implementation starts

from a retardation law stated at standard density

$$\rho_{\text{std}} = 1.225 \text{ kg/m}^3$$

and standard sound speed

$$a_{\text{std}} = 340.294 \text{ m/s.}$$

It converts that retardation to an equivalent reference $C_D(M)$, then applies the local density through the common physical form. Algebraically, this makes the local G1 acceleration proportional to ρ/ρ_{std} for a fixed Mach-indexed reference retardation and BC.

The conversion constant in the core is

$$k_{C_D, \text{ref}} = \frac{\rho_{\text{std}} A_1}{2(1 \text{ lb})}. \quad (7.5)$$

If R_{G1} is standard G1 retardation in metres per second squared for $BC = 1$, then

$$C_{D, G1}(M) = \frac{R_{G1}(M)}{k_{C_D, \text{ref}} (Ma_{\text{std}})^2}. \quad (7.6)$$

This equivalent C_D is useful for a common acceleration path and diagnostics; it remains the G1 standard projectile coefficient rather than the actual projectile's direct C_D .

7.3. Constant and velocity-banded BC

A reference drag definition contains exactly one of two coefficient representations. A constant definition supplies one value BC_0 for every airspeed. A banded definition supplies ordered pairs

$$(u_0, BC_0), (u_1, BC_1), \dots, (u_{K-1}, BC_{K-1}),$$

where $u_0 = 0$ and the thresholds are strictly increasing. The active coefficient is

$$BC(v_a) = BC_j, \quad j = \max\{i : u_i \leq v_a\}. \quad (7.7)$$

Thus each threshold begins a new half-open speed region, except the final region has no upper bound. At a threshold exactly, the new band's value applies. Selection uses airspeed, not ground speed, so a strong wind can move the aerodynamic calculation across a BC threshold without an immediate corresponding change in ground-speed magnitude.

The general projectile validator accepts two to sixteen bands, thresholds from 0 to 2,000 m/s, and coefficients $0 < BC \leq 2$. The calibration operation intentionally fits only two to four fixed-threshold bands, because each additional parameter requires independent velocity information and worsens identifiability.

Computational implementation

The active coefficient is found with an upper-bound search over ascending thresholds. The search returns the first threshold greater than v_a ; the implementation selects the preceding band, or the first band when no preceding element exists. This realizes Equation (7.7) without scanning every band.

7.4. G1 piecewise retardation

The registered G1 source data [7] are a descending set of speed thresholds in feet per second. For the first row whose threshold V_j is not greater than the current standard speed V , the retardation is

$$R_{G1, \text{fps}^2}(V, BC) = \frac{A_j V^{n_j}}{BC}. \quad (7.8)$$

A_j and n_j are the active row's coefficient and exponent. The code converts the result to metres per second squared by multiplying by 0.3048 m/ft. The standard speed supplied to this piecewise law is

$$V = Ma_{\text{std}}(3.280839895 \text{ ft/m}). \quad (7.9)$$

Using Mach to form V keeps the reference curve indexed to the local acoustic regime even though the legacy retardation coefficients are expressed as functions of feet per second.

Table 7.1.: G1 threshold table used by `g1_retardation`. The active row is the first row for which $V \geq V_j$.

V_j (ft/s)	A_j	n_j
4230	$1.477404177730177 \times 10^{-4}$	1.9565
3680	$1.920339268755614 \times 10^{-4}$	1.925
3450	$2.894751026819746 \times 10^{-4}$	1.875
3295	$4.349905111115636 \times 10^{-4}$	1.825
3130	$6.520421871892662 \times 10^{-4}$	1.775
2960	$9.748073985 \times 10^{-4}$	1.725
2830	$1.453721560286 \times 10^{-3}$	1.675
2680	$2.162887202930376 \times 10^{-3}$	1.625
2460	$3.209559783129881 \times 10^{-3}$	1.575
2225	$3.904368218691249 \times 10^{-3}$	1.55
2015	$3.222942271262614 \times 10^{-3}$	1.575
1890	$2.203329542809 \times 10^{-3}$	1.625
1810	$1.511001028891904 \times 10^{-3}$	1.675
1730	$8.609957592468259 \times 10^{-4}$	1.75
1595	$4.086146797305117 \times 10^{-4}$	1.85
1520	$1.954473210037398 \times 10^{-4}$	1.95
1420	$5.431896266462351 \times 10^{-5}$	2.125
1360	$8.847742581674416 \times 10^{-6}$	2.375
1315	$1.456922328720298 \times 10^{-6}$	2.625
1280	$2.419485191895565 \times 10^{-7}$	2.875
1220	$1.657956321067612 \times 10^{-8}$	3.25
1185	$4.745469537371 \times 10^{-10}$	3.75
1150	$1.379746590025088 \times 10^{-11}$	4.25
1100	$4.070157961147882 \times 10^{-13}$	4.75
1060	$2.938236954847331 \times 10^{-14}$	5.125
1025	$1.228597370014 \times 10^{-14}$	5.25
980	$2.916938264100495 \times 10^{-14}$	5.125
945	$3.855099424807451 \times 10^{-13}$	4.75
905	$3.953721585280791 \times 10^{-12}$	4.5
860	$1.051728804013 \times 10^{-10}$	4.0
810	$1.485749033551948 \times 10^{-9}$	3.75
780	$1.041162981722437 \times 10^{-8}$	3.5
750	$1.231911859621178 \times 10^{-7}$	3.0

V_j (ft/s)	A_j	n_j
700	$4.600998799 \times 10^{-7}$	2.875
640	$3.209559783129881 \times 10^{-6}$	2.5
600	$7.566129470914212 \times 10^{-6}$	2.375
550	$1.490005193169 \times 10^{-5}$	2.25
250	$1.241662783 \times 10^{-4}$	1.875
0	$1.477404177730177 \times 10^{-4}$	1.9565

The source array contains a final negative sentinel after the zero row, but normal nonnegative speeds always select one of the physical rows above. At $V \leq 0$, the retardation function returns zero before consulting the table.

Converting the piecewise retardation to an equivalent reference coefficient through Equation (7.6) lets the G1 standard curve be drawn on the same axes as any direct C_D . Figure 7.1 shows that curve over the supersonic and transonic range where small-arms trajectories spend most of their flight.

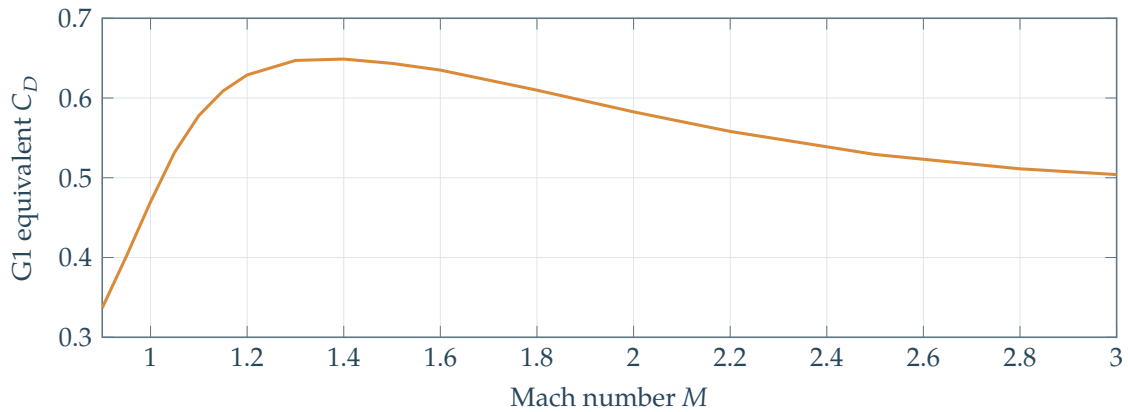


Figure 7.1.: The G1 standard-projectile drag, expressed as an equivalent $C_D(M)$ from the retardation table. Compared with G7 in Figure 7.2, the peak is broader, higher, and sits near $M \approx 1.35$ rather than just above Mach 1. This is the signature of a blunt flat-base reference shape, and it is why a G1 coefficient and a G7 coefficient of equal numerical value describe different deceleration histories.

Each power-law row was chosen as part of the standard G1 approximation. Because the program selects one expression at a hard threshold, the value and derivative are governed by the tabulation rather than smoothed by a spline. The adaptive solver naturally responds to the local change in the right-hand side; its error estimator can reduce the time step near a sharp drag feature.

The G1 implementation does not declare a separate supported Mach interval in the trajectory-validity structure. It applies the piecewise table down to nonnegative speed and uses the highest row above 4,230 ft/s. This numerical continuation does not amount to a claim that one BC remains suitable for every projectile and velocity regime.

7.5. G7 tabulated reference curve

G7 uses the McCoy/BRL standard-projectile coefficient table [8]. For a Mach value between adjacent knots (M_i, C_i) and (M_{i+1}, C_{i+1}) , the reference coefficient is

$$C_{D,G7}(M) = C_i + \frac{M - M_i}{M_{i+1} - M_i} (C_{i+1} - C_i). \quad (7.10)$$

Below the first knot the first value is held; above the final knot the final value is held. Because the table begins at $M = 0$ and ends at $M = 5$, ordinary positive small-arms conditions usually remain within its numerical knot range.

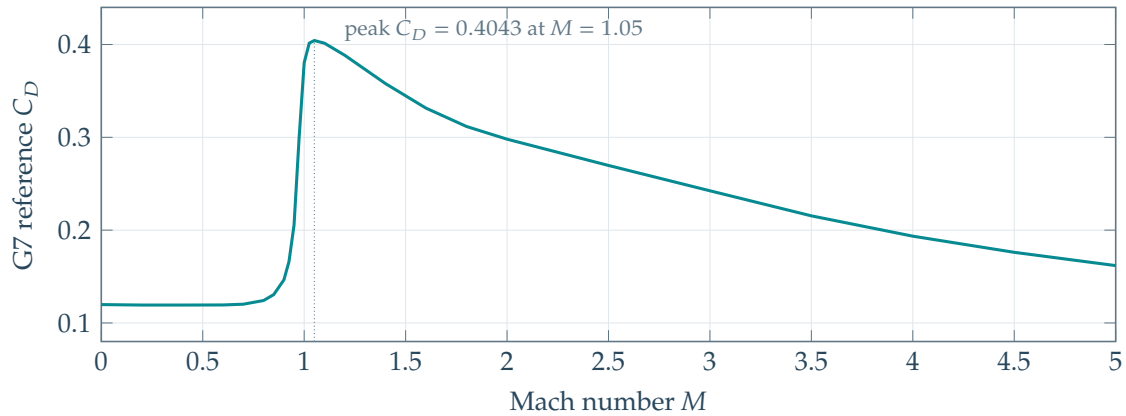


Figure 7.2.: Shape of the G7 reference curve. Drag is nearly flat below $M \approx 0.8$, rises steeply through the transonic region to a sharp peak just above Mach 1, then decays through the supersonic range. The production interpolation uses the full source table, not only the points drawn here.

The transonic region illustrates why a reference curve cannot be replaced by a constant C_D . The table rises from $C_D = 0.1464$ at $M = 0.9$ to 0.3803 at $M = 1.0$ and reaches 0.4043 at $M = 1.05$. It then falls gradually through the supersonic range. A trajectory crossing this interval changes aerodynamic response rapidly even when density and BC remain constant.

Table 7.2.: Complete G7 Mach- C_D knots used by the core, arranged as two pairs per row.

M	C_D	M	C_D
0.000	0.1198	1.400	0.3580
0.050	0.1197	1.500	0.3440
0.100	0.1196	1.550	0.3376
0.150	0.1194	1.600	0.3315
0.200	0.1193	1.650	0.3260
0.250	0.1194	1.700	0.3209
0.300	0.1194	1.750	0.3160
0.350	0.1194	1.800	0.3117
0.400	0.1193	1.850	0.3078
0.450	0.1193	1.900	0.3042
0.500	0.1194	1.950	0.3010
0.550	0.1193	2.000	0.2980
0.600	0.1194	2.050	0.2951
0.650	0.1197	2.100	0.2922
0.700	0.1202	2.150	0.2892
0.725	0.1207	2.200	0.2864
0.750	0.1215	2.250	0.2835
0.775	0.1226	2.300	0.2807
0.800	0.1242	2.350	0.2779
0.825	0.1266	2.400	0.2752
0.850	0.1306	2.450	0.2725
0.875	0.1368	2.500	0.2697
0.900	0.1464	2.550	0.2670

M	C_D	M	C_D
0.925	0.1660	2.600	0.2643
0.950	0.2054	2.650	0.2615
0.975	0.2993	2.700	0.2588
1.000	0.3803	2.750	0.2561
1.025	0.4015	2.800	0.2533
1.050	0.4043	2.850	0.2506
1.075	0.4034	2.900	0.2479
1.100	0.4014	2.950	0.2451
1.125	0.3987	3.000	0.2424
1.150	0.3955	3.100	0.2368
1.200	0.3884	3.200	0.2313
1.250	0.3810	3.300	0.2258
1.300	0.3732	3.400	0.2205
1.350	0.3657	3.500	0.2154
3.600	0.2106	4.400	0.1793
3.700	0.2060	4.600	0.1730
3.800	0.2017	4.800	0.1672
3.900	0.1975	5.000	0.1618
4.000	0.1935		
4.200	0.1861		

The table's source transcription and checksum are recorded in `validation/sources/g7-mccoy.csv` and `validation/manifest.json`. The independent G7 comparison uses a separately transcribed reference curve and fixed-step solver so that a successful comparison is not merely a second call to the production interpolation routine.

7.6. Reference-BC validity and interpretation

The trajectory-validity object declares explicit Mach/Reynolds domains only for tabulated and sphere definitions. G1/G7 return `not_declared` in that structure rather than claiming a universal projectile domain. The reference curves themselves are defined numerically over their implemented speeds, but the suitability of a particular BC for a real projectile is a separate empirical question.

A constant BC assumes one scale factor captures the velocity-loss behavior across the whole trajectory. A banded schedule allows the scale to change when airspeed crosses specified thresholds. Neither representation creates a direct C_D curve for the actual projectile. Calibration can make the effective curve agree closely with observed velocities while still leaving attitude effects, atmosphere errors, or model-form discrepancy embedded in the fitted numbers.

8. Direct Mach– C_D and spherical drag

Physical interpretation

The direct-drag families apply the physical force law to an explicit area and mass. A custom table supplies C_D as a function of Mach; a sphere derives mass from diameter and material density and evaluates a Reynolds-dependent correlation with a compressibility correction.

8.1. Custom Mach– C_D data

A tabulated drag definition contains an actual projectile mass m , a reference diameter d_{ref} , and two to sixty-four strictly increasing knots (M_i, C_i) . The reference area is

$$A = \frac{\pi d_{\text{ref}}^2}{4}.$$

At a Mach value between knots, the program uses piecewise-linear interpolation:

$$C_D(M) = C_i + \eta(C_{i+1} - C_i), \quad \eta = \frac{M - M_i}{M_{i+1} - M_i}. \quad (8.1)$$

Linear interpolation is exact at every knot and cannot overshoot the interval's endpoint coefficients. Its derivative changes discontinuously at a knot, but the coefficient itself is continuous.

For $M \leq M_0$, the program returns C_0 . For $M \geq M_{K-1}$, it returns C_{K-1} . This is *endpoint clamping*, not linear extrapolation. The drag-validity analysis nevertheless marks the result `extrapolated` if the observed Mach interval extends beyond the supplied knot interval, because the physical claim has left the data domain even though the numerical coefficient is held constant.

8.1.1. Interpolation and clamping example

The protocol fixture supplies the knots

$$(1.2, 0.36), \quad (1.6, 0.30).$$

At $M = 1.4$,

$$\eta = \frac{1.4 - 1.2}{1.6 - 1.2} = 0.5, \quad C_D = 0.36 + 0.5(0.30 - 0.36) = 0.33.$$

The full fixture begins at $(0.5, 0.25)$ and ends at $(2.0, 0.27)$. A calculation at $M = 0.3$ therefore uses $C_D = 0.25$; one at $M = 2.4$ uses $C_D = 0.27$. Both values are stable numerical endpoints, and both conditions are reported outside the declared knot domain.

The validator requires $0 \leq M_i \leq 10$, $0 < C_i \leq 5$, and $0.001 \leq d_{\text{ref}} \leq 0.05$ m. Projectile mass must be positive; the general native load validation further restricts muzzle velocity and the surrounding protocol applies its own request bounds.

8.2. Sphere geometry and mass

For a homogeneous sphere of diameter d and material density ρ_m , volume is

$$V = \frac{4}{3}\pi \left(\frac{d}{2}\right)^3 = \frac{\pi d^3}{6}. \quad (8.2)$$

The program therefore derives mass rather than accepting it independently:

$$m = \rho_m \frac{\pi d^3}{6}. \quad (8.3)$$

Validation checks consistency to a relative tolerance of 10^{-9} whenever a sphere reaches the core. The protocol forbids a separate mass for a custom sphere, eliminating contradictory geometry and mass at the request boundary.

The built-in 00-buck definition uses

$$d = 0.330 \text{ in} = 0.008382 \text{ m}, \quad \rho_m = 11,340 \text{ kg/m}^3.$$

Equation (8.3) gives

$$m = 11340 \frac{\pi (0.008382)^3}{6} = 0.003496669991 \text{ kg}.$$

The payload contains nine such nominal pellets, but, as explained in Chapter 3, the flight solve uses this one-pellet mass and area.

8.3. Reynolds-dependent base sphere correlation

The implemented base relation follows the registered Morrison smooth-sphere correlation [10]:

Let $C_{D,0}(Re)$ denote the incompressible or base Reynolds correlation. The implemented function is piecewise:

$$C_{D,0}(Re) = \begin{cases} \frac{24}{Re} + \frac{2.6(Re/5)}{1 + (Re/5)^{1.52}} \\ \quad + \frac{0.411(Re/263000)^{-7.94}}{1 + (Re/263000)^{-8}} + \frac{Re^{0.8}}{461000}, & Re \leq 4.77 \times 10^5, \\ -0.485 + 0.1 \log_{10} Re, & 4.77 \times 10^5 < Re \leq 1.2 \times 10^6, \\ 0.19 - \frac{8 \times 10^4}{Re}, & 1.2 \times 10^6 < Re \leq 2 \times 10^6, \\ 0.15, & Re > 2 \times 10^6. \end{cases} \quad (8.4)$$

The first branch blends low-Re Stokes behavior, intermediate transition terms, the drag crisis, and a high-Re contribution into a smooth empirical expression. The later branches extend the correlation through and beyond the declared upper Reynolds range with explicit formulas.

The input Re must be finite and positive. At the trajectory level,

$$Re = \frac{\rho v_a d}{\mu}.$$

Diameter therefore affects sphere flight three ways at once: frontal area varies as d^2 , derived mass as d^3 , and Reynolds number as d . With constant material density and a comparable C_D , the area-to-mass ratio scales approximately as $1/d$, so a larger sphere decelerates less rapidly.

8.4. Compressibility correction

The final sphere coefficient adds the registered Collins Mach-dependent shock and masking correction [11] to the base Reynolds function. First, Mach is clamped to the declared interval

$$M_c = \text{clamp}(M, 0.2, 1.5). \quad (8.5)$$

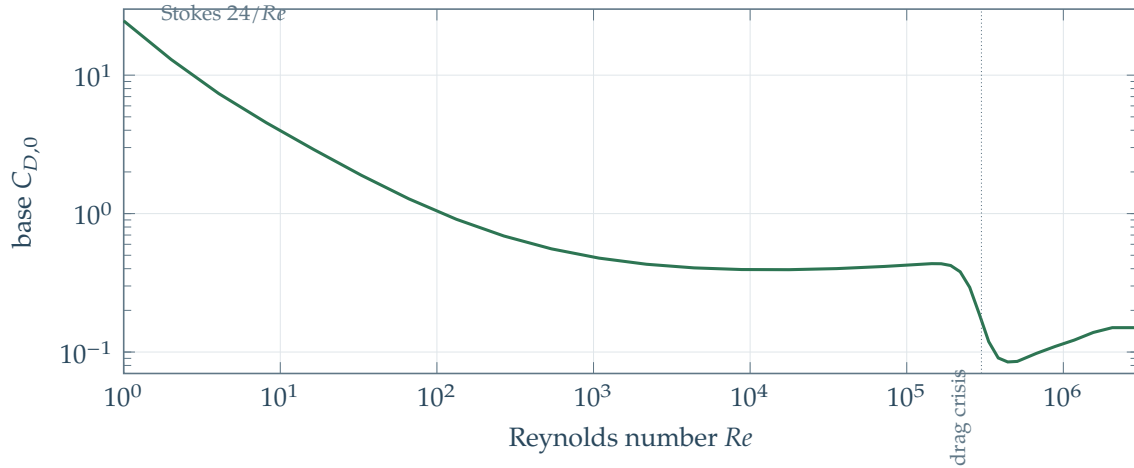


Figure 8.1.: The smooth-sphere base coefficient of Equation (8.4). At low Reynolds number it follows the Stokes $24/Re$ law; through the intermediate range it settles near 0.4; then near $Re \approx 3 \times 10^5$ the boundary layer turns turbulent and drag collapses to roughly 0.08 before recovering. A pellet passes through this range as it slows, so its drag is not a single constant.

The code represents each correction curve as a parametric cubic Bezier segment. For scalar control values $q_0, \dots, q_3$,

$$B(q; u) = q_0(1 - u)^3 + 3q_1u(1 - u)^2 + 3q_2u^2(1 - u) + q_3u^3, \quad 0 \leq u \leq 1. \quad (8.6)$$

Each curve has paired (x, y) control points. To evaluate y at a requested Mach $x = M_c$, the implementation solves $B(x; u) = M_c$ for u and returns $B(y; u)$. It attempts twelve safeguarded Newton iterations using

$$B'(q; u) = 3[(q_1 - q_0)(1 - u)^2 + 2(q_2 - q_1)(1 - u)u + (q_3 - q_2)u^2], \quad (8.7)$$

while maintaining a bracket. It then performs fifty-two bisection iterations as a deterministic full-precision fallback.

The shock curve control points are

$$(0.1, 0), (0.95, 0), (0.55, 0.95), (1.5, 1.0),$$

and the masking curve points are

$$(0, 1.1), (0.85, 1.1), (0.57, 0.05), (1.0, 0).$$

Although the internal control-point x values are not ordered, the resulting parametric $x(u)$ curves used here are treated as monotone; the safeguarded inversion avoids unbounded Newton steps.

Let $S(M_c)$ be the shock curve, with $S = 1$ explicitly at the upper Mach bound, and let $Q(M_c)$ be the mask curve, with $Q = 0$ for $M_c > 1$. The Reynolds scaling is

$$f_R(M_c) = 0.78 + 0.22 \arctan[-12(M_c - 0.23)]. \quad (8.8)$$

The final coefficient is

$$C_D(M, Re) = S(M_c) + Q(M_c)C_{D,0}(\max[f_R(M_c)Re, 10^{-12}]). \quad (8.9)$$

Below the sonic region, the mask retains the Reynolds-dependent base drag. Across the transonic region, the shock term grows and the mask falls. Above Mach 1, the current implementation sets the mask to zero, leaving the shock curve to govern the coefficient up to the clamped upper bound.

Computational implementation

The inner Bezier inversion is a small root-finding problem inside every sphere drag evaluation. Its safeguarded Newton step normally converges quickly; the fixed bisection fallback makes the result deterministic even when the derivative becomes unhelpful. This is separate from trajectory integration and separate again from zeroing root finding.

8.5. Declared sphere domain and behavior outside it

The declared sphere domain is

$$0.2 \leq M \leq 1.5, \quad 100 \leq Re \leq 2.0 \times 10^6. \quad (8.10)$$

The numerical behavior outside it is not identical for both coordinates:

- Mach is explicitly clamped before the compressibility curves are evaluated. Below 0.2 the $M = 0.2$ correction is held; above 1.5 the $M = 1.5$ correction is held.
- Reynolds number is not clamped to 100 or 2×10^6 . The correlation continues through its low-Re formula and uses $C_{D,0} = 0.15$ above 2×10^6 .
- The validity analysis records the observed Mach and Reynolds extrema and returns `extrapolated` if either lies outside the declared domain.

This is why “clamping,” “continuation,” and “validity” must be stated separately. A finite coefficient can always be produced under the numerical rules, but the response does not imply that the empirical correlation has the same support outside Equation (8.10).

8.6. Observed drag validity

For a sphere, the analysis scans retained trajectory samples through the requested display distance and records observed minima and maxima of M and Re . For a tabulated curve it records Mach extrema and compares them with the first and last knots. A result is `within_domain` only when every observed coordinate lies within the declared interval. It is `extrapolated` otherwise. Reference-BC loads receive `not_declared` because no corresponding per-load empirical domain is encoded.

Validity analysis operates on the retained 0.25 m-or-finer trajectory history, not merely the roughly 500 samples serialized for a chart. It is an analysis of the calculated path through aerodynamic parameter space. It does not prove that the chosen curve represents a specific projectile; it states whether the calculation remained within the curve or correlation interval that was supplied or declared.

8.7. Comparing the four aerodynamic families

Before tabulating the semantics, it helps to see the two reference curves on one pair of axes. Figure 8.2 overlays the G1 and G7 standard coefficients. A projectile assigned a G1 BC is compared against the upper curve; one assigned a G7 BC is compared against the lower curve. Because the curves differ by roughly a factor of two through the supersonic range, the same numerical BC placed on the wrong family would misstate deceleration substantially.

Model selection should therefore follow the meaning of available data. A published G1 BC belongs with G1. A G7 BC belongs with G7. A credible direct coefficient table belongs with its

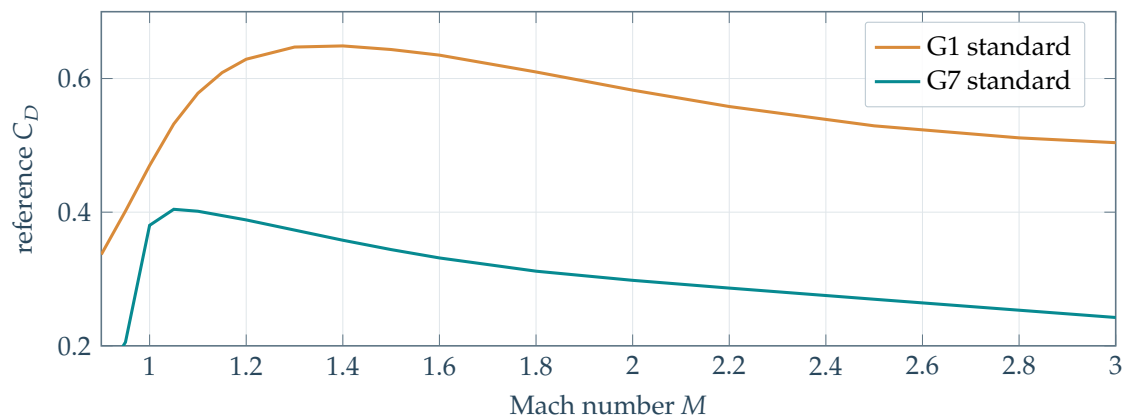


Figure 8.2.: G1 and G7 standard-projectile drag on common axes. The blunt G1 shape carries markedly more supersonic drag than the boat-tailed G7 shape, and the two peaks fall at different Mach numbers. A ballistic coefficient only has meaning together with the family whose curve it scales.

Table 8.1.: Drag-model semantics in Ballistics Workbench.

Family	Speed dependence	Geometry / scaling	Outside encoded data
G1	Piecewise standard retardation, converted through Mach to reference C_D	Conventional BC; constant or airspeed-banded	G1 Piecewise numerical continuation; no per-load explicit validity interval
G7	Linear interpolation of standard-projectile $C_D(M)$	Conventional BC; constant or airspeed-banded	G7 Endpoint values held below $M = 0$ / above $M = 5$; no per-load explicit validity interval
Mach-C_D	Custom knots with piecewise-linear interpolation	Actual mass and explicit reference diameter	Endpoint C_D clamped; validity marked extrapolated outside knots
Sphere	Reynolds base correlation plus Mach correction	Diameter, material density, derived mass, frontal area	Mach clamped; Reynolds correlation continued; validity marked outside declared M/Re domain

actual reference diameter and mass. A nominal sphere belongs with the sphere correlation when its shape and material approximation are defensible. Choosing a family only because its output seems favorable breaks the relationship between parameter and model.

9. Compressible flow around the projectile

Physical interpretation

The drag coefficient is a single number that stands in for the whole flow field around the body. Understanding why the curves of the previous chapters have the shapes they do means looking at that field: the shocks, expansions, and separated wake that form once the projectile approaches and exceeds the speed of sound.

The point-mass model never resolves the air around the projectile. It represents the entire aerodynamic state by one coefficient, $C_D(M)$ or $C_D(M, Re)$, and multiplies it into Equation (2.8). That compression of information is what makes the trajectory solve fast and stable. The shape of every drag curve in this part, though, follows directly from what that one number stands for. The schematics below are adapted from an interactive airflow visualization built alongside the workbench for two reference geometries: a 7.85 mm spitzer boat-tail and a 8.38 mm sphere. They are explanatory reduced-order pictures, not resolved fluid dynamics.

9.1. Streamlined supersonic flow

Figure 9.1 shows the flow around the spitzer boat-tail at roughly its muzzle Mach number. The silhouette is the real validated profile. Three features set the supersonic drag. An attached oblique shock springs from the sharp nose, standing at an angle β steeper than the local Mach angle $\mu = \arcsin(1/M)$; crossing it compresses and slows the air and costs energy, the origin of wave drag. Along the straight bearing surface the flow is nearly parallel. At the boat-tail shoulder the surface turns away from the flow, which expands and accelerates through a fan, recovering some pressure. Behind the flat base the boundary layer separates into a comparatively narrow low-pressure wake, the source of base drag.

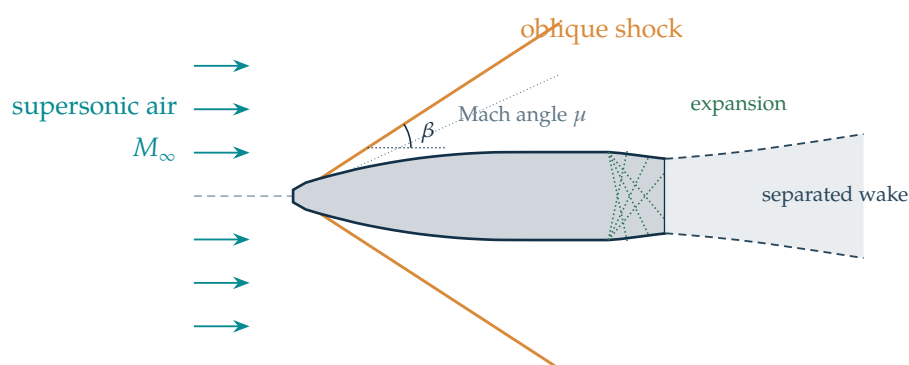


Figure 9.1.: Reduced-order flow around the spitzer boat-tail near $M \approx 2.4$, drawn on its real silhouette. An attached oblique shock (amber) leaves the nose at angle β , steeper than the Mach angle μ . The boat-tail shoulder sets off an expansion fan (green), and the base leaves a narrow separated wake. The streamlined shape keeps both the shock and the wake small, which is why its supersonic C_D is low.

The oblique shock angle is not free. For a given nose deflection θ , it follows the classical θ - β - M relation, and the interactive visualization solves that relation for the weak attached root. Figure 9.2 plots both the Mach angle and the oblique shock angle for a nose half-angle near the boat-tail's

own. As Mach falls toward the transonic region the shock steepens; below roughly $M = 1.6$ for this deflection the attached solution disappears and the shock detaches, moving forward and becoming a blunt bow shock that raises drag sharply. This is the flow-field counterpart of the transonic drag rise seen in Figures 7.2 and 7.1.

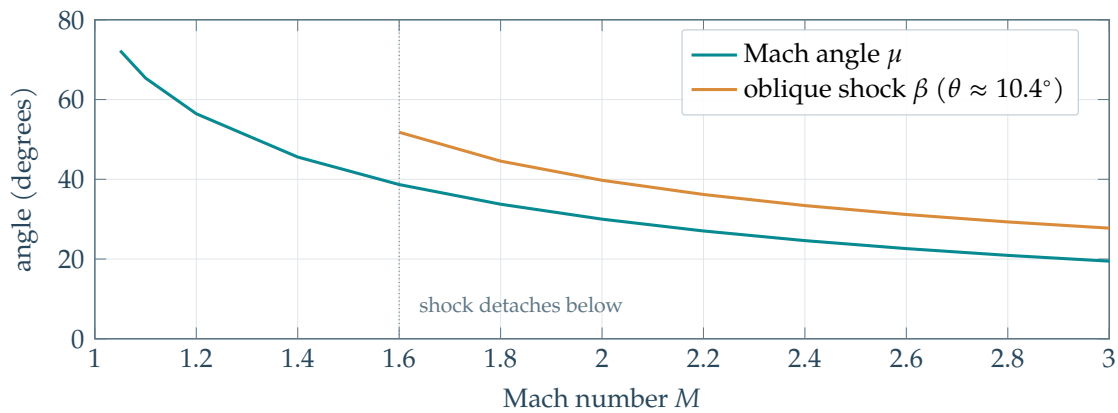


Figure 9.2.: Mach angle and attached oblique shock angle against Mach for a nose deflection near that of the spitzer profile. The shock always stands steeper than the Mach wave, and for this deflection it can no longer stay attached below $M \approx 1.6$.

9.2. Blunt-body flow and the price of a sphere

The sphere tells the opposite story. It has no sharp nose to anchor an attached shock, so above Mach 1 it carries a detached bow shock that stands off ahead of the body, as in Figure 9.3. Behind that shock the flow near the axis is subsonic and stagnates against the front face at high pressure. The boundary layer cannot follow the strongly curved rear surface and separates well before the base, leaving a wide, low-pressure turbulent wake. A strong bow shock and a broad wake together make the sphere's supersonic drag coefficient several times that of the streamlined bullet, and they are why the sphere correlation of Equation (8.9) sits so much higher than either reference curve.

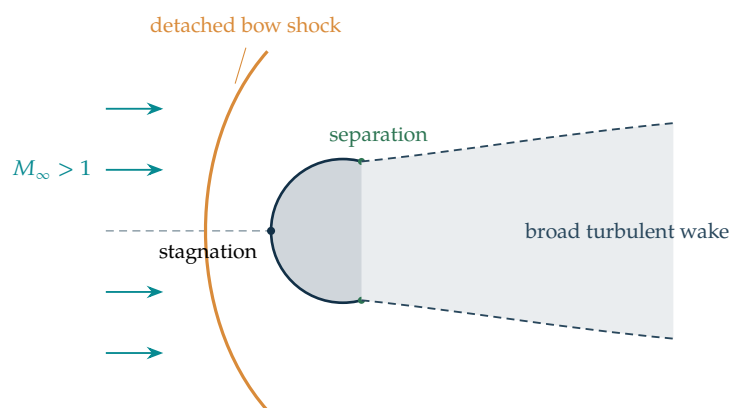


Figure 9.3.: Reduced-order flow around the sphere above Mach 1. With no sharp nose, the shock detaches and stands off as a bow shock, the flow stagnates at high pressure on the front face, and separation occurs early, leaving a wide wake. Both effects are large, which is the physical reason a sphere sheds speed so quickly.

The intensity of that frontal compression grows steeply with speed. Across a normal shock, the kind that sits on the axis of a bow shock, the static pressure and density jump by factors that climb with Mach number, as Figure 9.4 shows. By $M = 3$ the pressure behind the shock is more than

ten times the freestream value. The projectile carries that high-pressure region on its nose, and the work done against it is what the drag coefficient ultimately accounts for.

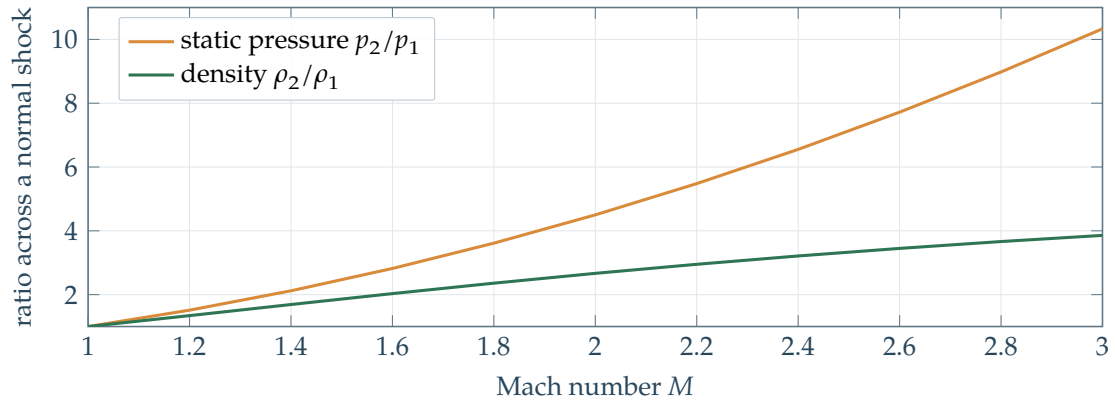


Figure 9.4.: Static pressure and density jump across a normal shock as functions of upstream Mach number. The pressure rise is steep; the density rise saturates toward the strong-shock limit of $(\gamma + 1)/(\gamma - 1) = 6$. A blunt body drives close to this compression on its front face at every point of a supersonic flight.

These pictures do not add anything to the numerical model. The trajectory solver still sees only C_D . They explain where the curves come from: a streamlined body pays a small wave-drag and base-drag penalty, a blunt body pays a large one, and both penalties turn on sharply through the transonic region as the shock system forms and then detaches.

Part IV.

Numerical trajectory solution

10. The trajectory as an initial-value problem

Physical interpretation

The state at the muzzle and the derivative law determine an ordinary differential equation initial-value problem. The production solver advances time because the physical equations are naturally expressed in time and remain regular as long as the state itself remains finite.

10.1. Compact ODE form

Equations (2.2) can be written as

$$\dot{s} = f(t, s; p), \quad s(0) = s_0, \quad (10.1)$$

where p denotes fixed scenario and projectile parameters. The derivative f is nonlinear because drag contains $v_a v_a$, C_D can vary sharply with Mach and Reynolds number, atmosphere and wind can vary with position, and Coriolis acceleration depends on velocity.

The system is ordinarily autonomous: f does not use the absolute time argument directly. Time still serves as the independent variable, and the environment is reached through $r(t)$. A future time-varying wind model could use t without changing the state layout or integration framework.

An initial-value problem differs from a boundary-value problem. The muzzle state is fully known after bore elevation is chosen, and the solver propagates forward. Sight zeroing imposes a boundary condition at a later range, but the core handles it outside the ODE by varying bore elevation and repeatedly solving initial-value problems until the endpoint error vanishes.

10.2. Why time is the production independent variable

A range-domain formulation can use x as the independent coordinate whenever $v_x = dx/dt$ stays positive. By the chain rule,

$$\frac{d}{dx} = \frac{1}{v_x} \frac{d}{dt}. \quad (10.2)$$

For example,

$$\frac{dy}{dx} = \frac{v_y}{v_x}, \quad \frac{dv_x}{dx} = \frac{a_x}{v_x}.$$

Every range derivative becomes singular as $v_x \rightarrow 0$. A steep, slow, or reversing trajectory therefore requires special handling precisely where the physical time-domain state remains ordinary.

The production solver instead integrates

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{z} = v_z, \quad \dot{v} = a.$$

It can detect $v_x = 0$ as an event and terminate at the localized reversal. It can also handle vertical acceleration and changing wind without repeatedly dividing by a small forward component.

Time integration creates a presentation problem: requested results are indexed by downrange distance, but accepted steps end at arbitrary x . The solver resolves that problem with dense cubic Hermite interpolation. This is preferable to reformulating the differential equations around the presentation coordinate.

10.3. The validation-only range solver

The code retains a classical fixed-step RK4 path that integrates over x . Its six-element legacy state is

$$(t, y, z, v_x, v_y, v_z).$$

It uses $\max(v_x, 10^{-6})$ in denominators, applies homogeneous atmosphere and constant gravity, and accepts a fixed distance increment. This solver is useful as a comparison implementation because its independent coordinate and integration algorithm differ from production. It is not used by the desktop's normal calculation path and does not support the full `Environment` object.

Away from the forward-speed floor, its range-domain derivative is obtained directly from Equation (10.2):

$$\frac{d}{dx} \begin{bmatrix} t & y & z & v_x & v_y & v_z \end{bmatrix}^T = \frac{1}{v_x} \begin{bmatrix} 1 & v_y & v_z & a_x & a_y & a_z \end{bmatrix}^T. \quad (10.3)$$

The explicit v_x denominator makes this a deliberately different numerical formulation, not a transcription of the time-domain loop. The 10^{-6} floor prevents an invalid division but also changes the range-domain equation near reversal; comparison cases are therefore chosen where forward velocity stays comfortably positive.

For a smooth solution, fixed-step RK4 has global error $O(\Delta x^4)$. Halving the range step should consequently reduce an asymptotic error measure by about $2^4 = 16$. The validation matrix uses several step sizes and multiple Mach regimes because agreement at one step size or one aerodynamic regime could be accidental.

Model limitation

Agreement between time-domain Dormand–Prince and range-domain RK4 is evidence that two numerical routes solve the declared equations consistently. It does not validate the physical suitability of a drag curve or ballistic coefficient. Both solvers can agree perfectly on the wrong physical inputs.

11. Dormand–Prince embedded Runge–Kutta 5(4)

Physical interpretation

One trial step evaluates the state derivative at seven carefully chosen provisional states. Two different weighted combinations of those derivatives produce fifth- and fourth-order end-point estimates. Their difference estimates local numerical error without requiring a second independent step.

11.1. What a Runge–Kutta stage means

Suppose the accepted state at time t_n is s_n and a trial step has size h . A Runge–Kutta stage is a derivative evaluation

$$k_i = f\left(t_n + c_i h, s_n + h \sum_{j < i} a_{ij} k_j\right). \quad (11.1)$$

The argument is a provisional state constructed from earlier stage slopes. A stage does not mean that the provisional state has been accepted or saved. It samples the vector field within the interval so that a weighted combination can approximate the curved solution.

For this ballistics system, every k_i contains three provisional velocities and three provisional accelerations. Computing one stage can evaluate lapse-rate atmosphere, layered wind, air-relative velocity, Mach, Reynolds number, a drag interpolation or sphere correlation, local gravity, and Coriolis acceleration. A seven-stage step is therefore seven complete local evaluations of the physical derivative.

After the stages, an order- p estimate has the form

$$s_{n+1}^{(p)} = s_n + h \sum_i b_i^{(p)} k_i. \quad (11.2)$$

The coefficients are chosen so that the Taylor expansion matches the exact solution through the required order.

11.2. The implemented tableau

The Dormand–Prince pair [5] used in `src/Ballistics.Core/src/ballistics.cpp` is represented by the following Butcher tableau. The main row $b^{(5)}$ gives the accepted fifth-order estimate; the embedded row $b^{(4)}$ gives the fourth-order comparison.

0								
$\frac{1}{5}$	$\frac{1}{5}$							
$\frac{3}{10}$	$\frac{3}{40}$	$\frac{9}{40}$						
$\frac{4}{5}$	$\frac{44}{45}$	$-\frac{56}{15}$	$\frac{32}{9}$					
$\frac{8}{9}$	$\frac{19372}{6561}$	$-\frac{25360}{2187}$	$\frac{64448}{6561}$	$-\frac{212}{729}$				(11.3)
1	$\frac{9017}{3168}$	$-\frac{355}{33}$	$\frac{46732}{5247}$	$\frac{49}{176}$	$-\frac{5103}{18656}$			
1	$\frac{35}{384}$	0	$\frac{500}{1113}$	$\frac{125}{192}$	$-\frac{2187}{6784}$	$\frac{11}{84}$		
$b^{(5)}$	$\frac{35}{384}$	0	$\frac{500}{1113}$	$\frac{125}{192}$	$-\frac{2187}{6784}$	$\frac{11}{84}$	0	
$b^{(4)}$	$\frac{5179}{57600}$	0	$\frac{7571}{16695}$	$\frac{393}{640}$	$-\frac{92097}{339200}$	$\frac{187}{2100}$	$\frac{1}{40}$	

The code constructs the stages explicitly:

$$\begin{aligned}
k_1 &= f(s_n), \\
k_2 &= f\left(s_n + h\frac{1}{5}k_1\right), \\
k_3 &= f\left(s_n + h\left(\frac{3}{40}k_1 + \frac{9}{40}k_2\right)\right), \\
k_4 &= f\left(s_n + h\left(\frac{44}{45}k_1 - \frac{56}{15}k_2 + \frac{32}{9}k_3\right)\right), \\
k_5 &= f\left(s_n + h\left(\frac{19372}{6561}k_1 - \frac{25360}{2187}k_2 + \frac{64448}{6561}k_3 - \frac{212}{729}k_4\right)\right), \\
k_6 &= f\left(s_n + h\left(\frac{9017}{3168}k_1 - \frac{355}{33}k_2 + \frac{46732}{5247}k_3 + \frac{49}{176}k_4 - \frac{5103}{18656}k_5\right)\right), \\
k_7 &= f\left(s_n + h\left(\frac{35}{384}k_1 + \frac{500}{1113}k_3 + \frac{125}{192}k_4 - \frac{2187}{6784}k_5 + \frac{11}{84}k_6\right)\right).
\end{aligned} \tag{11.4}$$

The fifth-order endpoint is

$$s_{n+1}^{(5)} = s_n + h \left(\frac{35}{384}k_1 + \frac{500}{1113}k_3 + \frac{125}{192}k_4 - \frac{2187}{6784}k_5 + \frac{11}{84}k_6 \right), \tag{11.5}$$

and the fourth-order endpoint is

$$s_{n+1}^{(4)} = s_n + h \left(\frac{5179}{57600}k_1 + \frac{7571}{16695}k_3 + \frac{393}{640}k_4 - \frac{92097}{339200}k_5 + \frac{187}{2100}k_6 + \frac{1}{40}k_7 \right). \tag{11.6}$$

The two estimates use nearly the same information but satisfy different order conditions. Their difference is dominated by the lower-order truncation term and acts as a local error estimate. The program accepts the fifth-order state when the normalized difference is within tolerance.

11.3. Component-wise error normalization

Let $e_j = s_{n+1,j}^{(5)} - s_{n+1,j}^{(4)}$. Each component receives a scale

$$\sigma_j = \varepsilon_{\text{abs},j} + \varepsilon_{\text{rel}} \max(|s_{n,j}|, |s_{n+1,j}^{(5)}|). \tag{11.7}$$

The normalized step error is the maximum norm

$$E = \max_{0 \leq j < 6} \frac{|e_j|}{\sigma_j}. \tag{11.8}$$

Using the maximum means every component must satisfy its own mixed tolerance. A large horizontal coordinate cannot hide an excessive vertical-velocity error through averaging.

Production defaults are

$$\begin{aligned}\varepsilon_{\text{rel}} &= 5 \times 10^{-10}, \\ \varepsilon_{\text{abs,pos}} &= 5 \times 10^{-10} \text{ m}, \\ \varepsilon_{\text{abs,vel}} &= 5 \times 10^{-9} \text{ m/s}.\end{aligned}$$

Absolute tolerance controls components near zero. Relative tolerance scales with the component magnitude. For a downrange coordinate near 1,000 m, the relative part is about 5×10^{-7} m; for a lateral coordinate near zero, the absolute term prevents division by an unusably small scale.

Numerical detail

The tolerances regulate the embedded local estimate, not a guaranteed global physical error. Global error accumulates across accepted steps and depends on smoothness, event localization, and model behavior. The convergence tests therefore compare full trajectories with much tighter reference configurations rather than interpreting a tolerance literally as final-position accuracy.

11.4. Accepting and rejecting a step

If $E > 1$ and h is above the configured minimum, the step is rejected. The state and time remain (s_n, t_n) , and only the trial step changes. The rejection factor is

$$f_{\text{reject}} = \text{clamp}(0.9E^{-1/5}, 0.1, 0.5), \quad h_{\text{new}} = \max(h_{\text{min}}, hf_{\text{reject}}). \quad (11.9)$$

The exponent $-1/5$ is appropriate to the embedded error order used for fifth-order step control. The safety factor 0.9 leaves margin. A rejected trial can at most halve the step and can shrink it by as much as a factor of ten.

If $E \leq 1$, the fifth-order state is accepted and time advances by h . The next-step factor is

$$f_{\text{accept}} = \begin{cases} 5, & E = 0, \\ \text{clamp}(0.9E^{-1/5}, 0.2, 5), & E > 0, \end{cases} \quad (11.10)$$

and

$$h_{\text{new}} = \text{clamp}(hf_{\text{accept}}, h_{\text{min}}, h_{\text{max}}). \quad (11.11)$$

An accepted step can shrink for the next trial if its error was close to one, or grow by as much as a factor of five if the field was smooth.

At exactly $h = h_{\text{min}}$, a finite step is accepted even if $E > 1$, because no smaller permitted time step exists. This behavior prevents an infinite rejection loop. Diagnostics retain the maximum observed normalized error, allowing the extreme case to remain visible.

11.5. Configured time-step and work bounds

The production defaults are

$$\begin{aligned}h_{\text{initial}} &= 1 \times 10^{-3} \text{ s}, & h_{\text{min}} &= 1 \times 10^{-8} \text{ s}, & h_{\text{max}} &= 0.05 \text{ s}, \\ t_{\text{max}} &= 120 \text{ s}, & N_{\text{max}} &= 2,000,000, & v_{x,\text{min}} &= 1 \text{ m/s}.\end{aligned}$$

Before a trial, h is limited so the new time cannot pass $t_{\max}$. Configuration validation requires all tolerances and time limits to be finite and positive, $h_{\min} \leq h_{\max}$, the maximum step count to be nonzero, and angular and geometry values to lie in their encoded domains.

Attempted-step count includes accepted and rejected trials. Accepted and rejected counts are tracked separately. The diagnostics also retain the minimum and maximum accepted h , the final accepted h , and the maximum normalized error seen. This makes an apparently complete trajectory inspectable for excessive rejection or step-size collapse.

11.6. The adaptive loop in the core

Listing 11.1: The production adaptive loop from `integrate_trajectory`. Sample emission and the several termination-event branches are collapsed into one comment; the accept, reject, and step-adaptation logic is shown verbatim.

```
while (!finished)
{
    dt = std::min(dt, configuration.maximum_time_s - time);
    const auto before = state;
    const auto step = dormand_prince_54(before, dt, p, environment,
        configuration.include_aerodynamic_drag, configuration.include_gravity,
        configuration.aerodynamic_drag_multiplier);
    ++trajectory.solver.attempted_steps;

    if (!finite_state(step.fifth_order) || !finite_state(step.fourth_order))
    {
        trajectory.termination = TrajectoryTermination::non_finite_state;
        break;
    }

    const auto normalized_error = error_norm(before, step, configuration);
    if (normalized_error > 1.0 && dt > configuration.minimum_time_step_s)
    {
        ++trajectory.solver.rejected_steps;
        const auto factor = std::clamp(0.9 * std::pow(normalized_error, -0.2), 0.1, 0.5);
        dt = std::max(configuration.minimum_time_step_s, dt * factor);
        continue; // retry the same interval with a smaller step
    }

    state = step.fifth_order; // accept the fifth-order estimate
    time += dt;
    ++trajectory.solver.accepted_steps;

    // ... emit every exact-range sample this step crossed, then check for
    // requested distance, horizontal reversal, ground impact, or slow-down ...

    if (!finished)
    {
        const auto factor = normalized_error == 0.0
            ? 5.0
            : std::clamp(0.9 * std::pow(normalized_error, -0.2), 0.2, 5.0);
        dt = std::clamp(dt * factor, configuration.minimum_time_step_s,
            configuration.maximum_time_step_s);
    }
}
```

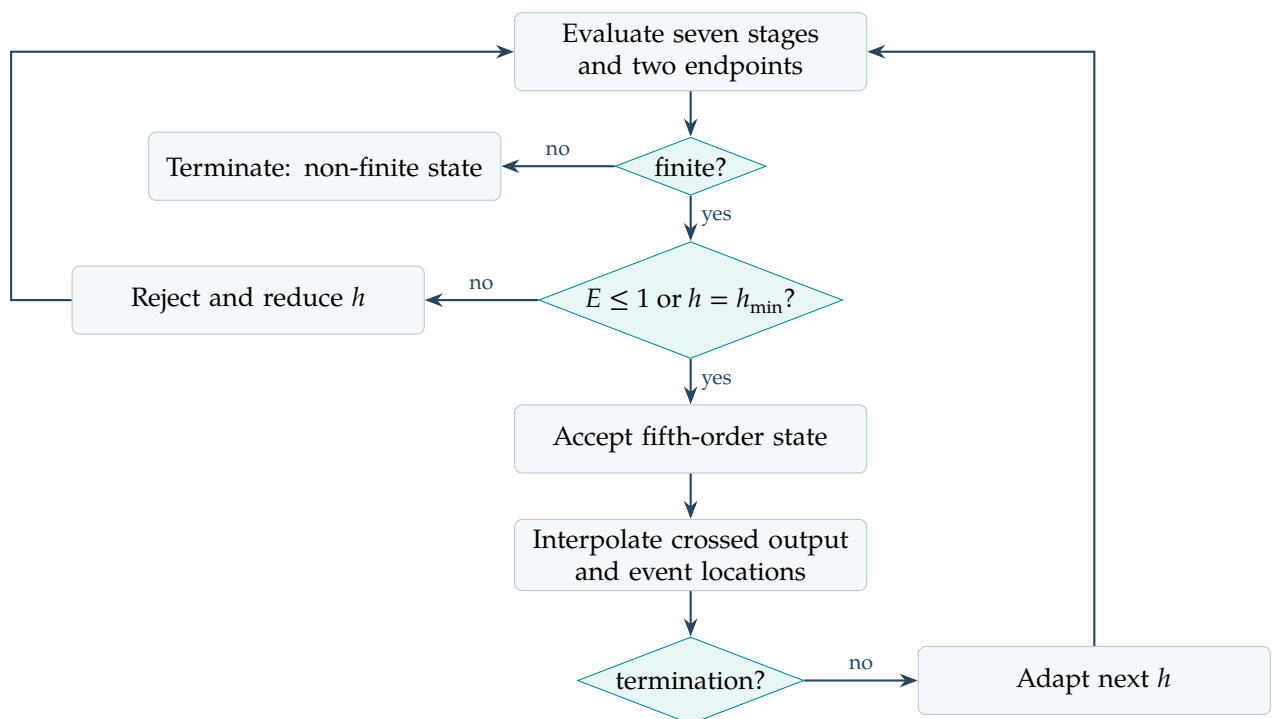


Figure 11.1.: Adaptive step control. Rejection changes only the trial step; acceptance changes the state and enables dense interpolation across the interval.

12. Dense output and numerical termination

Physical interpretation

Accepted time steps and requested range coordinates rarely coincide. Cubic Hermite interpolation uses endpoint positions and endpoint velocities to construct a continuous path through each accepted interval, allowing exact-range sampling and localized termination.

12.1. Cubic Hermite position

Consider one coordinate p over an accepted interval from t_0 to $t_1 = t_0 + h$. The solver knows endpoint positions p_0, p_1 and endpoint velocities v_0, v_1 . With normalized time

$$u = \frac{t - t_0}{h}, \quad 0 \leq u \leq 1,$$

the cubic Hermite interpolant is

$$\begin{aligned} H_p(u) = & (2u^3 - 3u^2 + 1)p_0 + (u^3 - 2u^2 + u)hv_0 \\ & + (-2u^3 + 3u^2)p_1 + (u^3 - u^2)hv_1. \end{aligned} \quad (12.1)$$

At $u = 0$, $H_p = p_0$ and $(1/h) dH_p/du = v_0$. At $u = 1$, $H_p = p_1$ and the time derivative is v_1 . The polynomial therefore matches both endpoint values and slopes.

The corresponding velocity interpolation is the time derivative

$$\begin{aligned} H_v(u) = & \frac{1}{h} \{ (6u^2 - 6u)p_0 + (3u^2 - 4u + 1)hv_0 \\ & + (-6u^2 + 6u)p_1 + (3u^2 - 2u)hv_1 \}. \end{aligned} \quad (12.2)$$

The implementation applies these equations independently to x , y , and z , producing a six-component interpolated state with mutually consistent position and velocity endpoints.

12.2. Finding an exact downrange coordinate

Suppose an accepted step begins with $x_0 < x_*$ and ends with $x_1 \geq x_*$. The program solves

$$H_x(u) - x_* = 0 \quad (12.3)$$

by sixty bisection iterations on $u \in [0, 1]$. Because forward flight makes $x(t)$ monotone within the interval, the sign test is straightforward. Once u_* is found, the complete state is interpolated at the same fraction and sample time is

$$t_* = t_0 + u_*h. \quad (12.4)$$

The saved sample's downrange coordinate is assigned exactly to x_* while its other state components come from the cubic.

Bisection is slower per root than an unguarded Newton iteration but exceptionally robust. Sixty halvings reduce the unit interval below double-precision relevance. The cost is small compared with repeated atmosphere and drag evaluations, and output roots occur only at retained range coordinates.

Physical interpretation

Hermite interpolation is more informative than a straight line because it uses the physical endpoint velocities. In vertical motion, it can represent curvature caused by gravity and drag across the step. It is still an interpolation of a numerically accepted interval, not an extra force evaluation or a new integration step.

12.3. The internal range grid

The requested output interval is a presentation choice, but MPBR and event calculations need a consistently fine trajectory. The solver therefore uses

$$\Delta x_{\text{retain}} = \min(\Delta x_{\text{requested}}, 0.25 \text{ m}). \quad (12.5)$$

Every accepted step emits all retained range coordinates it crosses, using Equation (12.3). A 2,000 m complete trajectory can consequently contain about 8001 retained samples even when the final chart will show far fewer.

At the CLI boundary, the available range is downsampled to roughly 500 points per load. The endpoint at the displayed distance is always added if the stride did not land on it. This reduction affects response size and plotting, not the internal event and validity analysis.

The public `Trajectory::sample_at` method performs linear interpolation between retained samples when the exact requested distance is not already present. It recomputes speed, energy, and momentum from the interpolated velocity. This is a second interpolation layer over a grid no coarser than 0.25 m, distinct from the within-step Hermite dense output that created that grid.

12.4. Finite-state detection

Both embedded endpoints are checked component by component with a finite-number test before their error is evaluated. A NaN or infinity terminates with `non_finite_state`. The normalized error itself is also checked. This ordering prevents a non-finite value from propagating into interpolation, comparisons, or JSON output.

Input validation catches many invalid divisions before integration: positive mass and area, positive sound speed and viscosity from a valid atmosphere, positive BC, ordered knots and bands, and bounded solver parameters. Runtime checks remain necessary because a nonlinear trajectory or extreme model combination can leave a valid initial domain later.

12.5. Termination conditions and their order

The production loop distinguishes the following outcomes.

12.5.1. Requested distance

If an accepted state reaches or passes x_{max} , the solver emits every regular sample through x_{max} , interpolates the exact endpoint if needed, and returns `requested_distance`. This is the normal complete result.

12.5.2. Horizontal reversal

If the accepted endpoint has $v_x \leq 0$, the solver brackets the zero of interpolated $v_x(u)$ and performs sixty bisections. It retains samples only through that event and terminates `horizontal_reversal`. This test precedes ground impact and minimum-forward-speed termination because once x ceases to increase, range-indexed dense output is no longer one-to-one.

12.5.3. Ground impact

When ground termination is enabled and an interval crosses from $y \geq y_g$ to $y < y_g$, the program bisects the interpolated vertical position to $y = y_g$. It emits regular samples through the localized endpoint and returns `ground_impact`. The zeroing trial trajectories disable ground termination so that a sight-line root can be sought without a local horizontal ground plane prematurely ending the solve.

12.5.4. Minimum forward speed

If no earlier event applies and $v_x \leq v_{x,\min}$, the current accepted state is retained and the solver returns `minimum_forward_speed`. The default threshold is 1 m/s. This limit prevents enormous computation after the projectile has effectively stopped progressing through the requested range.

12.5.5. Maximum time and maximum steps

At the top of the loop, the solver stops when attempted steps reach $N_{\max}$ or time reaches $t_{\max}$. If the current state lies beyond the last regular retained sample, it appends that state. A step that lands exactly at the time horizon is also handled after acceptance. These statuses indicate a valid partial trajectory bounded by work or time, not a fabricated endpoint at the requested distance.

12.5.6. Non-finite state

Any non-finite embedded endpoint or error norm ends the solve immediately. No interpolation is attempted across the invalid interval. The covered distance remains the last valid retained sample.

Table 12.1.: Production trajectory termination statuses.

Status	Trigger	Endpoint treatment
requested distance	x reaches configured maximum	Exact x localized by Hermite interpolation
horizontal reversal	v_x crosses zero	Zero of interpolated v_x localized
ground impact	enabled ground height crossed downward	$y = y_g$ localized
minimum forward speed	$v_x \leq v_{x,\min}$ after other event checks	Accepted state retained
maximum time	$t \geq t_{\max}$	Last accepted state retained when it extends coverage
maximum steps	attempted trials reach cap	Last accepted state retained when it extends coverage
non-finite state	endpoint or normalized error is not finite	Invalid interval discarded

12.6. Partial trajectories

A trajectory records both requested and covered distance. Analysis and serialization never assume they are equal. `sample_at` returns no value outside retained coverage. The CLI reports a warning when coverage is shorter than the display distance and names the termination condition. Event families that require a later crossing return horizon-limited statuses instead of substituting the final sample.

This distinction propagates into repeated calculations. A zeroing trial is unavailable if it cannot reach its zero range. A Monte Carlo sample is incomplete if its zeroed trajectory cannot cover the furthest requested output. Calibration returns solver failure if a trial coefficient prevents prediction at an observation. Numerical incompleteness is therefore data in the result, not an exception hidden by a plausible endpoint.

12.7. Analytical constant-drag comparison

The convergence suite includes a horizontal, gravity-free, constant-physical- C_D case. With

$$k = \frac{\rho C_D A}{2m}, \quad (12.6)$$

the equations are

$$\dot{v} = -kv^2, \quad \dot{x} = v.$$

Dividing gives

$$\frac{dv}{dx} = -kv,$$

so

$$v(x) = v_0 e^{-kx}. \quad (12.7)$$

Since $dt/dx = 1/v(x)$,

$$t(x) = \frac{e^{kx} - 1}{kv_0}. \quad (12.8)$$

These exact expressions independently test integration and exact-range interpolation. In the current checked report, maximum time-step refinement from 0.4 s through 0.05 s reduces the speed error at 500 m from about 0.002978 m/s to 8.234×10^{-6} m/s.

The same constant-drag case makes the order of the integrator visible directly. Figure 12.1 plots the terminal speed error of a fixed-step fourth-order Runge–Kutta solve against step size on logarithmic axes. The points fall along a line of slope four: each halving of the step reduces the error by roughly a factor of sixteen, exactly as $O(h^4)$ predicts, until rounding sets a floor near the end.

12.8. Convergence evidence for production settings

The checked adaptive report compares four scenarios against a configuration with tolerances one thousand times tighter and maximum time step twenty-five times smaller. Across those scenarios, the maximum production-to-reference discrepancies are

$$\begin{aligned} \text{position} &: 2.2466 \times 10^{-5} \text{ m}, \\ \text{velocity} &: 3.4805 \times 10^{-4} \text{ m/s}, \\ \text{time} &: 4.5378 \times 10^{-7} \text{ s}. \end{aligned}$$

The corresponding enforced budgets are 5×10^{-4} m, 0.01 m/s, and 1×10^{-5} s. Half-tolerance changes also remain below declared display resolutions in the tested cases.

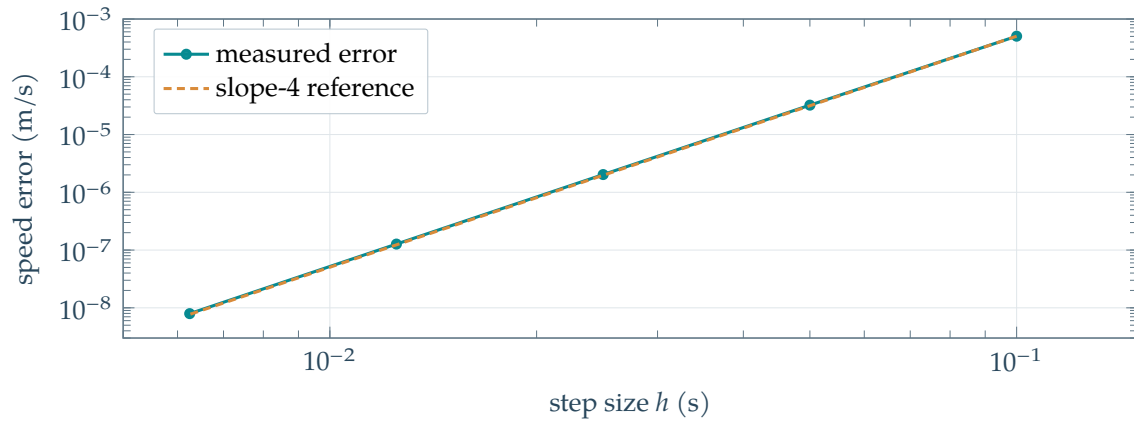


Figure 12.1.: Fourth-order convergence of fixed-step Runge–Kutta on the gravity-free constant-drag case. The measured terminal-speed error tracks the reference line of slope four across five step halvings, confirming the expected $O(h^4)$ behavior of the method independently of the adaptive controller.

These numbers describe numerical convergence for the registered scenarios, compiler, and model version. They do not include uncertainty in BC, atmosphere, projectile dimensions, or the physical adequacy of a point-mass model. Numerical error is only one layer of the complete prediction problem.

Part V.

Sight geometry and trajectory analysis

13. Bore line, sight line, and zeroing

Physical interpretation

A projectile begins on the bore axis at the muzzle, while the sight line begins above it. Zeroing chooses bore elevation so that the integrated projectile position intersects the configured sight line at a specified downrange distance. The angle is found numerically, not by a closed-form parabolic shortcut.

13.1. Three different geometric objects

The *bore line* is the launch direction set by θ_b . The *sight line* is the straight reference line

$$y_s(x) = h_s + x \tan \theta_s.$$

The *trajectory* is the nonlinear curve $y(x)$ produced by integrating the complete force model. At the muzzle, the projectile is at $y(0) = 0$ while the sight line is at h_s , so path begins at

$$q(0) = -h_s. \quad (13.1)$$

With ordinary positive sight height, the projectile starts below the sight line even when its velocity vector points upward relative to it.

Sight-line elevation is supplied in one of two mutually exclusive ways. A target inclination directly supplies θ_s . Alternatively, target elevation Δh at target distance x_T produces

$$\theta_s = \text{atan2}(\Delta h, x_T). \quad (13.2)$$

The protocol rejects simultaneous nonzero inclination and target elevation because they would define competing sight lines. Target elevation also requires a positive display distance.

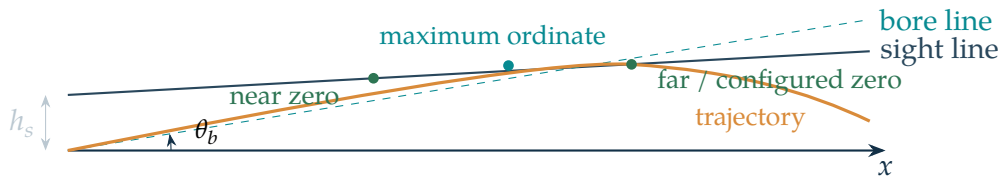


Figure 13.1.: Sight line, bore line, and a representative zeroed trajectory. Zeros are intersections of trajectory and sight line, not intersections with the horizontal axis.

The schematic exaggerates the vertical scale. Figure 13.2 shows the actual sight-relative path for a real engine solve: the built-in A-MAX load zeroed at 250 m, plotted with the vertical axis in millimetres. The projectile starts 40 mm below the sight line, crosses it near 19 m, rises to a maximum of about 120 mm near 138 m, and returns to the line at the configured 250 m zero.

13.2. The zero error function

For a chosen trial bore elevation θ , the program integrates only to the configured zero range x_z with ground termination disabled. If the endpoint exists, the scalar error is

$$F(\theta) = y(x_z; \theta) - h_s - x_z \tan \theta_s. \quad (13.3)$$

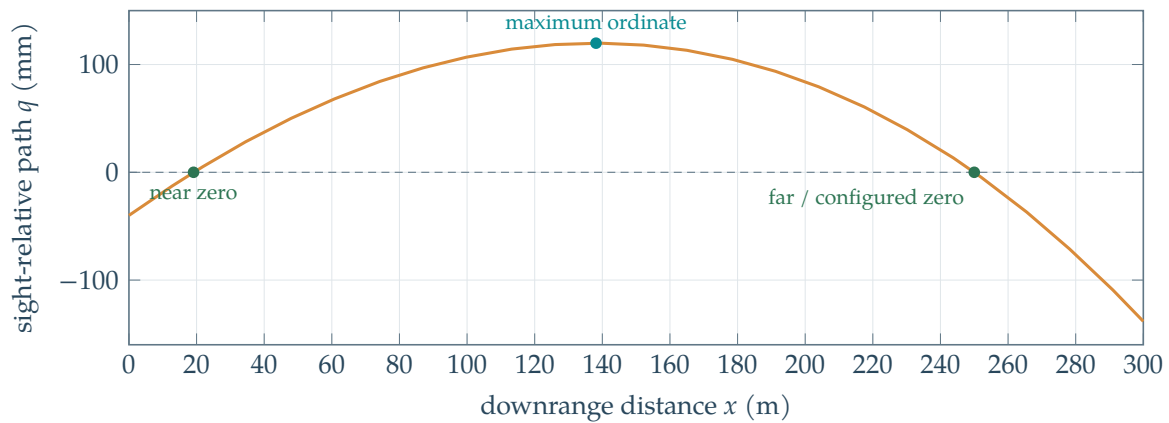


Figure 13.2.: Computed sight-relative path for the built-in A-MAX load at a 250 m zero, from the production solver. The near zero (19.1 m), maximum ordinate (119.8 mm at 138.1 m), and far zero (250 m) are the values returned by the event-analysis pass of Chapter 15, not estimates read off the curve.

The desired bore elevation satisfies

$$F(\theta_b) = 0. \quad (13.4)$$

A negative F means the projectile is below the sight line at the zero range and bore elevation must generally increase. A positive F means it is above and the angle must decrease.

There is no assumption that $y(x)$ is a vacuum parabola. Every evaluation of F invokes the adaptive trajectory solver with the selected atmosphere, wind, gravity, Coriolis setting, and drag representation. Zeroing therefore includes the same nonlinear speed loss and environmental changes as the final trajectory.

13.2.1. A sign-changing illustration

Suppose two trial angles produce

$$F(0.8 \text{ mrad}) = -21 \text{ mm}, \quad F(1.5 \text{ mrad}) = +26 \text{ mm}.$$

Continuity of the solved endpoint with respect to a small launch-angle change indicates a root between them. Bisection first tries 1.15 mrad. If its error is negative, the lower endpoint becomes 1.15 mrad; if positive, the upper endpoint becomes that value. The program uses actual calculated errors rather than the illustrative numbers above, but the sign logic is exactly this.

13.3. Bracketing the root

For a full `ShotScenario`, the initial bracket is

$$\theta_L = \theta_s - 0.05, \quad \theta_H = \theta_s + 0.05 \quad (13.5)$$

in radians. A valid increasing root bracket requires

$$F(\theta_L) \leq 0, \quad F(\theta_H) \geq 0. \quad (13.6)$$

If the signs do not enclose a root, the half-width doubles through at most four expansions. Angles are limited to the numerical interval $[-0.4, 0.4]$ rad. If a trial trajectory cannot reach x_z , zeroing returns `range_unavailable`. If all trial endpoints exist but no sign-changing bracket is established, it returns `no_solution`.

Bracketing is more than a numerical convenience. It prevents an unconstrained angle iteration from moving into a nonphysical or singular launch direction and supplies a clear distinction between “the trajectory could not be evaluated” and “the evaluated interval did not contain a zero.”

13.4. Bisection convergence

Once a bracket is available, the program iterates

$$\theta_M = \frac{\theta_L + \theta_H}{2} \quad (13.7)$$

for at most fifty iterations. It stops early when

$$|F(\theta_M)| \leq 1 \times 10^{-8} \text{ m.} \quad (13.8)$$

If $F(\theta_M) < 0$, the midpoint replaces the lower angle; otherwise it replaces the upper. A fifty-step bisection reduces the original bracket width by 2^{50} , far below any physically meaningful angle resolution in this model.

After the angle is found, the solver integrates a new trajectory to the complete solution horizon. It then samples the configured zero range again and records the final residual. A complete result carries trajectory, bore elevation, zero error, and complete status.

Listing 13.1: Zeroing loop from `integrate_zeroed_trajectory`, condensed to the bracket-and-bisect logic. The repeated status returns for an unreachable zero range are omitted.

```
// Zero-error function: integrate to the zero range at a trial bore angle and
// report how far the projectile lands above (+) or below (-) the sight line.
const auto evaluate = [&](double elevation) -> std::optional<double>
{
    solve_configuration.launch_elevation_rad = elevation;
    const auto candidate = integrate_trajectory(
        p, a, zero_range, multiplier, output_interval, solve_configuration);
    const auto endpoint = candidate.sample_at(zero_range);
    if (!endpoint || candidate.termination != TrajectoryTermination::requested_distance)
        return std::nullopt; // trial could not reach the zero range
    return line_of_sight_path_m(
        *endpoint, sight_height, configuration.sight_line_elevation_rad);
};

// Start with a bracket around the sight-line angle and widen it up to four times
// until the endpoints straddle a sign change (below the line, then above it).
double low = configuration.sight_line_elevation_rad - 0.05;
double high = configuration.sight_line_elevation_rad + 0.05;
auto low_error = evaluate(low);
auto high_error = evaluate(high);
for (int expansion = 0;
     expansion < 4 && (*low_error > 0.0 || *high_error < 0.0); ++expansion)
{
    low = std::max(-0.4, low * 2.0);
    high = std::min(0.4, high * 2.0);
    low_error = evaluate(low);
    high_error = evaluate(high);
}

// Bisect: each pass halves the angular interval, keeps the half that still
// changes sign, and stops once the vertical miss falls below 1e-8 m.
double midpoint = 0.0, midpoint_error = 0.0;
for (int iteration = 0; iteration < 50; ++iteration)
{
    midpoint = (low + high) / 2.0;
    midpoint_error = *evaluate(midpoint);
    if (std::abs(midpoint_error) <= 1e-8) break;
    if (midpoint_error < 0.0) low = midpoint; else high = midpoint;
}
```

13.5. Near zero, far zero, and configured zero

A zero is any positive-range root of $q(x) = 0$. In the common geometry, the projectile begins below the sight line, rises through it at the *near zero*, reaches a positive maximum ordinate, and falls back through it at the *far zero*. When the configured sight-in range lies on the descending branch, the configured zero is the far zero. Other geometries can produce different relationships or no pair within the calculated horizon.

The zeroing routine enforces an intersection at the configured range. The later event-analysis pass independently scans the final trajectory and localizes its near and far crossings. The two operations use different questions and should not be conflated: one chooses a bore angle; the other describes the resulting path.

13.6. Worked native zero

The checked multiple-load fixture uses the built-in Hornady A-MAX 168 gr projectile, a horizontal sight line, sight height 0.04 m, and configured zero 100 m. The core finds

$$\begin{aligned}\theta_b &= 0.00116295702755451 \text{ rad}, \\ F(\theta_b) &= 5.35560838038629 \times 10^{-9} \text{ m}.\end{aligned}$$

The angle is about 1.163 mrad. Event analysis on the 500 m solution horizon then finds near zero at 51.0635 m and far zero at 100.000013 m. The slight difference between the configured 100 m and analyzed far-zero coordinate comes from independent event interpolation and the recorded finite zero residual; it is not a second zeroing target.

14. Maximum point-blank range

Physical interpretation

MPBR is an optimization and crossing problem in sight-relative geometry. The program finds a bore angle whose maximum path reaches the upper radius of a specified vital zone, then finds where the descending path crosses the lower radius.

14.1. Vital-zone radius

The scenario supplies a vital-zone diameter D_v . The allowable vertical radius is

$$R_v = \frac{D_v}{2}. \quad (14.1)$$

A path remains inside the vertical zone while

$$-R_v \leq q(x) \leq R_v. \quad (14.2)$$

The current geometry requires $R_v > 0$, $h_s \geq 0$, and $h_s \leq R_v$. If sight height exceeds the radius, the muzzle already lies below the allowed lower boundary by more than the zone radius, and the native MPBR formulation returns `invalid_geometry`.

14.2. The optimal-bore condition

For a trial bore elevation θ , define the greatest sight-relative ordinate over the solution horizon

$$Q_{\max}(\theta) = \max_{0 \leq x \leq x_H} q(x; \theta). \quad (14.3)$$

The desired bore angle solves

$$G(\theta) = Q_{\max}(\theta) - R_v = 0. \quad (14.4)$$

If $Q_{\max} < R_v$, the trajectory could be raised while remaining inside the upper boundary; if it exceeds R_v , it violates the bound. The tangency-like optimum uses all available vertical allowance without crossing the upper edge.

Each evaluation of G runs a full trajectory to the solution horizon with ground termination disabled. The maximum is taken over the retained 0.25 m-or-finer range grid. Thus MPBR is a nested sequence of complete deterministic integrations, not a transformation of one table row.

14.3. Bracketing and refinement

The scenario implementation begins at

$$\theta_L = \theta_s - 0.01, \quad \theta_H = \theta_s + 0.05. \quad (14.5)$$

If the upper trajectory still has $Q_{\max} < R_v$, the positive half-width doubles through at most four expansions. A useful bracket requires

$$G(\theta_L) \leq 0 \leq G(\theta_H).$$

Failure to enclose the optimal condition is reported as horizon-limited rather than converted into an arbitrary zero.

Bisection then runs for at most forty-five iterations, stopping when

$$|Q_{\max}(\theta) - R_v| \leq 1 \times 10^{-8} \text{ m.} \quad (14.6)$$

The final bore angle is used for one more horizon trajectory. The program identifies the retained sample of greatest path, then searches only after that apex for two descending crossings:

$$q(x_{z,\text{MPBR}}) = 0, \quad (14.7)$$

$$q(x_{\text{MPBR}}) = -R_v. \quad (14.8)$$

Crossing coordinates are linearly interpolated between the bracketing retained samples.

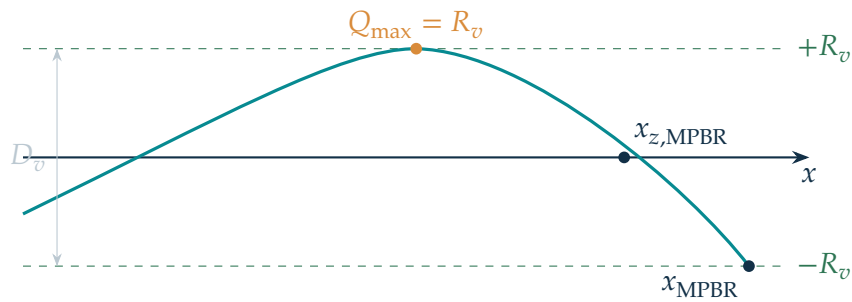


Figure 14.1.: MPBR geometry. The optimized trajectory touches the upper path limit and later crosses the lower limit.

14.4. Complete and horizon-limited results

A `complete` result requires both the post-apex sight-line zero and the later lower-bound crossing. If either lies beyond the supplied trajectory, the status is `horizon_limited`. The program may preserve a found zero while leaving MPBR unavailable, but the protocol publishes numerical zero and MPBR only when the status is complete.

`no_solution` indicates that a required trajectory evaluation itself failed. `invalid_geometry` identifies an input relation such as nonpositive vital zone, insufficient horizon, negative sight height, or sight height greater than radius. These meanings keep numerical work limits separate from geometric impossibility.

14.5. Configured zero versus MPBR zero

The configured sight zero belongs to the normal zeroed trajectory. The MPBR zero belongs to a separately optimized bore elevation. They need not be equal. In the same A-MAX fixture, the firearm sight zero is 100 m, while the native MPBR calculation for a 0.2 m vital-zone diameter returns

$$x_{z,\text{MPBR}} = 233.023764 \text{ m,}$$

$$x_{\text{MPBR}} = 274.059087 \text{ m.}$$

Presenting the MPBR zero as though it were the configured 100 m zero would mix two different bore-angle solutions.

15. Trajectory events and exact geometric analysis

Physical interpretation

Event localization searches the solved trajectory for roots and extrema that usually fall between retained range samples. It is distinct from graph downsampling and distinct from the time-domain interpolation that created the internal range grid.

15.1. Range-domain Hermite path interpolation

For event analysis, the retained independent coordinate is range rather than time. Between samples i and $i + 1$, let

$$\Delta x = x_{i+1} - x_i, \quad (15.1)$$

and let the trajectory slope be

$$y'(x_i) = \frac{v_{y,i}}{v_{x,i}} \quad (15.2)$$

when v_x is safely nonzero. For a reference line $b + mx$, define endpoint relative values

$$r_i = y_i - b - mx_i, \quad r_{i+1} = y_{i+1} - b - mx_{i+1}.$$

The event analyzer uses

$$\begin{aligned} H_r(u) = & (2u^3 - 3u^2 + 1)r_i + (u^3 - 2u^2 + u)\Delta x [y'(x_i) - m] \\ & + (-2u^3 + 3u^2)r_{i+1} + (u^3 - u^2)\Delta x [y'(x_{i+1}) - m]. \end{aligned} \quad (15.3)$$

This is a range-domain cubic whose slopes come from the integrated velocity. Bisection of $H_r(u) = 0$ localizes line crossings. Differentiating H_r and solving $H'_r(u) = 0$ localizes maximum ordinate relative to the sight line.

Time-domain Hermite interpolation and range-domain Hermite event analysis use the same mathematical family for different tasks. The first produces exact-range states inside one accepted time step. The second interprets an already retained trajectory between range samples.

15.2. Near and far sight-line zero

The analyzer computes $q_i = y_i - h_s - x_i \tan \theta_s$. With a value tolerance of 10^{-10} m, it records the first transition from negative to nonnegative path as near zero and the first later transition from positive to nonpositive path as far zero. Each is localized with sixty bisections of Equation (15.3).

Both must be present for `zeroCrossingsStatus=complete`. Otherwise the family is horizon-limited. The code does not label the final trajectory sample as a far zero merely because the configured horizon ended before the actual crossing.

15.3. Maximum ordinate

Maximum ordinate is the maximum of $q(x)$, not the maximum of $y(x)$ unless the sight line is horizontal. Its derivative is

$$q'(x) = \frac{v_y}{v_x} - \tan \theta_s. \quad (15.4)$$

The analyzer looks for a transition from $q' > 0$ to $q' \leq 0$ and bisects the derivative of the range Hermite cubic. It then evaluates the cubic at that fraction to obtain both ordinate and distance. If the initial slope is already no greater than sight-line slope, maximum ordinate is assigned at the muzzle with path $-h_s$.

If no apex occurs within the available trajectory, maximum-ordinate status is horizon-limited. This can happen on an ascending short-horizon path even though every displayed sample is valid.

15.4. Mach crossings and supersonic range

The event thresholds are

$$M \in \{1.2, 1.0, 0.8\}. \quad (15.5)$$

For consecutive sample Mach values M_i and M_{i+1} that bracket a threshold M_* , the crossing fraction is

$$u = \frac{M_i - M_*}{M_i - M_{i+1}}, \quad (15.6)$$

clamped to $[0, 1]$. The crossing range is linearly interpolated. A transition from above to at-or-below is `decelerating`; one from below to at-or-above is `accelerating`. Crossings are sorted by distance and, for equal distances, by descending threshold.

Mach interpolation is linear rather than Hermite because the retained sample does not carry a derivative of Mach with respect to range. The internal grid is no coarser than 0.25 m, limiting but not eliminating interpolation error. Event localization is still independent of the coarser public chart stride.

If muzzle Mach is at most 1, supersonic range is `not_applicable`. Otherwise it is the first decelerating Mach-1 crossing. If no such crossing lies within the trajectory, the status is horizon-limited.

Figure 15.1 traces Mach along the A-MAX flight to 1,500 m. The three event thresholds are marked at the ranges the analyzer reports. Deceleration is steepest early, while airspeed is high, and the curve flattens as the projectile slows, so the spacing between successive crossings widens with range.

15.5. Ground intersection

Event analysis can locate a later intersection with the horizontal muzzle plane even when the production trajectory was not configured to terminate at ground. It searches for $y_i > 10^{-10}$ followed by $y_{i+1} \leq 10^{-10}$ and solves the range Hermite root relative to $y = 0$. Requiring a positive earlier height prevents the muzzle origin itself from being reported as ground intersection.

This analyzed ground crossing is geometric. The integration termination named ground impact uses a configured ground height and time-domain Hermite interpolation. The two results agree in the corresponding horizontal-ground scenario but arise from different layers.

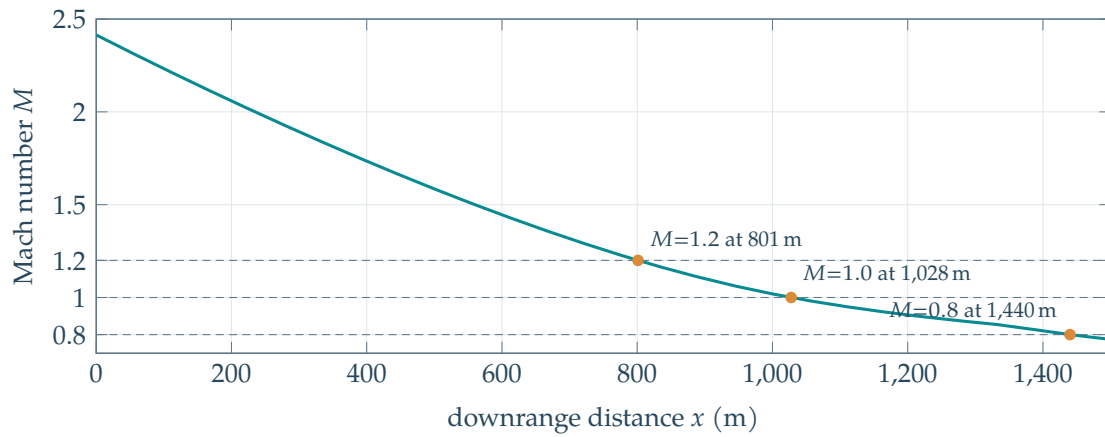


Figure 15.1.: Mach decay for the A-MAX load and the three tracked crossings. Supersonic range is the first decelerating Mach-1 crossing, here 1,028 m. The projectile is transonic across a long stretch of its flight, which is where a reference curve changes most rapidly.

15.6. Worked event set

For the A-MAX fixture described above, the solution horizon is 500 m. Event analysis returns

near zero = 51.0635266 m,
 far zero = 100.0000133 m,
 maximum-ordinate distance = 75.6890771 m,
 maximum-ordinate path = 0.00487704 m,
 ground intersection = 148.4796560 m.

The projectile remains supersonic through the 500 m horizon, so supersonic-range status is horizon-limited and no false endpoint range is supplied.

The localized events satisfy the expected geometric order

$$x_n < x_a < x_f < x_{\text{ground}} < x_{\text{horizon}}.$$

The analyzed far zero differs from the configured 100 m zero by only 13.3×10^{-6} m. That small difference is not a second target: the configured zero comes from time-domain interpolation inside repeated trial integrations, whereas event analysis later locates a root of the retained range-domain Hermite path. Agreement at the micrometre scale is a cross-check between those independent localization layers.

The maximum path is 4.87704 mm, attained between the two sight-line crossings rather than at a public chart point. The ground intersection lies beyond the displayed 100 m range but within the solved horizon. Together, these facts demonstrate why event status must refer to the internal trajectory coverage instead of the visible table extent.

16. Spin stability and empirical spin drift

Physical interpretation

Spin drift is an empirical, separately reported correction. The implementation first estimates Miller gyroscopic stability [12] from bullet geometry, twist, mass, muzzle velocity, and firing-point density, then applies a Litz time-of-flight relation [13] only inside the encoded stability domain.

16.1. Required geometry and units

Spin analysis applies only to a load classified as rifle. It requires bullet diameter d in inches, bullet length L in inches, positive twist rate t in inches per turn, muzzle velocity V_0 , and flight time. The dimensionless twist and length in calibers are

$$t_c = \frac{t}{d}, \quad \ell_c = \frac{L}{d}. \quad (16.1)$$

Projectile mass is converted from kilograms to grains using

$$1 \text{ grain} = 6.479891 \times 10^{-5} \text{ kg}.$$

Missing or nonpositive geometry produces `missing_geometry`; a shotgun load produces `not_applicable`.

16.2. Implemented Miller stability equation

Let m_g be bullet mass in grains and V_f muzzle velocity in feet per second. The base geometric term is

$$S_{g,0} = \frac{30m_g}{t_c^2 d^3 \ell_c (1 + \ell_c^2)}. \quad (16.2)$$

The velocity factor is

$$f_V = \left[\max\left(\frac{V_f}{2800}, 0.05\right) \right]^{1/3}, \quad (16.3)$$

and the density correction is

$$f_\rho = \frac{1.225 \text{ kg/m}^3}{\max(\rho, 0.1 \text{ kg/m}^3)}. \quad (16.4)$$

The implemented gyroscopic stability factor is

$$S_g = S_{g,0} f_V f_\rho. \quad (16.5)$$

Twist appears squared in the denominator. Because twist rate is expressed as inches per turn, a smaller number means faster spin and produces larger S_g . Doubling the twist length from 9 to 18 inches per turn divides the modeled stability by four when other inputs are fixed. Longer bullets increase $\ell_c(1 + \ell_c^2)$ and reduce stability. Lower air density increases the correction factor.

The density is the firing-point atmosphere supplied to the spin function. The calculation does not integrate changing rotational stability along an altitude-dependent trajectory. Muzzle velocity supplies the one-third-power velocity scale; current downrange speed is not substituted at each sample.

16.3. Status thresholds

If S_g or time is non-finite, $S_g \leq 0$, or time is negative, status is `invalid_stability`. For

$$S_g < 1, \quad (16.6)$$

the result is `unstable` and no drift is reported. For

$$S_g > 3.5, \quad (16.7)$$

status is `outside_empirical_domain`. The interval $1 \leq S_g \leq 3.5$ is accepted by the empirical drift relation.

These thresholds do not turn $S_g = 1$ into a guarantee of complete dynamic stability. They encode the domain used by this correction. Suppressing drift outside it is preferable to extending an empirical expression while presenting the result with unchanged confidence.

16.4. Time-of-flight drift relation

Within the domain, drift magnitude in inches is

$$D_{\text{in}} = 1.25(S_g + 1.2)t_f^{1.83}, \quad (16.8)$$

where t_f is time of flight in seconds. The signed metre result is

$$D_s = s_t D_{\text{in}}(0.0254), \quad s_t \in \{-1, +1\}. \quad (16.9)$$

The twist-direction sign controls left or right; any nonnegative sign is treated as positive and a negative sign as negative.

The exponent 1.83 makes drift grow faster than linearly with time. Because time itself depends on the complete drag and atmosphere solution, spin drift remains coupled to trajectory even though it does not feed back into the integrated state.

Model limitation

Equations (16.5) and (16.8) are empirical. They do not integrate spin rate, yaw of repose, overturning moment, aerodynamic jump, coning, or attitude-dependent drag. The returned drift is a correction layered on a point-mass trajectory, not a six-degree-of-freedom lateral solution.

16.5. Relationship to the lateral trajectory

At each serialized point, the core reports integrated z as `windDriftM` and D_s separately as `spinDriftM`; it never adds them. A rightward crosswind and right-hand twist can therefore yield two positive fields, while enabled Coriolis is already inside z . This separation preserves the empirical status: an unavailable stability result leaves the integrated state unchanged, and presentation can show or suppress spin drift without altering the authoritative point-mass trajectory.

17. An end-to-end deterministic example

Physical interpretation

The checked native fixture provides a compact example of the complete chain: current built-in data, moist atmosphere, G1 reference drag, numerical zeroing, adaptive integration, event analysis, and derived output.

17.1. Scenario and projectile

The projectile is the built-in Hornady BLACK .308 Win A-MAX 168 gr load:

$$m = 0.01088621688 \text{ kg}, \quad V_0 = 822.96 \text{ m/s}, \quad BC_{G1} = 0.475.$$

The scenario uses 15 °C, 1,013.25 hPa station pressure, 50 percent relative humidity, no wind, homogeneous atmosphere, constant gravity, a 40 mm sight height, and a horizontal 100 m sight zero. The calculation horizon is 500 m; the displayed table ends at 100 m.

Atmosphere creation produces

$$\rho = 1.2210893898 \text{ kg/m}^3, \quad a = 340.8756270 \text{ m/s}, \quad \mu = 1.7892858604 \times 10^{-5} \text{ Pa s}.$$

Zeroing repeatedly integrates to 100 m and converges to bore elevation 0.0011629570 rad. The final horizon trajectory is then integrated once at that angle.

17.2. Representative states

Table 17.1.: Selected native results from the current protocol fixture.

x (m)	v_g (m/s)	M	t (s)	E_k (J)	q (m)
0	822.9600	2.414253	0	3686.417	-0.0400000
50	791.6360	2.322360	0.0619473	3411.129	-0.0004274
100	761.0101	2.232515	0.1263666	3152.302	5.36×10^{-9}

Figure 17.1 extends this example to the full 500 m calculation horizon. Ground speed falls smoothly from 823 m/s to about 540 m/s, while geometric drop grows past 1.8 m. Both curves come from the same retained trajectory that produced the three table rows.

At the muzzle, path is $-h_s$. At 50 m, the projectile is nearly on the sight line but still below it. At 100 m, the final zero residual is only about 5.36×10^{-9} m. The reference G1 C_D diagnostic rises from 0.536214 at the muzzle to 0.554196 at 100 m as Mach changes along the selected G1 curve.

The solver attempted 97 steps: 47 accepted and 50 rejected. Accepted steps ranged from about 2.746×10^{-4} s to the 0.05 s maximum. The largest recorded error norm was 141.67 because diagnostics include rejected trials; all non-minimum accepted steps satisfied the acceptance rule after adaptation.

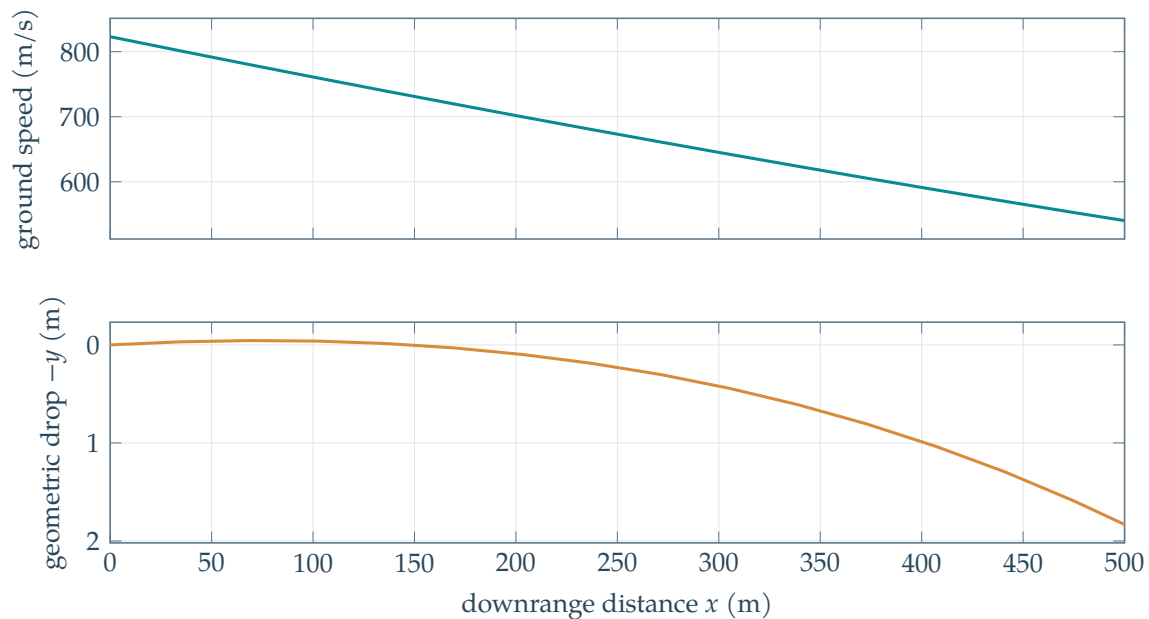


Figure 17.1.: Ground speed and geometric drop for the worked A-MAX fixture over the full 500 m horizon. Drop is drawn with the axis reversed so the trajectory falls downward as it does in flight; it is briefly negative where the zeroed bore angle lifts the projectile above the muzzle plane before gravity dominates.

17.3. Analysis after integration

The event results in Chapter 15 come from the dense internal trajectory, not the three rows above. Near and far zero, maximum ordinate, and ground intersection are localized between 0.25 m-spaced retained states. MPBR is produced by a separate nested bore-angle optimization and therefore has its own zero.

This example also demonstrates the distinction between calculation horizon and display distance. Although only points through 100 m are returned for the displayed range, the core solved through 500 m so it could determine far-reaching event and MPBR availability honestly.

The resulting coverage relationships are

$$\begin{aligned} x_{\text{display}} &= 100 \text{ m}, & x_{\text{ground}} &= 148.479656 \text{ m}, \\ R_{\text{MPBR}} &= 274.059087 \text{ m}, & x_{\text{horizon}} &= 500 \text{ m}. \end{aligned}$$

Thus the public table deliberately stops before two computed geometric results, yet both results are supported by retained internal coverage. If the horizon were shorter than either required event, the corresponding status would become horizon-limited instead of borrowing the display endpoint as a substitute.

This distinction also bounds interpretation of the three-row table. It is sufficient to reproduce muzzle and exact-range observables at 50 m and 100 m; it is not the data structure from which the program derives MPBR, maximum ordinate, or the later ground crossing. Those analyses operate on the dense retained path produced by the same final solve.

Part VI.

Estimation and uncertainty

18. Ballistic-coefficient calibration

Physical interpretation

Calibration estimates effective G1 or G7 coefficients from velocity observations. Every trial parameter set is converted into a projectile definition, solved through the greatest observation range, and compared with measured velocities in a weighted nonlinear least-squares objective.

18.1. Observations and predictions

An observation i contains downrange distance x_i , measured velocity $v_{i,\text{obs}}$, supplied one-standard-deviation uncertainty σ_i , and a role. Distances must be strictly increasing and lie in $(0, 2000]$ m. Velocity must lie from 1 to 2,000 m/s; $0.01 \leq \sigma_i \leq 200$ m/s. A request can contain at most 32 observations.

For trial parameter vector β , the program builds a G1 or G7 projectile, integrates from the muzzle through $\max_i x_i$, and evaluates ground speed at every observation range:

$$v_{i,\text{pred}}(\beta) = \|v(x_i; \beta)\|. \quad (18.1)$$

The normalized residual is

$$r_i(\beta) = \frac{v_{i,\text{pred}}(\beta) - v_{i,\text{obs}}}{\sigma_i}. \quad (18.2)$$

A positive residual means the model retains too much speed at that range; a negative residual means it predicts too little.

Only observations marked `calibration` enter the objective

$$\Phi(\beta) = \sum_{i \in \mathcal{C}} r_i(\beta)^2. \quad (18.3)$$

Holdout observations are predicted and reported after fitting but excluded from Φ , the Jacobian, and the parameter update. At least as many calibration observations as fitted parameters are required, although equality leaves no residual degrees of freedom for a covariance estimate.

18.2. Constant and velocity-banded fits

A constant fit has one parameter. A velocity-banded fit has two to four coefficients associated with fixed, strictly increasing airspeed thresholds beginning at zero. Calibration estimates coefficient values but does not move the thresholds. Fixing thresholds avoids an additional discontinuous parameter-selection problem and makes the Jacobian dimensions stable.

Because the active band is selected from airspeed, a single observation's trajectory can pass through several coefficient regions before reaching x_i . Its residual can therefore depend on more than one fitted coefficient. Observations confined to a narrow speed span may fail to constrain a band that the trajectory scarcely enters.

18.3. Logarithmic parameterization

The optimizer works with

$$\beta_j = \log BC_j, \quad BC_j = e^{\beta_j}. \quad (18.4)$$

This guarantees positive coefficients before explicit bounds and makes multiplicative changes approximately additive. A step $\Delta\beta = 0.01$ changes BC by a factor $e^{0.01} \approx 1.01005$, regardless of the current coefficient scale.

The physical bounds are

$$0.005 \leq BC_j \leq 2.0, \quad (18.5)$$

so the optimizer clamps β_j to $[\log 0.005, \log 2]$. Initial values come from the supplied projectile's effective coefficient at each threshold, then are clamped into this interval.

Physical interpretation

Calibration does not discover a universal material property. It estimates a coefficient that makes the chosen reference drag model reproduce the selected velocity data under the stated atmosphere and muzzle condition. A close fit can coexist with model-form error outside the fitted speeds or environmental conditions.

18.4. Numerical Jacobian

Let n be calibration-observation count and p parameter count. The residual Jacobian is

$$J_{ij} = \frac{\partial r_i}{\partial \beta_j}. \quad (18.6)$$

The program estimates it with central differences. The nominal log-space step is

$$h_\beta = 10^{-4}. \quad (18.7)$$

Near a bound, the higher and lower coordinates are clipped, and the derivative uses the actual span:

$$J_{ij} \approx \frac{r_i(\beta_j^+) - r_i(\beta_j^-)}{\beta_j^+ - \beta_j^-}. \quad (18.8)$$

Each column normally requires two complete trajectory solutions. A p -parameter iteration therefore needs at least $2p$ objective evaluations before any trial update is tested.

Finite differences introduce a scale tradeoff. A step too large measures curvature rather than a local derivative; a step too small suffers cancellation and solver noise. Log space makes 10^{-4} a consistent relative coefficient perturbation, while the production trajectory tolerances keep numerical noise well below the velocity changes in ordinary fits.

18.5. Gauss–Newton model and Levenberg–Marquardt damping

Near the current parameters, residuals are linearized:

$$r(\beta + \delta) \approx r + J\delta. \quad (18.9)$$

Minimizing the squared norm of this approximation yields the Gauss–Newton normal equations

$$J^T J \delta = -J^T r. \quad (18.10)$$

Define

$$N = J^T J, \quad g = J^T r. \quad (18.11)$$

Levenberg–Marquardt [14, 15] adds diagonal damping:

$$(N + \lambda D)\delta = -g, \quad D_{jj} = \max(N_{jj}, 10^{-9}). \quad (18.12)$$

For small λ , the update approaches Gauss–Newton. For large λ , the diagonal term limits steps along poorly scaled or weakly constrained directions.

The initial damping is $\lambda = 10^{-3}$. A small dense linear system is solved with pivoted elimination. If a pivot has magnitude below 10^{-14} or is non-finite, that solve fails and damping is increased by a factor of ten.

18.6. Trial-step acceptance

After solving Equation (18.12), the program forms

$$\beta_{\text{trial}} = \text{clamp}(\beta + \delta, \log 0.005, \log 2) \quad (18.13)$$

component by component and evaluates the complete nonlinear objective. The update is accepted only if

$$\Phi(\beta_{\text{trial}}) < \Phi(\beta). \quad (18.14)$$

On acceptance, λ is divided by three but not below 10^{-12} . On rejection, it is multiplied by ten. Up to eight damping attempts are made from one Jacobian.

If no update is accepted but the largest trial component satisfies

$$\|\delta\|_{\infty} < 10^{-7}, \quad (18.15)$$

the solution is treated as converged at numerical resolution. Otherwise the algorithm continues unless $\lambda > 10^{14}$ or the iteration budget is exhausted.

Additional convergence tests are

$$\|g\|_{\infty} < 10^{-10}, \quad (18.16)$$

$$\|\delta\|_{\infty} < 10^{-7}, \quad (18.17)$$

$$\Phi_{\text{old}} - \Phi_{\text{new}} < 10^{-10}(1 + \Phi_{\text{new}}). \quad (18.18)$$

The default maximum is 30 iterations; the public API accepts 1 through 100.

Listing 18.1: The Levenberg–Marquardt iteration from `calibrate_reference_ballistic_coefficient`, condensed. `evaluate` runs the trajectory solver for a trial parameter set and returns the objective while filling its residual vector; the index loops that accumulate the normal matrix are shown as comments.

```
auto lambda = 1e-3;
for (std::size_t iteration = 0; iteration < maximum_iterations; ++iteration)
{
    // Central-difference Jacobian: two trajectory solves per parameter.
    Matrix jacobian(calibration_count, std::vector<double>(parameters.size()));
    for (std::size_t parameter = 0; parameter < parameters.size(); ++parameter)
    {
        auto higher = parameters, lower = parameters;
        higher[parameter] = std::min(log_maximum, higher[parameter] + finite_difference_step);
        lower[parameter] = std::max(log_minimum, lower[parameter] - finite_difference_step);
        std::vector<double> higher_residuals, lower_residuals;
        evaluate(higher, higher_residuals);
        evaluate(lower, lower_residuals);
        const auto span = higher[parameter] - lower[parameter];
```

```

    for (std::size_t row = 0; row < calibration_count; ++row)
        jacobian[row][parameter] = (higher_residuals[row] - lower_residuals[row]) / span;
}

// Normal equations N = JtJ and gradient g = Jt r (accumulated over rows).
// ... build normal[first][second] and gradient[first] ...
if (std::abs(maximum_gradient) < 1e-10) { converged = true; break; }

for (int attempt = 0; attempt < 8; ++attempt)    // Levenberg-Marquardt damping
{
    auto damped = normal;
    for (std::size_t index = 0; index < damped.size(); ++index)
        damped[index][index] += lambda * std::max(normal[index][index], 1e-9);
    std::vector<double> delta;
    if (!solve_linear_system(damped, negative_gradient, delta))
        { lambda *= 10.0; continue; }           // singular system: damp harder
    auto trial = parameters;
    for (std::size_t index = 0; index < trial.size(); ++index)
        trial[index] = std::clamp(trial[index] + delta[index], log_minimum, log_maximum);
    std::vector<double> trial_residuals;
    const auto trial_objective = evaluate(trial, trial_residuals);
    if (std::isfinite(trial_objective) && trial_objective < objective)
    {
        parameters = std::move(trial);           // accept a strict decrease
        lambda = std::max(lambda / 3.0, 1e-12);
        break;
    }
    lambda *= 10.0;                               // reject: damp harder, try again
}
}

```

18.7. Residual diagnostics

The unweighted calibration root-mean-square error is

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i \in C} (v_{i,\text{pred}} - v_{i,\text{obs}})^2}. \quad (18.19)$$

The weighted RMSE is

$$\text{WRMSE} = \sqrt{\frac{\Phi}{n}}. \quad (18.20)$$

Because residuals are divided by their supplied standard deviations, WRMSE is dimensionless. Reduced chi-square is

$$\chi^2_\nu = \frac{\Phi}{n - p} \quad (18.21)$$

when $n > p$; otherwise the implementation reports zero and later treats covariance information as insufficient. Values substantially above one can indicate underestimated observation uncertainty, lack of fit, or both. Values well below one can indicate generous uncertainty or correlation among observations.

Holdout RMSE uses only holdout observations and is reported separately. A holdout provides evidence about prediction at excluded points; it does not enter the parameter estimate. A value fitted to a manufacturer's own table and evaluated on that same table is calibration or conformance, not independent empirical holdout evidence.

18.8. Covariance approximation and coefficient intervals

At the final parameters, the Jacobian is recomputed. If $n - p > 0$ and $J^T J$ is invertible, the log-parameter covariance approximation is

$$\text{Cov}(\hat{\beta}) \approx s^2 (J^T J)^{-1}, \quad s^2 = \max(1, \chi_\nu^2). \quad (18.22)$$

The lower bound of one prevents an unusually small residual scale from narrowing intervals below those implied by the supplied σ_i . For component j ,

$$\text{SE}(\hat{\beta}_j) = \sqrt{s^2 [(J^T J)^{-1}]_{jj}}. \quad (18.23)$$

The approximate 95 percent interval is transformed back from log space:

$$\left[\exp(\hat{\beta}_j - z_{0.975} \text{SE}), \exp(\hat{\beta}_j + z_{0.975} \text{SE}) \right], \quad z_{0.975} = 1.95996398454, \quad (18.24)$$

then clamped to the BC bounds.

If the normal matrix cannot be inverted or there are no residual degrees of freedom, status becomes `insufficient_information`. A point estimate and residuals may still exist, but confidence limits are withheld.

Model limitation

The covariance interval is a local linear approximation. It assumes the residual weighting is meaningful near the optimum and does not account for uncertain atmosphere, uncertain range, model-form discrepancy, or selection of band thresholds. A narrow interval can describe parameter precision inside the chosen model while the physical model remains biased.

18.9. Current fitted built-ins

The checked fit report reproduces three implemented effective G1 coefficients:

Table 18.1.: Current deterministic coefficient-recovery results.

Load	Calibration rows	Fitted G1 BC	Approximate 95 percent interval / status
White Blackout HV	2	0.0546247165	0.053127 to 0.056165
BlackShock	1	0.0709673761	no interval; zero residual degrees of freedom
Federal SP 150	5	0.3123681448	0.311999 to 0.312738

None of these checked fit definitions contains a holdout. The BlackShock point estimate remains reproducible, but one observation for one parameter supplies no residual variance with which to form Equation (18.24). The other two intervals describe local precision under their supplied weights and fixed G1 model.

These results demonstrate deterministic recovery of the coefficients used by the program. Their evidential meaning remains tied to the source data: a fit to the same publication points is not a separate accuracy test.

19. First-order uncertainty propagation

Physical interpretation

First-order propagation estimates how small independent input variations change each output near one nominal zeroed trajectory. It uses central finite differences and adds variance contributions in quadrature.

19.1. Local linearization

Let $Y = f(X_1, \dots, X_k)$ be one trajectory output at one range. A first-order Taylor expansion around the nominal inputs μ is

$$Y \approx f(\mu) + \sum_{j=1}^k \left. \frac{\partial f}{\partial X_j} \right|_{\mu} (X_j - \mu_j). \quad (19.1)$$

For independent inputs with standard deviations σ_j , covariance cross terms vanish and

$$\sigma_Y^2 \approx \sum_{j=1}^k \left(\left. \frac{\partial Y}{\partial X_j} \right|_{\mu} \sigma_j \right)^2. \quad (19.2)$$

Each squared term is a nonnegative variance contribution. The program accumulates those squares for every requested distance and takes square roots after all active inputs have been processed.

The supported inputs are muzzle velocity, a relative drag parameter, temperature, station pressure, headwind, crosswind, and zero range. Outputs are ground speed, energy, momentum, time, drop, sight path, holdover, and integrated lateral displacement.

19.2. Central finite differences

For a perturbation half-width h_j ,

$$\frac{\partial Y}{\partial X_j} \approx \frac{Y(X_j^+) - Y(X_j^-)}{X_j^+ - X_j^-}. \quad (19.3)$$

The denominator uses the actual span after bounds are applied. This matters near an input boundary, where a symmetric nominal $\mu_j \pm h_j$ pair can be clipped asymmetrically.

The step is normally ten percent of the supplied standard deviation, but each variable has numerical floors and ceilings:

Step floors keep the output difference above avoidable cancellation; ceilings keep the derivative local when the supplied uncertainty is large. The physical standard deviation σ_j , not the finite-difference step, multiplies the estimated derivative in Equation (19.2).

Table 19.1.: First-order finite-difference step selection.

Input	Step h_j	Perturbation bounds
Muzzle velocity	$\text{clamp}(0.1\sigma, 0.05, 1)$ m/s	$0.5V_{\text{nominal}}$ to $1.5V_{\text{nominal}}$
Log drag / log BC	$\text{clamp}(0.1\sigma, 10^{-4}, 0.01)$	multiplicative exponential pair
Temperature	$\text{clamp}(0.1\sigma, 0.01, 0.25)$ °C	−60 to 60 °C
Pressure	$\text{clamp}(0.1\sigma, 0.1, 2)$ hPa	500 to 1100 hPa
Headwind	$\text{clamp}(0.1\sigma, 0.01, 0.25)$ m/s	−100 to 100 m/s
Crosswind	$\text{clamp}(0.1\sigma, 0.01, 0.25)$ m/s	−100 to 100 m/s
Zero range	$\text{clamp}(0.1\sigma, 0.05, 1)$ m	5 to 1000 m

19.3. Drag perturbation semantics

The uncertainty input named relative drag standard deviation is represented as a log-scale perturbation. For a direct physical- C_D or sphere model, the aerodynamic multiplier pair is

$$m_D^+ = e^h, \quad m_D^- = e^{-h}. \quad (19.4)$$

For a reference-BC model, higher BC means lower drag, so the pair is inverted:

$$m_D^+ = e^{-h}, \quad m_D^- = e^h. \quad (19.5)$$

The coordinate span remains $2h$. This makes the reference variable behave like log BC while the derivative routine still receives its generic multiplicative acceleration factor.

19.4. Fixed bore versus re-zeroing

The nominal baseline must be a complete zeroed trajectory. For muzzle velocity, drag, temperature, pressure, headwind, and crosswind perturbations, the program fixes

$$\theta_b = \theta_{b,\text{nominal}}. \quad (19.6)$$

It then integrates perturbed trajectories without ground termination. This asks: if the already-zeroed firearm and sight remain mechanically unchanged while an input varies, how do outputs change?

Zero-range uncertainty has different semantics. A changed requested zero represents a different bore-angle solution, so the higher and lower zero ranges each invoke `integrate_zeroed_trajectory`. This asks how output changes when the intended zero itself changes and the firearm is re-zeroed accordingly.

This distinction has a large effect on path uncertainty. Re-zeroing every muzzle-velocity perturbation would artificially force path uncertainty to zero at the configured range and answer a different question. Holding bore fixed preserves the effect of shot-to-shot velocity variation after a nominal sight adjustment.

The CLI exposes first-order propagation only for the non-advanced homogeneous, level-geometry scenario. Altitude-dependent atmosphere, local gravity, Coriolis, wind layers, inclined geometry, and temperature-velocity profiles require Monte Carlo in the desktop workflow. This keeps the simpler first-order path definition consistent with its implementation.

19.5. Output construction and availability

For each active input, a higher and lower trajectory must provide both samples at every requested range. If either sample is absent, that output row becomes unavailable. An input counts as completed only if its full pair covers every output distance. Result status is complete only when all active inputs complete and every row remains available; otherwise it is partial. A zero-input request returns `no_inputs`; an invalid nominal zero returns `baseline_unavailable`.

At a sample, path for this method is $y - h_s$ because the CLI restricts it to level non-advanced geometry. Holdover is evaluated with `atan2` for the higher and lower paths before differencing. Energy and momentum sensitivities are taken from the recomputed perturbed samples, not approximated from a separately coded closed form.

19.6. Worked variance contribution

At the muzzle, $E = \frac{1}{2}mV^2$, so

$$\frac{\partial E}{\partial V} = mV = p. \quad (19.7)$$

For the A-MAX example, $p_0 = 8.958921$ kg m/s. With a muzzle-velocity standard deviation $\sigma_V = 3$ m/s, the first-order energy standard-deviation contribution is

$$\sigma_{E,V} \approx \left| \frac{\partial E}{\partial V} \right| \sigma_V = (8.958921)(3) = 26.8768 \text{ J}. \quad (19.8)$$

Its variance contribution is about 722.36 J^2 . A drag contribution at the muzzle is zero because drag has not yet changed launch speed, but it grows downrange. At 100 m, the fixture's combined muzzle-velocity and drag inputs produce a native energy standard deviation of 25.7832 J; the complete result follows from the numerically evaluated sensitivities rather than the muzzle identity alone.

Figure 19.1 traces how the speed standard deviation builds along the flight for two inputs: a 3 m/s muzzle-velocity spread and a two-percent drag spread. The two contributions behave oppositely. Velocity spread starts at the full 3 m/s and slowly shrinks as drag scrubs the fast and slow shots toward each other, while drag spread starts at zero and grows with distance. Added in quadrature per Equation (19.2), the total dips slightly, then climbs once drag uncertainty dominates near 300 m.

Model limitation

First-order propagation assumes local linearity and independent inputs. It does not represent configured correlations, skewness, multimodality, hard-boundary pileup, or model-form uncertainty. A symmetric output standard deviation also does not capture an asymmetric nonlinear response. Monte Carlo is the available alternative for correlated or materially nonlinear cases.

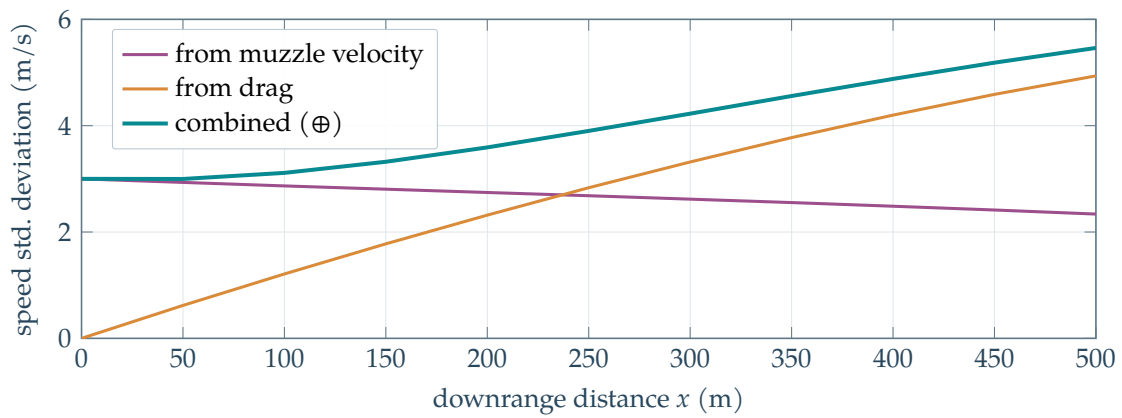


Figure 19.1.: First-order speed uncertainty for the A-MAX fixture, computed by the same central-difference perturbations the engine uses. The combined curve is the quadrature sum of the two contributions; its early dip and later rise show that no single input dominates the whole trajectory.

20. Monte Carlo uncertainty

Physical interpretation

Monte Carlo samples a joint input distribution, transforms independent standard normals through a Cholesky factor, applies physical perturbations and bounds, re-zeroes the complete scenario, and forms empirical output percentiles from completed trajectories.

20.1. Input random vector

The seven-dimensional standardized vector is ordered as

$$z = (z_V, z_D, z_T, z_P, z_H, z_C, z_Z)^T, \quad (20.1)$$

corresponding to muzzle velocity, drag, temperature, pressure, headwind, crosswind, and zero range. Before correlation, components are independent $N(0, 1)$ variables.

The request can declare pairwise correlations ρ_{ij} with $|\rho_{ij}| < 1$. Diagonal values are one. Unspecified off-diagonal values are zero. Conflicting duplicate declarations are rejected. The resulting symmetric correlation matrix is

$$R = \begin{bmatrix} 1 & \rho_{12} & \cdots \\ \rho_{12} & 1 & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}. \quad (20.2)$$

It must be positive semidefinite. Pairwise-valid numbers can still form an invalid matrix; for example, three strong correlations can contradict one another globally.

20.2. Cholesky construction

The program constructs a lower-triangular matrix L such that

$$R = LL^T. \quad (20.3)$$

For row i and column $j \leq i$, it subtracts previously formed products:

$$a_{ij} = R_{ij} - \sum_{k < j} L_{ik} L_{jk}. \quad (20.4)$$

Then

$$L_{ii} = \sqrt{\max(0, a_{ii})}, \quad (20.5)$$

$$L_{ij} = \frac{a_{ij}}{L_{jj}}, \quad i > j, \quad (20.6)$$

unless a near-zero diagonal identifies a singular row. Values below -10^{-12} on a diagonal reject the matrix; consistent positive-semidefinite singular directions are retained with zero factors.

If $q \sim N(0, I)$ is independent, the correlated sample is

$$z = Lq. \quad (20.7)$$

Its covariance is

$$\text{Cov}(z) = LIL^T = R.$$

This identity is the reason Cholesky multiplication produces the requested correlation.

20.2.1. Two-variable example

For two standardized inputs with correlation $\rho = 0.6$,

$$R = \begin{bmatrix} 1 & 0.6 \\ 0.6 & 1 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 \\ 0.6 & 0.8 \end{bmatrix}.$$

The transformed values are

$$z_1 = q_1, \quad z_2 = 0.6q_1 + 0.8q_2.$$

Because q_1 and q_2 have unit variance and zero covariance,

$$\text{Cov}(z_1, z_2) = 0.6, \quad \text{Var}(z_2) = 0.6^2 + 0.8^2 = 1.$$

20.3. Deterministic pseudorandom normals

The generator state is a 64-bit unsigned integer. Each uniform draw advances it by

$$0 \times 9\text{e}3779\text{b}97\text{f}4\text{a}7\text{c}15$$

and applies SplitMix64-style xor-shift and multiplication mixing constants. The upper 53 bits are scaled by 2^{-53} to form a double-precision uniform value on $[0, 1)$.

Normal values are generated in pairs with the Box–Muller transform [16]. For independent uniforms u_1, u_2 ,

$$r = \sqrt{-2 \log(\max[u_1, \text{smallest positive double}])}, \quad (20.8)$$

$$\theta = 2\pi u_2, \quad (20.9)$$

$$q_1 = r \cos \theta, \quad q_2 = r \sin \theta. \quad (20.10)$$

One value is returned and the other cached for the next call. The seed is recorded in the result. The API default is hexadecimal `0x42574d43`, equal to decimal 1113017667 and suggestive of “BWMC.” A fixed seed, sample count, correlation matrix, and deterministic model produce repeatable draws and results on the same numerical implementation.

20.4. Transforming samples into physical inputs

Each standardized component is multiplied by its supplied standard deviation and added to the baseline where appropriate:

$$\begin{aligned} T' &= \text{clamp}(T + z_T \sigma_T, -60, 60), \\ P' &= \text{clamp}(P + z_P \sigma_P, 500, 1100), \\ w'_h &= \text{clamp}(w_h + z_H \sigma_H, -100, 100), \\ w'_c &= \text{clamp}(w_c + z_C \sigma_C, -100, 100), \\ V'_0 &= \text{clamp}(V_0 + z_V \sigma_V, 0.5V_{\text{base}}, 1.5V_{\text{base}}), \\ x'_z &= \text{clamp}(x_z + z_Z \sigma_Z, 5, 1000). \end{aligned}$$

Temperature, pressure, and winds construct a new firing-point `Atmosphere`; relative humidity remains at its baseline value. Other advanced environment settings, including layers, lapse option, local gravity, and Coriolis, remain in the copied scenario.

Drag is perturbed logarithmically:

$$m'_D = m_{D,0} \exp(s_D z_D \sigma_D), \quad s_D = \begin{cases} -1, & \text{reference BC,} \\ +1, & \text{direct drag.} \end{cases} \quad (20.11)$$

The negative sign for reference BC gives a positive random coefficient change less acceleration, consistent with Equation (7.4).

Clamping makes the actual physical input distribution differ from an unbounded multivariate normal near a limit. Probability beyond the limit accumulates at the boundary. The output percentiles therefore describe the implemented bounded sampling model, not a mathematically untruncated Gaussian.

20.5. Complete scenario solution per draw

For every draw, the program calls `integrate_shot_scenario`. Because zero range is sampled and other inputs affect drop, this call re-solves bore elevation for the sampled scenario before integrating the horizon. The sample is retained only if zeroing is complete and the trajectory covers the greatest requested output distance.

At each output distance, the solver extracts ground speed, energy, momentum, time, drop, sight-relative path, holdover, and integrated lateral displacement. If any output sample is unavailable, the complete Monte Carlo draw is discarded rather than mixing metrics from different sample counts.

Listing 20.1: The per-draw loop from `propagate_monte_carlo_uncertainty`. `correlated_normal` returns one Lq sample; the temperature, pressure, and wind clamps are shown, the remaining inputs abbreviated. Metric collection and quantile formation follow after the loop.

```
const auto lower = correlation_cholesky(correlations); // R = L Lt, built once
for (std::size_t sample_index = 0; sample_index < sample_count; ++sample_index)
{
    const auto random = correlated_normal(generator, lower); // seven correlated normals
    auto scenario = baseline_scenario;
    const auto temperature_c = std::clamp(
        baseline_atmosphere.temperature_c + random[2] * deviations[2], -60.0, 60.0);
    const auto pressure_hpa = std::clamp(
        baseline_atmosphere.station_pressure_hpa + random[3] * deviations[3], 500.0, 1100.0);
    // ... headwind, crosswind, muzzle velocity, and zero range clamped the same way ...
    scenario.environment.firing_point = Atmosphere::create(
        temperature_c, pressure_hpa, baseline_atmosphere.relative_humidity_percent,
        headwind_mps, crosswind_mps);

    const auto drag_log_delta = random[1] * deviations[1]; // drag perturbation
    scenario.solver.aerodynamic_drag_multiplier =
        baseline_scenario.solver.aerodynamic_drag_multiplier *
        std::exp(reference_bc ? -drag_log_delta : drag_log_delta); // sign flips for BC

    const auto zeroed = integrate_shot_scenario(projectile, scenario); // re-zero + integrate
    if (zeroed.status != ZeroingStatus::complete ||
        zeroed.trajectory.covered_distance_m + 1e-9 < output_distances_m.back())
        continue; // discard an incomplete draw

    ++result.completed_sample_count;
    // ... append speed, energy, momentum, time, drop, path, holdover, wind drift ...
}
```

Sample count must lie from 100 to 10000. If no draw completes, status is `baseline_unavailable`. If all requested draws complete, status is `complete`; otherwise it is `partial`. If every standard deviation is zero, deterministic repeated results yield `no_inputs` even though the requested calculations can complete.

20.6. Empirical quantiles

For n completed values sorted as $y_{(0)} \leq \dots \leq y_{(n-1)}$, the quantile at probability p uses position

$$a = p(n - 1). \quad (20.12)$$

With $j = \lfloor a \rfloor$, $k = \lceil a \rceil$, and $f = a - j$,

$$Q(p) = y_{(j)} + f[y_{(k)} - y_{(j)}]. \quad (20.13)$$

The reported interval is

$$[Q(0.025), Q(0.5), Q(0.975)], \quad (20.14)$$

stored as low 95, median, and high 95. This is an empirical central interval conditional on the input distribution and model. It is not automatically centered on the nominal output and need not be symmetric.

20.7. Split-sample quantile diagnostic

The generated sequence for each metric is divided into its first and second halves. If at least forty completed values exist, the program compares the two lower-tail and two upper-tail estimates. With full-sample median m , the normalized discrepancy is

$$\Delta_Q = \frac{\max\{|Q_{L,1} - Q_{L,2}|, |Q_{H,1} - Q_{H,2}|\}}{\max(1, |m|)}. \quad (20.15)$$

The result retains the maximum Δ_Q over all metrics and distances. If fewer than forty completed values exist, the diagnostic is infinity. This number is an internal stability indicator: large disagreement between halves warns that tail estimates are still noisy for the chosen sample size.

It is not a formal convergence proof. The halves are finite pseudorandom samples, and a small discrepancy in one split cannot guarantee accurate extreme quantiles. It does, however, expose a reproducible sensitivity of reported interval endpoints to sample partition.

20.8. First-order and Monte Carlo compared

Table 20.1.: Interpretive differences between the two uncertainty methods.

Property	First order	Monte Carlo
Mathematical approximation	Local derivative and variance sum	Empirical distribution of repeated nonlinear solves
Input dependence	Independent only	Configured correlation matrix
Zeroing	Fixed nominal bore for most inputs; re-zero only zero-range perturbation	Re-zero every sampled scenario
Output form	Standard deviation, commonly displayed with a normal multiplier	Median and empirical 2.5 / 97.5 percentiles
Nonlinearity	Retained only through local symmetric derivative	Retained across sampled domain
Bounds	Finite-difference endpoints clipped	Every draw physically clamped
Cost	About two trajectories per active input, plus zeroing where needed	One complete zeroed scenario per requested draw
Diagnostics	Active / completed input pairs and row availability	Requested / completed draws and split-quantile delta

The methods answer related but nonidentical questions because their zeroing semantics differ. First-order propagation models variation around an already-set nominal bore for most inputs. Monte Carlo models a population of complete scenarios that are re-zeroed individually. Even in a nearly linear case, those definitions can produce different path intervals.

Model limitation

Neither method samples an automatic model-discrepancy term. Their intervals are conditional on chosen standard deviations, chosen correlations, normal input shapes before clamping, the deterministic point-mass model, and the selected drag representation. Monte Carlo can propagate input nonlinearity; it cannot reveal an omitted physical mechanism unless that omission is represented explicitly as an input model.

Part VII.

Buckshot pattern analysis

21. Empirical D90 pattern model

Physical interpretation

The buckshot module models the spatial distribution of pellet impacts, not their exterior-flight interaction as a swarm. It fits linear growth of a measured 90-percent pattern diameter, separates calibration from holdout observations, and carries slope uncertainty into target probability.

21.1. What D90 means

D_{90} is the diameter of a centered circle expected to contain 90 percent of impacts under the fitted isotropic distribution. It is a pattern-spread statistic. It is not projectile diameter, group extreme spread, or the diameter that contains every pellet in one shell.

An observation contains range r_i , measured diameter D_i , standard uncertainty s_i , shell count N_i , and calibration or holdout role. The analyzer requires three to sixty-four observations, including at least two calibration observations and one holdout. Range and diameter must be positive; uncertainty must be positive; shell count lies from 1 to 1000.

The scenario also identifies pellet count, mean muzzle velocity, pellet velocity standard deviation, choke class, deformation class, target geometry, and target range. In the current mathematics, choke, deformation class, mean velocity, and velocity spread identify the conditions to which the observations belong; they are not numerical regressors in the D90 line. Changing one of those labels without changing observations does not change the fitted slope. This is an important boundary of the implemented empirical model.

21.2. Through-origin linear growth

The model assumes

$$D_{90}(r) = \alpha r, \quad (21.1)$$

where α is an angular diameter in radians when diameter and range use the same length unit. The intercept is fixed at zero: at zero range, the idealized pattern diameter is zero. This is a simplifying far-field growth model; it does not represent muzzle offset, wad behavior, an initial shot-column diameter, or a nonlinear transition near the muzzle.

For calibration observation i , the weight is

$$w_i = \frac{N_i}{s_i^2}. \quad (21.2)$$

More shells increase weight linearly; larger reported uncertainty decreases it quadratically. The weighted least-squares objective is

$$\Psi(\alpha) = \sum_{i \in \mathcal{C}} w_i (\alpha r_i - D_i)^2. \quad (21.3)$$

Differentiating and setting the result to zero,

$$\frac{d\Psi}{d\alpha} = 2 \sum_{i \in \mathcal{C}} w_i r_i (\alpha r_i - D_i) = 0,$$

gives the closed-form estimator

$$\hat{\alpha} = \frac{\sum_{i \in \mathcal{C}} w_i r_i D_i}{\sum_{i \in \mathcal{C}} w_i r_i^2}. \quad (21.4)$$

The denominator must be finite and positive. Unlike BC calibration, no iterative optimizer or trajectory solve is needed because the model is linear in its one parameter.

Figure 21.1 illustrates the model: a single slope $\hat{\alpha}$ fixes a line through the origin, and the slope uncertainty of Equation (21.11) opens a widening band around it. Because the line passes through the origin, the absolute width of the band grows with range while its relative width stays fixed.

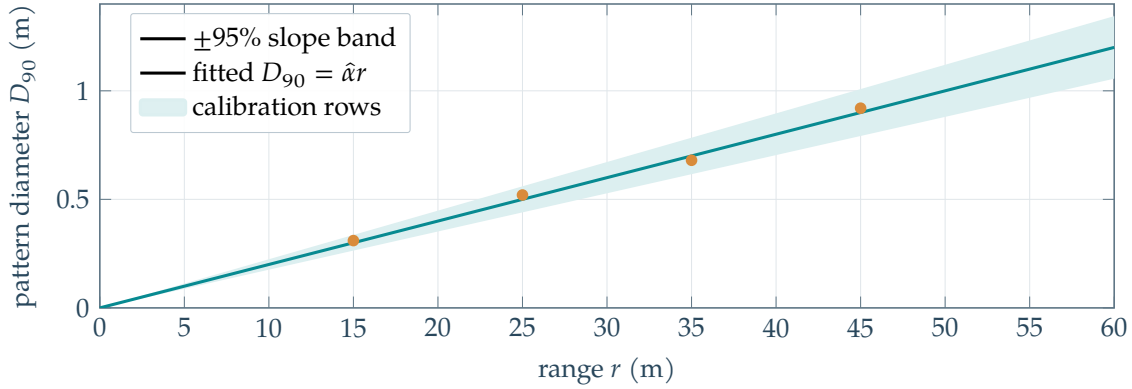


Figure 21.1.: Through-origin D90 growth model with its slope-uncertainty band (values illustrative). The fit has a single parameter, so the whole family of admissible lines pivots about the origin. Calibration observations at longer ranges therefore constrain the slope more tightly than observations near the muzzle.

Physical interpretation

Shell count is treated as repeated information at the stated row uncertainty. The implementation multiplies weight by N_i ; it does not divide the supplied uncertainty by $\sqrt{N_i}$ first and then apply a second count factor. The meaning of s_i should therefore be consistent with this weighting convention.

21.3. Calibration and holdout residuals

For every observation, including holdouts, predicted diameter and residual are

$$\hat{D}_i = \hat{\alpha} r_i, \quad (21.5)$$

$$e_i = \hat{D}_i - D_i, \quad (21.6)$$

$$\tilde{e}_i = \frac{e_i}{s_i}. \quad (21.7)$$

The calibration RMSE is the unweighted row-level value

$$\text{RMSE}_{\mathcal{C}} = \sqrt{\frac{1}{n_{\mathcal{C}}} \sum_{i \in \mathcal{C}} e_i^2}, \quad (21.8)$$

and holdout RMSE uses only the n_H excluded observations:

$$\text{RMSE}_{\mathcal{H}} = \sqrt{\frac{1}{n_H} \sum_{i \in \mathcal{H}} e_i^2}. \quad (21.9)$$

Holdouts never enter Equation (21.4). Their residuals therefore describe performance at data withheld from parameter estimation, subject to whether they are physically separate and representative.

Reduced chi-square uses shell-count weighting:

$$\chi_v^2 = \frac{\sum_{i \in C} N_i \tilde{e}_i^2}{n_C - 1}. \quad (21.10)$$

There is one fitted parameter, so degrees of freedom are $n_C - 1$.

21.4. Slope uncertainty and predicted diameter interval

The slope standard uncertainty is

$$s_\alpha = \sqrt{\frac{\max(1, \chi_v^2)}{\sum_{i \in C} w_i r_i^2}}. \quad (21.11)$$

As in BC calibration, the variance scale is not allowed below one. The approximate 95 percent slope bounds are

$$\alpha_L = \max(\hat{\alpha} - z_{0.975} s_\alpha, 10^{-6} \hat{\alpha}), \quad (21.12)$$

$$\alpha_H = \hat{\alpha} + z_{0.975} s_\alpha, \quad z_{0.975} = 1.95996398454. \quad (21.13)$$

At target range r_T ,

$$\hat{D}_T = \hat{\alpha} r_T, \quad (21.14)$$

$$D_{T,L} = \alpha_L r_T, \quad (21.15)$$

$$D_{T,H} = \alpha_H r_T. \quad (21.16)$$

The result status is `validated_in_domain` when r_T lies between the minimum and maximum calibration ranges, inclusive. It is `extrapolated` otherwise. This status concerns range only. It does not automatically test whether choke, pellet material, velocity distribution, or other ammunition conditions match the observations, and it does not turn a poor holdout RMSE into an in-domain failure status.

Model limitation

The confidence construction accounts for fitted slope uncertainty under the weighted linear model. It does not add shell-to-shell overdispersion, uncertain aim point, target measurement error beyond supplied s_i , uncertainty in the through-origin assumption, or changes in pattern shape with range.

22. From D90 to target probability

Physical interpretation

A fitted D90 is converted into the scale of an isotropic two-dimensional Gaussian. Rectangular target probability separates into normal CDF differences; circular target probability is integrated numerically. A binomial model then converts one-pellet probability into a pellet-count distribution.

22.1. Isotropic Gaussian impact model

Let the horizontal and vertical impact coordinates be independent

$$X \sim N(0, \sigma^2), \quad Y \sim N(0, \sigma^2). \quad (22.1)$$

The joint density is

$$f(x, y) = \frac{1}{2\pi\sigma^2} \exp\left[-\frac{x^2 + y^2}{2\sigma^2}\right]. \quad (22.2)$$

Radial distance $R = \sqrt{X^2 + Y^2}$ follows a Rayleigh distribution with CDF

$$P(R \leq r) = 1 - \exp\left(-\frac{r^2}{2\sigma^2}\right). \quad (22.3)$$

Setting this probability to 0.9 gives

$$r_{90} = \sigma \sqrt{2 \ln 10} = 2.145966026289347 \sigma. \quad (22.4)$$

Because $D_{90} = 2r_{90}$,

$$\sigma = \frac{D_{90}}{2(2.145966026289347)} = \frac{D_{90}}{4.291932052578694}. \quad (22.5)$$

22.1.1. Worked D90 conversion

For $D_{90} = 0.60$ m,

$$\sigma = \frac{0.60}{4.2919320526} = 0.1397972 \text{ m}. \quad (22.6)$$

A centered circle of radius 0.10 m has analytical probability

$$P(R \leq 0.10) = 1 - \exp\left[-\frac{0.10^2}{2(0.1397972)^2}\right] = 0.225736. \quad (22.7)$$

The production circle calculation obtains the same result numerically through the general offset-circle integration described below.

Holding the target fixed and letting range vary shows why buckshot is a short-range proposition. Figure 22.1 plots the single-pellet probability of striking a 0.10 m-radius centered circle as the pattern spreads with range, for a representative slope $\alpha = 0.02$ rad. The probability falls from the definitional 0.9 at the range where D_{90} equals the pattern's own r_{90} scale down to a few percent by 60 m. The marked point reproduces the worked 0.60 m conversion above at 30 m.

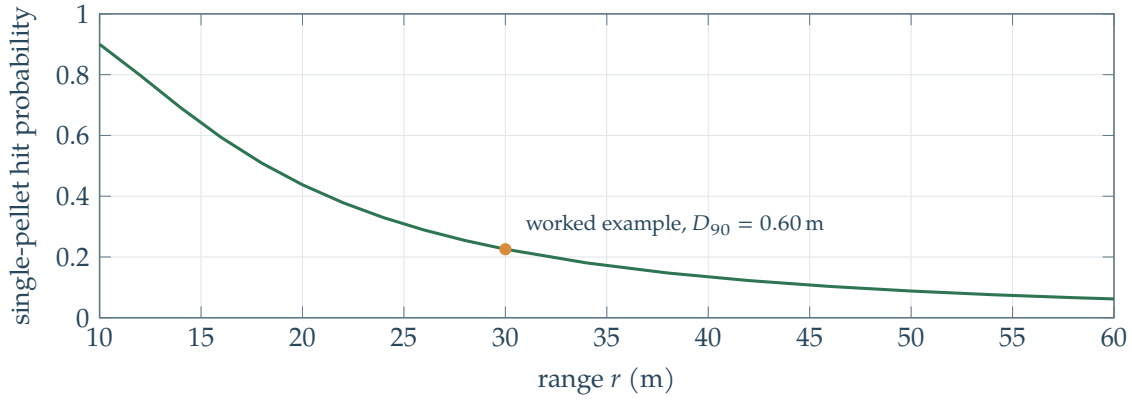


Figure 22.1.: Single-pellet probability of hitting a fixed 0.10 m-radius circle as range grows and the pattern widens, for slope $\alpha = 0.02$ rad. The isotropic Gaussian scale σ grows linearly with range, so the probability follows the Rayleigh law $1 - \exp(-R_T^2/2\sigma^2)$.

22.2. Rectangular target probability

For a rectangle of width W , height H , and center (c_x, c_y) , the bounds are

$$\begin{aligned} x_L &= c_x - W/2, & x_H &= c_x + W/2, \\ y_L &= c_y - H/2, & y_H &= c_y + H/2. \end{aligned}$$

Independence allows the two-dimensional integral to factor:

$$p_{\text{rect}} = \left[\Phi\left(\frac{x_H}{\sigma}\right) - \Phi\left(\frac{x_L}{\sigma}\right) \right] \left[\Phi\left(\frac{y_H}{\sigma}\right) - \Phi\left(\frac{y_L}{\sigma}\right) \right], \quad (22.8)$$

where

$$\Phi(z) = \frac{1}{2} \operatorname{erfc}\left(-\frac{z}{\sqrt{2}}\right). \quad (22.9)$$

The final probability is clamped to $[0, 1]$ to remove any tiny floating-point overshoot.

Offsets are measured from the Gaussian mean or aim center. A positive horizontal or vertical center moves the target region away from the mean without changing the distribution. The factorization remains valid because the target edges are aligned with the modeled X and Y axes.

22.3. Circular target probability

For a circle, the target width is interpreted as diameter and

$$R_T = W/2. \quad (22.10)$$

At horizontal coordinate x between $c_x - R_T$ and $c_x + R_T$, the circular half-height is

$$h(x) = \sqrt{\max[0, R_T^2 - (x - c_x)^2]}. \quad (22.11)$$

Conditional vertical probability through the circle is

$$P_y(x) = \Phi\left(\frac{c_y + h(x)}{\sigma}\right) - \Phi\left(\frac{c_y - h(x)}{\sigma}\right). \quad (22.12)$$

The marginal horizontal density is

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right). \quad (22.13)$$

Thus

$$p_{\text{circle}} = \int_{c_x - R_T}^{c_x + R_T} f_X(x) P_Y(x) dx. \quad (22.14)$$

The implementation evaluates this integral with composite Simpson's rule using exactly 1024 equal intervals. Let $a = c_x - R_T$, $b = c_x + R_T$, $h_s = (b - a)/1024$, and $g(x) = f_X(x)P_Y(x)$. Then

$$\int_a^b g(x) dx \approx \frac{h_s}{3} \left[g(a) + g(b) + 4 \sum_{j \text{ odd}} g(a + jh_s) + 2 \sum_{j \text{ even}, j \neq 0, 1024} g(a + jh_s) \right]. \quad (22.15)$$

Using an even fixed interval count makes the evaluation deterministic. Simpson's rule is appropriate because the Gaussian-CDF integrand is smooth inside the circle; the square-root boundary is handled at exact zero half-height at the endpoints.

Computational implementation

The circle algorithm uses the general offset geometry even for a centered circle, rather than switching to the Rayleigh closed form. This keeps one code path for centered and offset targets. The analytical centered result remains useful as a numerical check.

22.4. Diameter interval to probability interval

The analyzer evaluates target probability at the nominal, low, and high predicted D90 values. It reports the minimum and maximum of those three probabilities as approximate 95 percent probability bounds. For a centered target, hit probability normally decreases as D90 grows; for an offset target, the relationship can be nonmonotonic over some ranges, so explicitly taking the extrema of all three evaluations is safer than assuming an order.

This transformation includes only the slope-derived D90 uncertainty. It does not integrate over a posterior distribution of α , and it does not propagate uncertainty in aim center, target dimensions, or pellet count.

If the predicted diameters are D_L, D_N, D_H , the implementation forms

$$p_j = P_T(D_j), \quad p_{\min} = \min_j p_j, \quad p_{\max} = \max_j p_j, \quad j \in \{L, N, H\}. \quad (22.16)$$

For a centered circular target of radius R_T , Equations (22.5) and (22.3) give

$$P_T(D) = 1 - \exp\left(-\frac{c_{90}^2 R_T^2}{2D^2}\right), \quad c_{90} = 4.291932052578694,$$

whose derivative is strictly negative for $D > 0$. In that special geometry the high-D90 endpoint necessarily gives the low probability. The general rectangle and offset-circle paths do not rely on that monotonic shortcut; evaluating all three points keeps the interval construction valid when moving probability mass can first enter and later leave an offset target.

The reported probability bounds are therefore a deterministic transformation of three fitted-diameter summaries. They are not binomial confidence limits and do not include uncertainty in the subsequent pellet-count model. The next chapter applies the nominal one-pellet probability to the discrete count distribution separately.

23. Pellet-count probabilities and assumptions

Physical interpretation

Once one-pellet hit probability is known, the current implementation assumes independent, identically distributed pellet impacts. The number of hits is binomial; stable logarithmic formulas avoid overflowing factorials.

23.1. Binomial model

Let N be pellet count and p one-pellet target probability. Under independent identical trials,

$$K \sim \text{Binomial}(N, p), \quad (23.1)$$

so

$$P(K = k) = \binom{N}{k} p^k (1 - p)^{N-k}, \quad k = 0, \dots, N. \quad (23.2)$$

The expected pellet count is

$$E[K] = Np. \quad (23.3)$$

For a requested minimum $k_{\min}$,

$$P(K \geq k_{\min}) = \sum_{k=k_{\min}}^N P(K = k). \quad (23.4)$$

For the worked $D_{90} = 0.60$ m centered-circle example, $p = 0.225736$. With nine pellets,

$$\begin{aligned} E[K] &= 9p = 2.03163, \\ P(K \geq 3) &\approx 0.331601. \end{aligned}$$

These results describe the binomial model for the stated target and spread; they are not a trajectory calculation and do not use nine times the per-pellet exterior-flight mass.

Figure 23.1 shows the full distribution behind those two summaries. With a per-pellet probability near 0.23 and nine pellets, the most likely single outcome is two hits, but the distribution is broad: zero hits carries a full ten percent, and the probability of three or more (the shaded bars) is about a third.

23.2. Stable logarithmic calculation

Direct factorials in $\binom{N}{k} = N!/[k!(N-k)!]$ overflow quickly even when the final probability is moderate. The program evaluates

$$\begin{aligned} \log P(K = k) &= \log \Gamma(N + 1) - \log \Gamma(k + 1) - \log \Gamma(N - k + 1) \\ &\quad + k \log p + (N - k) \log(1 - p). \end{aligned} \quad (23.5)$$

The final term uses $\log_{1p}(-p)$ for accuracy when p is small. Exponentiating gives each probability. The vector is then normalized by its computed sum to remove accumulated roundoff.

The edge cases are handled exactly: if $p \leq 0$, all mass is assigned to $K = 0$; if $p \geq 1$, all mass is assigned to $K = N$. This avoids logarithms of zero.

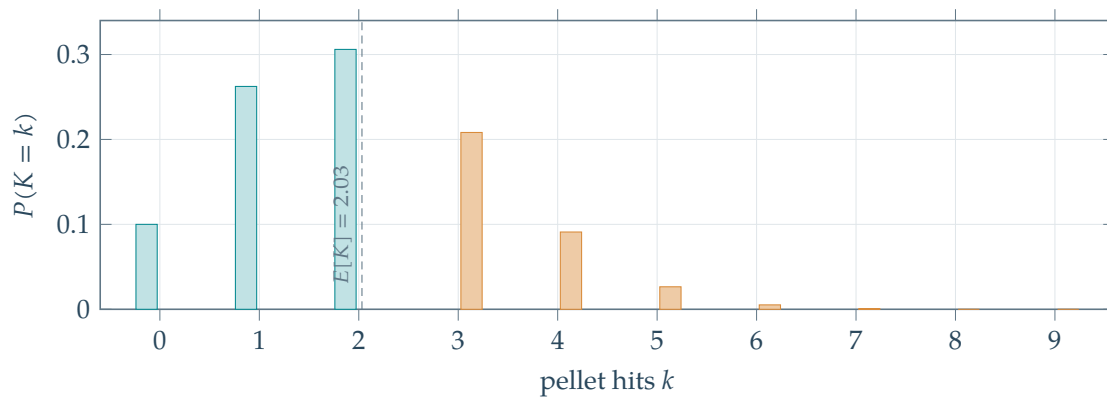


Figure 23.1.: Binomial pellet-count distribution for the worked example ($N = 9$, $p = 0.2257$). Shaded amber bars are the outcomes counted by $P(K \geq 3) \approx 0.33$. The spread of this distribution, not just its mean, is what a minimum-hit requirement depends on.

23.3. What independence assumes

The binomial model assumes pellet impacts are conditionally independent draws from the same centered Gaussian. Real pellets from one shell can share muzzle disturbance, wad interaction, common aim error, aerodynamic wake effects, deformation, and shell-specific pattern shifts. Such shared effects create correlation or overdispersion relative to a binomial distribution.

The fitted D90 can still summarize marginal spread while the pellet-count tails are misrepresented by independence. A hierarchical model with shell-level center shifts or a beta-binomial count layer would answer a different question and is not part of the current implementation.

23.4. Complete model pipeline

The buckshot calculation can be summarized as

1. validate observation roles, dimensions, shell counts, and target geometry;
2. compute weights N_i/s_i^2 for calibration rows;
3. solve the closed-form through-origin slope $\hat{a}$;
4. report calibration and holdout residuals separately;
5. estimate slope uncertainty from the weighted residual scale;
6. predict D90 and approximate bounds at the target range;
7. convert each D90 to isotropic Gaussian σ ;
8. integrate one-pellet probability over the rectangle or circle;
9. generate the full binomial pellet-count distribution;
10. report expected hits and the probability of meeting the requested minimum.

This pipeline contains regression, uncertainty transformation, spatial probability integration, and count probability. It never calls the exterior trajectory integrator. The CLI associates it with a selected shotgun load, overwrites pellet count from that load, and obtains mean muzzle speed from the solved trajectory's first sample, but those conditioning values do not enter the current slope or Gaussian equations.

Model limitation

The explicit model assumptions are linear D90 growth through the origin, one isotropic centered Gaussian, range validity determined by calibration-range inclusion, and independent identical pellet hits. Pellet wakes, collective swarm aerodynamics, aim error, shell-level shifts, non-Gaussian rings or voids, choke transitions, deformation physics, and transfer to other ammunition conditions are not modeled. Choke and deformation fields document conditioning; they are not fitted effects.

Part VIII.

How the calculations are implemented

24. Calculation-oriented core design

Physical interpretation

The C++ core separates physical definitions, deterministic integration, geometric analysis, parameter estimation, and stochastic repetition. The design rests on one idea: each layer has a single mathematical responsibility and reaches the layers below it through explicit data, independent of any language detail.

24.1. Core data as physical concepts

A load is assembled from four conceptual pieces. The projectile definition owns name, one-projectile mass, optional rifle geometry, and exactly one aerodynamic definition. Ammunition owns nominal muzzle velocity and payload count. Firearm configuration identifies shotgun or rifle and can carry a load-specific twist. Provenance owns stable identity and a description of how the drag parameter was obtained.

The aerodynamic definition is mutually exclusive:

$$\text{drag} = \begin{cases} \text{reference G1/G7 with constant or banded BC,} \\ \text{tabulated Mach-}C_D \text{ with reference diameter,} \\ \text{sphere with diameter and material density.} \end{cases} \quad (24.1)$$

This structural exclusivity prevents a calculation from accidentally combining an old BC with newly selected sphere geometry. Helper functions such as `reference_bc_drag`, `tabulated_drag`, and `sphere_drag` expose the active representation to algorithms.

A shot scenario similarly combines environment, geometry, solver configuration, and muzzle-velocity multiplier. The environment owns the firing-point atmosphere and optional spatial behavior. Geometry owns horizon, zero range, sight height, target geometry, vital-zone diameter, and desired output interval. Solver configuration owns tolerances, step bounds, termination settings, launch angles, and optional force switches.

The types carry SI values internally. Conversions from grains, feet per second, inches, or presentation units occur when built-in data are defined or when the application forms a request. This makes the derivative and solver equations dimensionally consistent and avoids repeated unit branching in inner loops.

24.2. Five computational roles

The program uses five roles that are easy to blur in a monolithic description.

24.2.1. Physical model

A physical model evaluates a local quantity from state and parameters. `Atmosphere::create` evaluates thermodynamic properties. `Environment::atmosphere_at` evaluates a spatial atmosphere. `drag_retardation_mps2` maps airspeed, projectile, and atmosphere to scalar decelera-

Table 24.1.: Important calculation data structures and their meaning.

Concept	Information carried
Projectile definition	Single-projectile mass, optional bullet length/diameter, and one drag representation
Ammunition load	Nominal muzzle velocity and payload count
Environment	Firing-point thermodynamics, altitude behavior, local gravity, Coriolis location, and wind layers
Solver configuration	Error tolerances, time-step limits, work bounds, launch/sight angles, ground rule, and force multipliers
Shot geometry	Horizon, sight zero, sight height, target inclination/elevation, vital-zone diameter, and output spacing
Time state	Numerical array (x, y, z, v_x, v_y, v_z)
Trajectory sample	Position and two velocity vectors plus speeds, energy, momentum, time, drop, lateral displacement, and aerodynamic diagnostics
Trajectory	Retained samples, mass, requested/covered range, termination, and solver diagnostics
Zeroed trajectory	Trajectory plus bore elevation, final zero error, and zeroing status

tion. `coriolis_acceleration` maps ground velocity and geographic inputs to a vector correction.

24.2.2. Differential-equation solver

`integrate_trajectory` advances the six-component state. It does not know what a near zero or MPBR means. It only knows initial state, derivative, adaptive error, retained range coordinates, and termination.

24.2.3. Root-finding operation

`integrate_shot_scenario` varies bore elevation to drive path at zero range to zero. `compute_scenario_mpbr` varies a different bore elevation to drive maximum path to the vital-zone radius. Their bisection logic calls the solver but does not modify the solver's integration formula.

24.2.4. Interpolation and derived analysis

Time-domain Hermite interpolation produces exact-range states inside an accepted integration step. `analyze_trajectory_events` then uses a range-domain cubic to locate geometric events on the solved history. Energy, path, holdover, Mach crossings, and statuses are derived rather than integrated as independent dynamics.

24.2.5. Estimator or stochastic wrapper

Calibration, first-order propagation, and Monte Carlo create many modified inputs and call the deterministic solver. Their state is an optimization iteration, sensitivity accumulation, or random sample collection, not an alternate trajectory state.

Computational implementation

This separation makes a result traceable. A change in Cramer sound speed belongs to the physical-model layer. A change in error tolerance belongs to the solver. A different zero bracket belongs to root finding. A new event threshold belongs to analysis. A new input distribution belongs to the stochastic wrapper.

The same separation gives a practical change-impact map. A physical-model change can alter every derivative evaluation and all downstream results. A solver-tolerance change should alter numerical convergence but not the declared force law. A root-definition change can alter the selected bore angle while leaving one fixed-angle trajectory solver untouched. An output-format change should alter none of those values. I use these ownership boundaries when deciding which validation package must respond to a code change.

Changed concern	Owning role	Expected propagation
Atmospheric formula	Physical model	Derivative, trajectory, roots, and repeated analyses
Step tolerance	ODE solver	Accepted steps and numerical residuals
Zero condition	Root finding	Bore elevation and the resulting final trajectory
Mach-event threshold	Derived analysis	Event output only
Serialized field name	Boundary / presentation	Transport shape, not calculated values

25. The deterministic execution path

Physical interpretation

A normal calculation proceeds from validated canonical inputs to model objects, nested zero and MPBR solves, retained trajectories, analysis passes, optional repeated uncertainty solves, and one structured result. This chapter follows that sequence at the level of algorithms.

25.1. 1. Validate and normalize the request

The desktop constructs protocol-version 2 data in canonical SI. Custom sphere mass is derived from diameter and density; reference and tabulated projectiles require explicit mass. The native parser rejects unknown fields, wrong types, non-finite numbers, unsupported versions, ambiguous geometry, unordered arrays, duplicate identities, and model combinations that violate the core's invariants.

Validation is intentionally performed before numerical work. It is cheaper and clearer to reject a non-increasing Mach table than to discover a zero interpolation denominator inside a Runge-Kutta stage. The parser returns structured issues with code, field, message, and severity. A request with errors never reaches the calculation writer.

One temperature-velocity profile can modify each firearm's base muzzle multiplier. The profile contains two to twelve ascending temperature knots. Temperature outside the knot interval uses the nearest endpoint multiplier; inside, the multiplier is linearly interpolated. The effective scenario multiplier is

$$m_V = m_{V,\text{firearm}} m_{V,\text{temperature}}(T). \quad (25.1)$$

The launch speed is then $v_0 = m_V V_{0,\text{load}}$.

25.2. 2. Construct atmosphere and environment

`write_response` first calls `Atmosphere::create` with scenario temperature, station pressure, humidity, and homogeneous wind. It then builds an `Environment` containing firing-point altitude, lapse option, local-gravity flag, Coriolis flag, latitude, geographic azimuth, layers, and wind provenance.

Atmosphere validity flags are calculated once for the firing point and placed in the response. If the Cramer result is outside its declared domain, a warning is added before individual loads are solved.

25.3. 3. Assemble the load set

The six built-in loads are copied and the validated custom loads appended. A request supports at most three custom loads, so the normal complete calculation contains at most nine. Stable IDs, not array positions, identify them across calculation and presentation.

Each load selects either shotgun or rifle sight, zero, multiplier, and twist configuration. The program constructs a separate `ShotScenario` per load because firearm geometry and temperature-velocity multiplier can differ even when atmosphere and display range are shared.

25.4. 4. Select drag behavior

At runtime, drag dispatch follows the active definition:

1. For a sphere, calculate Mach and Reynolds number, evaluate the corrected sphere C_D , use $A = \pi d^2/4$, and apply actual mass.
2. For a tabulated definition, calculate Mach, interpolate or clamp C_D , use its reference diameter, and apply actual mass.
3. For reference drag, calculate Mach, obtain G1 or G7 reference C_D , choose the effective constant or airspeed band, and apply the conventional one-inch area and BC reference mass.

All three routes return a scalar retardation magnitude in metres per second squared. The derivative routine provides the common vector direction $-v_a/v_a$.

This dispatch occurs inside every derivative stage, not once per load. A banded BC can change bands; a table can change intervals; a sphere can move through the drag crisis or transonic correction as the trajectory evolves.

25.5. 5. Build the trial launch state

The solver validates the projectile and solver configuration, applies the effective muzzle multiplier, and computes Equation (1.8). It creates the first trajectory sample at $x = 0$ by evaluating local atmosphere and aerodynamic diagnostics. Energy and momentum use projectile mass and ground speed; airspeed uses local wind.

If maximum distance is zero, this muzzle sample is already a complete result. Otherwise the adaptive loop begins with $h = 1 \times 10^{-3}$ s clamped to configured bounds.

25.6. 6. Evaluate the derivative at a provisional state

The inner derivative performs the most frequently repeated calculation:

Listing 25.1: The derivative routine `time_derivative` exactly as the core evaluates it. The six-element state is $\{x, y, z, v_x, v_y, v_z\}$; the return value is its time derivative.

```
TimeState time_derivative(const TimeState& s, const Projectile& p,
    const Environment& environment, bool include_drag, bool include_gravity,
    double aerodynamic_drag_multiplier)
{
    const Vec3 position { s[0], s[1], s[2] };
    const auto atmosphere = environment.atmosphere_at(position);
    const auto rx = s[3] + atmosphere.headwind_mps; // air-relative velocity
    const auto ry = s[4];
    const auto rz = s[5] - atmosphere.crosswind_mps;
    const auto relative = std::hypot(rx, ry, rz); // airspeed
    const auto retard = include_drag
        ? aerodynamic_drag_multiplier * drag_retardation_mps2(relative, p, atmosphere)
        : 0.0;
    const auto drag_scale = relative > 1e-12 ? -retard / relative : 0.0; // zero-safe

    const auto gravity = include_gravity ? environment.gravity_at_height(s[1]) : 0.0;
    const auto coriolis = environment.coriolis_acceleration({ s[3], s[4], s[5] }, s[1]);
    return {
```

```

    s[3], s[4], s[5],
    drag_scale * rx + coriolis.x,
    drag_scale * ry - gravity + coriolis.y,
    drag_scale * rz + coriolis.z
};
}

```

The first three returned components restate that position differentiates to velocity. The last three assemble the acceleration: the drag scale multiplies each air-relative velocity component so the drag vector stays antiparallel to the flow, gravity subtracts from the vertical component, and Coriolis adds where enabled. The single guard `relative > 1e-12` is the only branch in the hot path; a validation configuration can zero the drag or gravity term, and the aerodynamic multiplier must remain positive and finite.

25.7. 7. Advance adaptively

`dormand_prince_54` evaluates seven derivatives and returns two endpoint arrays. `error_norm` applies Equation (11.8). The solver rejects or accepts using Equations (11.9) through (11.11). Counts and accepted-step extrema are updated only in the appropriate branches.

Accepted states remain ordinary double-precision arrays. The solver does not allocate a new object graph for every stage; it reuses a six-component provisional array and combines fixed-size derivatives. This keeps the calculation-oriented inner loop compact without changing its mathematical structure.

25.8. 8. Retain exact-range states

After acceptance, the interval may cross several 0.25 m grid points. For each one, the solver bisects Hermite $x(u)$, interpolates the full state, evaluates atmosphere and diagnostics at the interpolated position, and appends a `TrajectorySample`. It also localizes any terminal requested range, reversal, or ground crossing.

The retained sample stores both the reconstructed air-relative vector and scalar diagnostics. This prevents a later presentation layer from having to guess the wind transformation from incomplete data.

25.9. 9. Solve sight zero by repetition

The normal per-load path is `integrate_shot_scenario`. It first creates short trial trajectories to the zero range under changing launch elevations. After bracketing and bisection, it runs the complete horizon once. The final `ZeroedTrajectory` carries both numerical state history and the root-finding result.

The number of trajectory calls is not fixed because bisection can stop early, bracket expansion can add trials, and unavailable ranges can terminate the search. With a valid initial bracket, at most two endpoint evaluations plus fifty midpoint evaluations precede the final solve.

25.10. 10. Solve native MPBR separately

The response writer currently calls `compute_scenario_mpbr` for each load as a separate nested process. It does not reuse the normal zeroed trajectory because MPBR seeks a different bore

elevation. Each trial runs to the solution horizon and scans maximum path; the final trajectory supplies MPBR zero and lower-bound crossing.

This operation is computationally expensive relative to one trajectory. The native benchmark accordingly identifies MPBR as a distinct workload. Reuse would be mathematically valid only if two searches evaluated the same bore angle and horizon, which they generally do not.

25.11. 11. Analyze events and drag validity

Once the normal zeroed trajectory exists, `analyze_trajectory_events` scans its dense range history for sight-line roots, path apex, ground intersection, and Mach thresholds. `evaluate_drag_validity` separately scans Mach and Reynolds extrema through display distance.

Both are read-only passes over the trajectory. They do not reintegrate or change samples. Their statuses are added to the result even when a crossing is horizon-limited.

25.12. 12. Select public output samples

The CLI determines coverage through the lesser of display distance and actual covered distance. It selects retained samples at a stride

$$s = \max\left(1, \left\lfloor \frac{N_{\text{available}}}{500} \right\rfloor\right) \quad (25.2)$$

and adds the exact covered endpoint if needed. This bounds chart and response size while the internal trajectory remains dense.

Every serialized point recalculates sight path and holdover and evaluates empirical spin drift at its time of flight. It emits physical C_D and Reynolds for direct models or reference C_D for G1/G7.

25.13. 13. Run optional uncertainty

If uncertainty is enabled, the selected public output distances become the aligned range set for uncertainty results. Monte Carlo accepts advanced scenarios and repeatedly calls `integrate_shot_scenario`. First-order propagation is used only for the restricted homogeneous level case and repeatedly calls fixed-bore `integrate_trajectory`, except for zero-range perturbations.

The wrapper's result is optional per load. A warning is attached for partial coverage or unavailable baseline. Uncertainty does not alter the central trajectory stored for that load.

25.14. 14. Run optional buckshot pattern analysis

If a pattern configuration is present, the CLI locates its selected load by stable ID. It overwrites pattern pellet count with the load payload count and mean muzzle velocity with the trajectory's first ground-speed sample. It then calls `analyze_buckshot_pattern`. Only that selected load receives a pattern result.

An out-of-calibration-range target produces an extrapolation warning but still returns the calculated distribution. A missing or invalid selected load is rejected earlier by request validation.

25.15. 15. Assemble the structured result

The response begins with protocol, engine, and model identity plus request ID and issues. It then writes firing-point atmosphere identity, scenario features, and one load result per built-in or custom load. Each load contains definition, trajectory coverage, solver diagnostics, drag validity, MPBR, zeroing, spin metadata, event analysis, optional uncertainty, optional pattern analysis, and public points.

Numbers are written with fifteen significant digits. Optional unavailable values are JSON null rather than invented zeros. Enumerated statuses become stable string values. A newline terminates the one-response document.

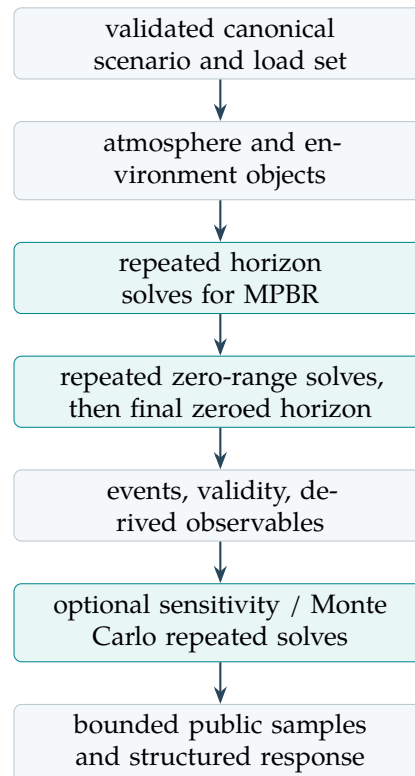


Figure 25.1.: Per-load calculation sequence at the native response boundary. Blue blocks contain repeated calls to the same deterministic trajectory solver.

26. Repeated solves, bounds, and failure containment

Physical interpretation

Reliable calculation requires more than implementing equations. The core must prevent invalid divisions, bound work, localize crossings, preserve partial results, and keep failures in one repeated solve from being mistaken for complete evidence.

26.1. Validation before evaluation

Projectile validation checks positive mass, finite muzzle velocity from 1 to 2,000 m/s, payload count 1 to 1000, positive optional geometry, and the active drag model's own invariants. A sphere diameter lies from 1 to 50 mm and material density from 500 to 25,000 kg/m³; its mass must match geometry. A Mach table has two to sixty-four ordered knots. A reference BC is positive and no greater than 2; schedules begin at zero and are ordered.

Environment construction validates thermodynamic ranges and wind. Layer validation verifies finite endpoints and positive interval width on every local atmosphere evaluation. Solver validation checks tolerances, time steps, angle domains, ground height, work count, and positive drag multiplier.

These overlapping checks are deliberate. The protocol prevents invalid external data; public C++ functions still protect themselves when called directly by tests or future adapters.

26.2. Avoiding invalid divisions

Several denominators receive explicit treatment:

- Drag direction uses zero scale when $v_a \leq 10^{-12}$ rather than dividing by airspeed.
- Range-event slope uses zero when $|v_x| \leq 10^{-12}$.
- The validation-only range solver uses $\max(v_x, 10^{-6})$ because its formulation is singular at reversal.
- Matrix solves reject pivots below 10^{-14} .
- Cholesky handles near-zero diagonals only when the remaining row is consistent.
- Sphere Bezier Newton steps fall back to a bracket midpoint when the derivative is unusable.
- Binomial edge probabilities bypass logarithms of zero.

Each threshold belongs to a local numerical problem. They should not be combined into one generic epsilon because their units and consequences differ.

26.3. Bracketing rather than unbounded iteration

The core uses bisection when a continuous scalar event can be bracketed: exact range inside a time step, horizontal reversal, ground crossing, sight zero, path roots, apex derivative, MPBR objective, and post-apex crossings. Bisection trades superlinear speed for monotone, deterministic convergence.

Sphere Bezier inversion is the main hybrid: safeguarded Newton provides rapid convergence while a persistent bracket and fixed bisection fallback provide certainty. LM is also safeguarded, but by objective acceptance and damping rather than a scalar sign bracket.

26.4. Finite states and partial trajectories

A non-finite Dormand–Prince endpoint is never accepted. A trajectory that ends for time, work, speed, reversal, ground, or non-finite state retains only valid samples through its covered range. The result records both requested and covered distance and a typed termination.

Downstream code asks `sample_at` for an optional value. Zeroing returns range unavailable if the trial cannot reach its target. Calibration converts a missing prediction into an infinite objective and solver-failure status. First-order propagation marks rows unavailable. Monte Carlo rejects the entire draw. This propagation of absence prevents a missing state from turning into a default numerical zero.

26.5. Determinism

For fixed inputs and platform arithmetic, deterministic calculations contain no hidden clock or random source. Adaptive decisions depend only on computed state and configuration. Monte Carlo randomness is seeded and generated by a specified internal algorithm. Quantile interpolation and split diagnostics are deterministic. Response load order is built-ins followed by request custom loads.

Determinism helps reproduce bugs and compare model versions. It does not guarantee bitwise identity across every compiler and architecture, because elementary math functions and floating-point contraction can differ slightly. Cross-platform validation therefore uses tolerances and records compiler identity.

26.6. Bounding computational work

Several nested layers have independent caps:

- two million attempted trajectory steps and 120 seconds modeled flight time;
- maximum integration time step 0.05 s and minimum 1×10^{-8} s;
- fixed iteration counts for Hermite roots and Bezier fallback;
- fifty zero bisections and forty-five scenario-MPBR bisections;
- up to thirty default LM iterations, eight damping trials each, and at most thirty-two observations;
- Monte Carlo sample count from 100 to 10000;
- at most sixteen wind layers, sixty-four Mach knots, and sixty-four pattern observations;
- roughly five hundred serialized trajectory points per load.

These are not all accuracy parameters. Some limit worst-case work or response size; others set deterministic numerical resolution. The documentation identifies their separate purpose rather than treating a larger bound as automatically more accurate.

26.7. Numerical diagnostics as part of the result

A completed path still carries attempted, accepted, and rejected counts; accepted-step extrema; final time step; and maximum error norm. Drag validity carries encoded and observed domains. Root-finding layers carry zero error or status. Uncertainty carries completed input or sample counts. Calibration carries iterations and objective-evaluation count. Pattern analysis carries residuals and range status.

These values make the calculation explainable from within the program. A high rejected-step count near a transonic feature is different from a missing far zero beyond the horizon, and both are different from a sphere correlation outside declared Reynolds range.

I read the diagnostics as four distinct families: step-control evidence, coverage evidence, model-domain evidence, and repeated-solve evidence. They should not be collapsed into one success flag. A trajectory can be numerically complete while its sphere correlation is extrapolated; a Monte Carlo result can be partial even though every retained deterministic trajectory is individually finite; and an event set can be horizon-limited after a fully successful solve to the requested horizon.

For the native A-MAX example, $(N_{\text{attempt}}, N_{\text{accept}}, N_{\text{reject}}) = (97, 47, 50)$ explains adaptation activity, while the 500 m covered range establishes event support. The maximum error norm of 141.67 belongs to rejected trials and cannot be read as the error of the accepted endpoint. Keeping these fields together but semantically separate makes that distinction inspectable.

27. Source-to-equation landmarks

Physical interpretation

The following map connects the mathematical chapters to the functions that evaluate them. It is a navigation aid, not an alternate specification.

Table 27.1.: Calculation features and implementation landmarks.

Feature	Source / function	Mathematical role
Moist atmosphere	<code>environment.cpp:</code> <code>Atmosphere::create</code>	Equations (4.7), (4.12), (4.13)
Altitude and layers	<code>environment.cpp:</code> <code>Environment::atmosphere_</code> <code>at</code>	Lapse pressure/temperature and Equation (6.2)
Gravity	<code>environment.cpp:</code> <code>gravity_at_height</code>	Equations (6.3)–(6.4)
Coriolis	<code>environment.cpp:</code> <code>coriolis_acceleration</code>	Equations (6.10)–(6.15)
G1 / G7 curves	<code>ballistics.cpp:</code> <code>g1_retardation,</code> <code>g1_drag_coefficient,</code> <code>g7_drag_coefficient</code>	Reference coefficient or retar- dation versus Mach
BC schedule	<code>ballistics.cpp:</code> <code>effective_</code> <code>ballistic_coefficient</code>	Equation (7.7)
Tabulated drag	<code>ballistics.cpp:</code> <code>tabulated_drag_coefficient</code>	Equation (8.1) with endpoint clamping
Sphere drag	<code>ballistics.cpp:</code> <code>sphere_drag_vs_reynolds,</code> <code>sphere_drag_coefficient</code>	Equations (8.4)–(8.9)
State derivative	<code>ballistics.cpp:</code> <code>time_derivative</code>	Equations (2.4)–(2.5)
Embedded step	<code>ballistics.cpp:</code> <code>dormand_prince_54,</code> <code>error_norm</code>	Equations (11.3)–(11.8)
Dense interpolation	<code>ballistics.cpp:</code> <code>hermite_position,</code> <code>interpolate_time_state</code>	Equations (12.1)–(12.3)
Solver loop	<code>ballistics.cpp:</code> <code>integrate_trajectory</code>	Adaptation, retained grid, and termination
Sight zero	<code>trajectory_analysis.cpp:</code> <code>integrate_shot_scenario</code>	Equation (13.3) and bisection
MPBR	<code>trajectory_analysis.cpp:</code> <code>compute_scenario_mpbr</code>	Equations (14.4)–(14.8)
Trajectory events	<code>trajectory_analysis.cpp:</code> <code>analyze_trajectory_events</code>	Range Hermite roots, apex, and Mach crossings
BC calibration	<code>calibration.cpp:</code> <code>calibrate_reference_</code> <code>ballistic_coefficient</code>	Equations (18.3)–(18.24)

Feature	Source / function	Mathematical role
First-order uncertainty	<code>uncertainty.cpp: propagate_ trajectory_uncertainty</code>	Equations (19.2)–(19.3)
Spin drift	<code>uncertainty.cpp: compute_spin_drift</code>	Equations (16.5)–(16.9)
Monte Carlo	<code>monte_carlo.cpp: propagate_ monte_carlo_uncertainty</code>	Cholesky sampling, repeated zeroing, and quantiles
Pattern analysis	<code>buckshot_pattern.cpp: analyze_buckshot_pattern</code>	D90 fit, spatial integration, and binomial counts
Native orchestration	<code>Ballistics.Cli/response_ writer.cpp: write_response</code>	Per-load sequence and struc- tured output

The map is intentionally many-to-one in both directions. Several source functions implement one mathematical chain. Atmosphere creation, local environment evaluation, and the derivative all participate in a single drag stage, for instance. Conversely, `integrate_trajectory` realizes adaptation, interpolation, retention, and termination rules described in several chapters. A one-row-per-file inventory would hide those relationships.

For change analysis I start at the mathematical role, follow the middle column to the owning function, and then inspect its callers. For a result discrepancy I traverse in the opposite direction: response field to orchestration, orchestration to analysis or solver, and solver to local physical evaluation. The table is therefore a dependency index, while the equations remain the specification of meaning.

Part IX.

Desktop workflow

28. From interface input to native result

Physical interpretation

React presents and validates the workbench state, Electron hosts the desktop boundary, and the native C++ executable performs the authoritative calculations. Structured JSON carries canonical inputs and typed results between those layers; it is transport, not the mathematical model.

28.1. Desktop calculation workflow

The renderer maintains environment, firearm, geometry, uncertainty, buckshot, and custom-load drafts. Unit selection affects input and display conversion, while calculation requests use canonical SI. A custom load is converted into exactly one drag shape: reference BC, tabulated Mach- C_D , or sphere. Sphere diameter is converted from millimetres to metres; reference and tabulated masses are converted from grams to kilograms; rifle geometry remains explicitly identified in inches because the Miller equation uses that convention.

The request builder supplies protocol version, request ID, display distance, a 2,000 m solution horizon in the normal interface, vital-zone diameter, sight geometry, atmosphere, firearm settings, optional uncertainty, optional pattern configuration, and every custom load. One native request evaluates all built-ins and custom loads, preserving their stable identities.

React does not implement a second ballistic solver. It converts units, constructs the scenario, validates the returned structure, and presents tables, charts, event summaries, statuses, and exports. The C++ result remains the authoritative source for trajectory, zero, MPBR, aerodynamic diagnostics, calibration, and uncertainty.

28.2. Calculation boundary

In the packaged application, Electron starts one `ballistics_cli` process for a request, writes the serialized protocol document to standard input, and collects a bounded standard-output response. The CLI parses, calculates, and emits one JSON document. Its small `main.cpp` selects structured standard-input mode when no legacy command-line arguments are present, returns structured parse errors with exit code 2, and returns a structured calculation failure with exit code 3 if an exception reaches the boundary.

The response identifies protocol, engine, model, and request ID. The renderer checks that the request ID matches the calculation it is awaiting and validates the expected result shape. This prevents a syntactically valid but unrelated response from being attached to the current draft.

28.3. Debouncing, cancellation, and stale results

Ordinary edits start a new calculation generation after a 180 ms debounce. Beginning a generation creates a new request ID and abort controller. Cleanup cancels the timer or propagates cancellation to Electron, which terminates the corresponding native child process. Electron also aborts any older active calculation from the same renderer before starting the new one.

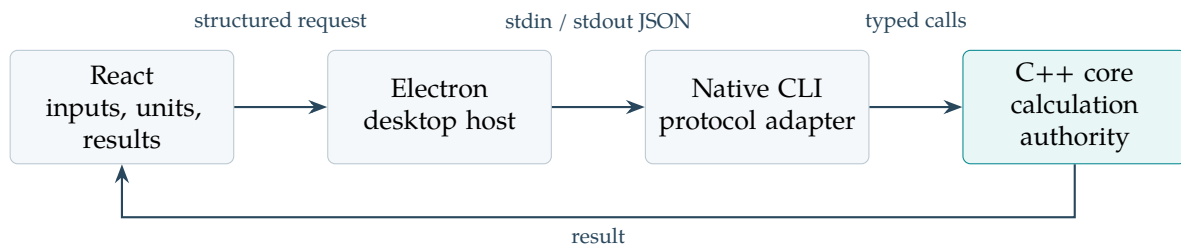


Figure 28.1.: Concise desktop calculation path. The surrounding application transports and presents the calculation; the C++ core evaluates it.

When a response arrives, the renderer accepts it only if its generation is still current and its signal has not been aborted. A late result from an older draft is ignored. If a new calculation fails after a previous valid result existed, the previous result can remain visible but is explicitly marked stale. Export is disabled for stale or busy state, so an old table is not silently presented as the current input’s result.

The native child has a 30-second host timeout. Calculation cancellation and timeout are application-level work controls; they do not change the core’s modeled-time limit of 120 seconds or its numerical maximum-step status.

28.4. Custom loads, profiles, and export

Custom load editing exposes the active aerodynamic family and the data it requires. Reference-BC loads can use constant or velocity-banded coefficients. Direct tables carry explicit Mach knots and reference diameter. Spheres carry diameter and material density, from which mass is derived. Portable drag-data documents preserve these distinctions and their source text.

Named environment, firearm, ammunition, and combined-scenario profiles save input configurations. Profiles and local persistence are conveniences around scenario construction, not inputs hidden from the calculation. A calculated request carries the complete scenario values and complete custom-load definitions the engine needs.

Results can be viewed in selected presentation units and exported to CSV. Conversions occur after native SI calculation. Payload totals in presentation are derived from per-projectile values and payload count; the native trajectory points remain per projectile.

Model limitation

Field names, profile files, and process mechanics do not define the physics. The equations are what the calculation actually applies; the surrounding layers only move inputs and results between the interface and the numerical core. A change to how those values are transported, with no change to the numerical method, is not a change to the model.

Part X.

Verification and model limitations

29. How I verify the calculations

Physical interpretation

I check different claims with different kinds of tests: mathematical identities, convergence under tighter numerical settings, comparison with separately implemented solvers, comparison with physical-property references, reproduction of source tables, parameter-recovery tests, and protocol robustness. No single comparison establishes every part of the model.

29.1. Mathematical identity and analytical cases

The most direct tests remove enough complexity to obtain known results. Vacuum constant-gravity flight checks the signs and kinematics of the state equations. The constant- C_D horizontal case checks Equations (12.7) and (12.8). These cases isolate integration, exact-range sampling, and observables from uncertainties in a reference drag table.

Sphere mass is checked against Equation (8.3). Wind signs are checked by reversing headwind and crosswind. Momentum and energy are checked from their definitions. Miller stability tests include the inverse-square twist relationship and a published example. Pattern tests check D90-to- σ conversion, centered probabilities, weighted fit behavior, and binomial normalization.

An identity test can prove that code follows a particular equation at tested cases. It cannot show that the equation is an adequate model for every physical projectile.

29.2. Adaptive convergence

The convergence executable records production, half-tolerance, tenth-tolerance, and very tight reference solutions. It covers a standard G1 case, cold dense headwind, hot thin G7 crosswind, and low-velocity tabulated drag. The reference uses tolerances one thousand times tighter and a maximum step twenty-five times smaller than production.

The current MSVC report passes all declared budgets. Its maximum production-to-reference errors are reproduced in Chapter 12. The report also verifies that half-tolerance changes stay below the program's displayed resolution and that the tighter solution improves the registered comparison metrics.

Convergence establishes that the computed solution stabilizes as numerical controls tighten. It does not establish that BC, C_D , atmosphere, or empirical corrections match unobserved reality.

29.3. Independent trajectory implementations

One validation path compares production G7 trajectories with a separately implemented fixed-step RK4 solver in supersonic, transonic, and subsonic regimes. The current maximum relative velocity residual is 5.7491×10^{-7} and maximum absolute position residual is 2.2082×10^{-7} m.

A broader fixed-time RK4 flight matrix covers thirteen trajectories and sixty-five range samples across built-in loads, environment extremes, wind vectors, inclined geometry, low speed, tabulated drag, and sphere drag. It also includes an independent sight-zero case and an MPBR case.

The current maximum aggregate residuals are

position : 8.9135×10^{-6} m,
 velocity : 0.001237 m/s,
 time : 2.1351×10^{-6} s,
 drag coefficient : 7.7716×10^{-16} .

Because the comparison implementation has a different numerical path, agreement is stronger evidence than calling the production function twice. Both still share the declared physical equations and source data, so agreement does not test those assumptions externally.

29.4. Atmospheric properties

The atmosphere report compares ideal-mixture density with independently generated CIPM-2007 values, Cramer sound speed with published humid-air reference rows, and SI Sutherland viscosity with NACA Report 1135 values. The current checked results are summarized in Table 29.1.

Table 29.1.: Current atmosphere-property comparisons.

Property	Cases	Maximum relative residual	Tolerance
Density	9	4.5202×10^{-4}	10^{-3}
Speed of sound	4	3.5061×10^{-4}	10^{-3}
Dynamic viscosity	8	3.6071×10^{-4}	10^{-3}

The density comparison is agreement between different formulations over the declared comparison domain, not evidence that production implements CIPM-2007. The sound-speed set has only a small number of published cases. The viscosity comparison spans the full accepted temperature range but does not test viscosity sensor error.

29.5. Reference drag and manufacturer tables

G1 coefficients and G7 knots have checksummed source transcriptions. Tests pin interpolation at knots, reference conversions, band selection, and independent curve behavior. Manufacturer/publication comparisons evaluate five built-in loads under a documented assumed atmosphere because the source tables do not fully publish atmospheric conditions.

The current maximum relative velocity residual across those load-level reports is 0.007801. The meaning differs by load. Winchester X123RS15 and Hornady A-MAX use manufacturer-published G1 coefficients and are compared with their published velocity series. White Blackout HV, BlackShock, and Federal SP 150 use effective coefficients fitted to the same source values or series; close agreement for those loads is calibration reproduction, not independent validation.

29.6. Calibration recovery and uncertainty behavior

Synthetic calibration tests generate velocities from known coefficients and verify recovery for a constant G1 model and a two-band G7 model. Protocol tests exercise separate calibration and holdout residuals, confidence-bound availability, iteration counts, and model identity. The built-in fit runner invokes the real native protocol and checks that the fitted values reproduce the implemented coefficients.

First-order tests compare launch-point speed, energy, and momentum sensitivities with analytical derivatives, check root-sum-square accumulation, and verify that fixed-bore and re-zero semantics remain distinct. Monte Carlo tests check deterministic seeding, correlation construction, quantile ordering, completion, and full scenario re-zeroing. These tests verify implementation behavior under constructed distributions; they do not supply missing empirical uncertainty for a real load.

29.7. Buckshot pattern checks

Pattern tests cover the through-origin slope, calibration/holdout role separation, slope uncertainty, rectangular and circular probability, Simpson integration, binomial normalization, and stable edge cases. At least one holdout is structurally required. That requirement makes excluded residuals visible but does not guarantee that a supplied holdout is physically independent, sufficiently large, or representative of other ammunition conditions.

29.8. Protocol robustness

The native parser is tested with valid calculation and calibration fixtures, unknown fields, unsupported versions, duplicate keys, malformed numbers, invalid UTF-8, and bounded mutation schedules. A deterministic mutation target exercises twelve thousand altered documents; the fuzz target uses coverage feedback under sanitizers. The response path is checked for structured issues, bounded output, matching request identity, and current engine/model identity.

Protocol tests support the claim that malformed transport data are contained. They are peripheral to the physical model but important to ensuring that the core receives the values the calculation claims to use.

29.9. Current summary

The verification suite is organized into the families summarized in Table 29.2: seven that gate acceptance, all currently passing, and one benchmark family recorded for performance rather than for accuracy.

Table 29.2.: Current calculation verification summary.

Package	Current status	Principal scope
Independent G7 RK4	pass	Three Mach regimes; separate fixed-step solver
Independent flight matrix	pass	Thirteen trajectories plus zero and MPBR cases
Atmosphere properties	pass	Density, sound speed, and viscosity references
Adaptive convergence	pass	Analytical constant drag and tightened production scenarios
Manufacturer/publication tables	pass	Five loads under documented normalization
Built-in BC fits	pass	Three deterministic fit reproductions; no hold-outs
Native benchmarks	recorded	Performance measurement, not a physical accuracy claim

Interpreting agreement

Direct identity checks establish algebraic implementation. Tolerance refinement establishes numerical convergence. A separately implemented solver checks the same declared equations by another numerical route. A manufacturer-table comparison checks conformance under stated normalization. A fit to the same data checks calibration. Only physically separate excluded measurements act as empirical holdouts. These statements are complementary, not interchangeable grades of one generic “accuracy.”

The evidence therefore forms a set of claim boundaries rather than a single ranking. An identity test can isolate a sign or unit error with high precision but says nothing about omitted physics. A convergence test can show that discretization error is shrinking while converging to a model with an unsuitable drag input. An independent implementation reduces the chance of sharing control-flow mistakes, yet it can still share the same physical assumptions and source data.

A manufacturer comparison adds an external tabulation, but its atmosphere, velocity definition, firing geometry, and rounding must be normalized before residuals have meaning. A calibration recovery closes a different loop: it proves that the optimizer can reproduce the coefficient encoded by its own observation set. Only a measurement excluded from estimation can test prediction without that direct reuse.

I therefore report the evidence by type in Table 29.2 and preserve the corresponding residuals. The strongest defensible statement is specific, for example that the fixed-step G7 implementation agrees with the production equations over the registered matrix, rather than a context-free claim that the entire application has one scalar accuracy.

30. Scientific and numerical limitations

Physical interpretation

Ballistics Workbench makes bounded calculations under explicit models. The important limits are the physical mechanisms omitted, the domains and semantics of aerodynamic data, the numerical horizon and interpolation rules, and the conditional assumptions of estimation and probability.

30.1. Translational point-mass boundary

The integrated state contains translation only. It does not model projectile attitude, angular rates, six-degree-of-freedom moments, yaw-dependent drag, coning, pitch damping, dynamic stability evolution, aerodynamic jump, muzzle disturbance, internal ballistics, recoil, or barrel time. A bullet's geometry enters Miller stability, not a full rotational force and moment system.

The empirical spin correction is valid only for rifle loads with geometry and $1 \leq S_g \leq 3.5$ under the encoded formula. It uses firing-point density and muzzle velocity and does not feed back into drag or trajectory.

30.2. Local geometry and Earth model

The frame is a fixed local tangent coordinate system. Optional normal gravity varies scalar g with latitude and height; optional Coriolis acceleration uses geographic azimuth. The trajectory does not follow Earth curvature, a geodesic, or a rotating local basis. Terrain, arbitrary ground profiles, obstacles, and target motion are absent. Ground termination uses one configured horizontal height.

Sight line is straight in local coordinates. Bore and sight alignment have no mechanical flex, sight adjustment quantization, or optical parallax model. MPBR treats the vital zone as a vertical band around the line of sight, not a three-dimensional target shape.

30.3. Atmosphere and wind

Moist density is an ideal mixture rather than full CIPM-2007. Cramer sound speed fixes carbon dioxide at 400 ppm. Sutherland viscosity depends only on temperature. Altitude dependence is one standard lapse relation anchored at the firing point with constant relative humidity. Wind is homogeneous or the first active one-dimensional linear layer, indexed by height or downrange; there is no vertical wind, turbulence, gust time series, or multidimensional weather grid.

Model-domain flags are narrower than accepted atmosphere inputs. Continuing a finite calculation outside a declared correlation interval does not make its source support wider.

30.4. Aerodynamic data

G1 and G7 BCs are scale factors tied to their reference projectiles. A constant coefficient can fail to represent velocity-dependent behavior; bands introduce discontinuous parameter changes at thresholds and can be weakly identified. A fitted coefficient is conditional on its observations and scenario.

Tabulated Mach- C_D interpolation is linear. Outside knots, the nearest endpoint is held rather than a trend extrapolated; the result is flagged outside domain. Reynolds dependence is not part of this model even though diagnostics can report Re .

Sphere drag assumes a smooth, undeformed sphere and a nominal homogeneous density. The built-in buckshot pellet mass is a pure-lead-sphere derivation from nominal diameter and density. Real alloy, plating, deformation, roughness, wakes, and pellet interactions can change both mass and drag. Mach is clamped to the sphere correction domain and Reynolds behavior is continued under the formulas described in Chapter 8.

30.5. Numerical solution

Adaptive tolerance controls a local embedded estimate, not a proof of global error. Hermite exact-range output assumes monotone downrange motion inside each accepted interval. Public `sample_at` uses linear interpolation over the dense retained grid. Mach crossings use linear interpolation. MPBR maximum path is scanned on the retained grid and its final crossings are linear between samples.

Time, attempted-step, minimum-forward-speed, reversal, non-finite, and ground limits can end a trajectory before its requested distance. Event status distinguishes a complete crossing from a horizon-limited one. Extending a horizon can reveal a later event; it does not improve an aerodynamic model's validity.

The current diagnostics record a maximum error norm that includes rejected trials. A value greater than one therefore does not imply that the final state was accepted at that error; the accepted/rejected counts and step history must be interpreted together.

30.6. Calibration

BC calibration assumes observation ranges and velocities are correctly paired, standard deviations are meaningful, and G1/G7 with fixed thresholds is the selected model. LM can converge to a local solution, stop at a bound, reach its iteration limit, or lose covariance information through insufficient rank. Approximate confidence intervals use local Jacobian linearization and do not incorporate uncertain atmosphere or model discrepancy.

Holdout labels are honored computationally, but independence and data quality are properties of how measurements were obtained. A label alone cannot make reused or closely coupled data into external evidence.

30.7. Uncertainty

First-order propagation assumes small independent variations and a locally linear mapping. It fixes nominal bore for most inputs and is unavailable through the desktop for advanced scenarios. It returns standard deviations, not empirical distributions.

Monte Carlo assumes a multivariate normal standardized input vector before bounds. Cholesky

correlation represents linear correlation only. Physical clamping creates boundary mass. Failed or short trajectories are excluded, so heavy incompleteness can condition the empirical interval on successful solves. The split-quantile diagnostic measures one reproducible partition, not formal convergence.

Neither method adds uncertainty for omitted physics, reference-table provenance, bullet-to-bullet shape, sight alignment, or atmospheric representativeness unless such variation is encoded in the input standard deviations and model.

30.8. Buckshot probability

The pattern line is constrained through the origin and grows linearly with range. Choke, deformation class, mean velocity, and velocity spread are conditioning metadata rather than numerical predictors. Target-range status only compares with calibration range. The Gaussian assumes a centered circularly symmetric pattern; the binomial assumes independent identical pellet impacts. Shared aim error, shell-level pattern shifts, voids, rings, pellet correlation, and overdispersion are absent.

The D90 interval propagates fitted slope uncertainty only. Probability bounds sample nominal and two diameter endpoints rather than integrating a full parameter distribution. A holdout is required but does not eliminate these model assumptions.

30.9. Derived totals and interpretation

Energy and momentum are mechanical state observables, not models of terminal interaction. Payload totals are presentation multiplications over identical per-pellet results. The core does not solve a combined massive payload. Integrated lateral z includes any enabled wind and Coriolis response, while empirical spin drift remains a separate field and is not automatically combined.

30.10. Software boundary

The request-response protocol communicates calculated values and typed absence. It cannot make incomplete source data complete. Profiles preserve configured inputs but do not verify scientific provenance. Checksums on portable drag data establish integrity of that document, not truth of its curve. A successful result means the request was accepted and evaluated under the encoded model; its scientific meaning remains conditional on the choices described throughout this document.

Part XI.

Appendices

A. Symbols, frames, and units

This appendix gathers the notation used across the mathematical development. A symbol has one primary meaning unless a local derivation states otherwise. Vector quantities are bold, scalar magnitudes are italic, and SI units are canonical inside the calculation core. Angles enter the native trigonometric calculations in radians. The desktop converts display units before constructing a request and converts returned SI values only for presentation.

A.1. Coordinate and state symbols

Table A.1.: Coordinate, state, and kinematic notation.

Symbol	Meaning	Canonical unit
t	Elapsed flight time	s
x	Downrange position along the shot-axis horizontal projection	m
y	Vertical position, positive upward from the bore origin	m
z	Lateral position, positive to the right of the shot direction	m
$r = (x, y, z)$	Projectile position in the shot frame	m
v_x, v_y, v_z	Ground-relative velocity components	m/s
$v = (v_x, v_y, v_z)$	Ground-relative velocity vector	m/s
$v_g = \ v\ $	Scalar ground speed	m/s
w	Air-mass velocity in the shot frame	m/s
$v_a = v - w$	Projectile velocity relative to the air	m/s
$v_a = \ v_a\ $	Scalar airspeed used by aerodynamic models	m/s
$s =$ (x, y, z, v_x, v_y, v_z)	Six-component numerical state	mixed
$\dot{s} = f(t, s)$	State derivative returned by the physical model	mixed
θ	Bore elevation above the local horizontal	rad
ψ	Initial lateral azimuth in the shot frame	rad
A	Geographic shot azimuth, clockwise from north	rad
ϕ	Geographic latitude	rad
v_E, v_N, v_U	East, north, and up velocity components	m/s
a_E, a_N, a_U	East, north, and up acceleration components	m/s ²

A.2. Dimensional and sign audit conventions

Every state coordinate, derivative, and stored observable in the core has a canonical SI interpretation even when a source formula is conventionally published in other units. I treat a conversion as part of the model boundary: G1 retardation converts feet per second and feet per second squared explicitly, Miller stability converts kilograms and metres per second into grains and feet per second, and sphere geometry remains SI throughout. A unit conversion hidden inside a fitted coefficient would defeat dimensional review and is therefore avoided.

Vector signs are audited before scalar magnitudes. Headwind and crosswind are user-facing directional conventions, while the derivative requires an air-mass velocity vector; sight path is

signed relative to the sight line, while drop is $-y$; empirical spin drift has its own twist-direction sign and is not inserted into integrated z . Checking those transformations independently prevents a plausible positive magnitude from concealing a reversed physical direction.

The symbol tables also distinguish state from reference geometry. Changing h_s changes q and η without changing (r, v) , whereas changing ρ changes drag and therefore the state. This is the quickest dimensional test for deciding whether a quantity belongs inside the differential equation or in analysis after integration.

The signs in this table fix the interpretation of every lateral quantity. A positive crosswind input denotes wind blowing from left to right. The corresponding air-mass vector has positive z component, so subtracting it produces $v_{a,z} = v_z - w_C$. The implementation stores the equivalent expression as $v_z - w_C$ while it stores headwind as a positive opposing scalar, giving $v_{a,x} = v_x + w_H$. These are the same relative-motion rule under the two input conventions developed in Chapter 1.

A.3. Aerodynamic and projectile symbols

Table A.2.: Aerodynamic and projectile notation.

Symbol	Meaning	Canonical unit
m	Mass of the single projectile being integrated	kg
d	Projectile or aerodynamic reference diameter	m
$A_p = \pi d^2/4$	Circular frontal reference area	m ²
ρ	Local moist-air mass density	kg/m ³
ρ_m	Projectile material density for a derived sphere	kg/m ³
μ	Dynamic viscosity of air	Pa s
a	Local thermodynamic speed of sound	m/s
$M = v_a/a$	Mach number	dimensionless
$Re = \rho v_a d/\mu$	Reynolds number	dimensionless
C_D	Direct dimensionless drag coefficient	dimensionless
BC	G1 or G7 ballistic coefficient in the program's reference convention	dimensionless stored value
$G(M)$	Reference-projectile drag coefficient or retardation relation	model-dependent
F_D	Scalar aerodynamic drag magnitude	N
a_D	Vector drag acceleration	m/s ²
$E_k = mv_g^2/2$	Translational kinetic energy	J
$p = mv_g$	Linear-momentum magnitude	kg m/s
N	Payload or pellet count used for derived totals	count

The two meanings of density are deliberately distinguished. Air density ρ controls aerodynamic force. Material density ρ_m enters only when a sphere's mass is derived from its diameter and assumed composition. Similarly, A_p belongs to direct physical drag. The reference-BC formulation converts its one-inch reference area and equivalent reference mass into the reference retardation scaling rather than substituting the actual projectile area into Equation (2.8).

A.4. Atmospheric symbols

Table A.3.: Atmospheric notation.

Symbol	Meaning	Canonical unit
T_C	Celsius temperature supplied to the scenario	°C
T	Absolute thermodynamic temperature	K
P	Station pressure at the firing origin	Pa
P_0, T_0	Pressure and temperature at the local atmosphere origin	Pa, K
p_{sv}	Saturation vapor pressure	Pa
p_v	Water-vapor partial pressure	Pa
$p_d = P - p_v$	Dry-air partial pressure	Pa
H	Relative humidity as a fraction from zero to one	dimensionless
R_d, R_v	Specific gas constants for dry air and water vapor	J/(kg K)
x_w	Water-vapor mole fraction in the Cramer formulation	dimensionless
x_c	Carbon-dioxide mole fraction	dimensionless
L	Positive tropospheric temperature lapse rate	K/m
h	Height above the local firing origin, equal to state coordinate y	m
h_f	Geographic firing altitude used by local gravity	m
h_p	Pressure altitude	m
h_ρ	Density altitude	m

Station pressure and firing altitude are separate inputs because they answer different questions. Station pressure describes the actual thermodynamic state at the local origin and therefore enters density and sound speed. Firing altitude locates the origin geometrically for latitude/height gravity. The optional altitude-dependent atmosphere treats state y as a height increment relative to the supplied origin and evolves T and P from T_0 and P_0 ; it does not replace the station pressure with a standard-atmosphere pressure inferred from firing altitude.

A.5. Sight and event symbols

Table A.4.: Sight-geometry and event notation.

Symbol	Meaning	Canonical unit
h_s	Sight height above the bore origin	m
γ	Sight-line or target inclination above horizontal	rad
$y_s(x) = h_s + x \tan \gamma$	Sight-line height expressed at downrange coordinate x	m
$q(x) = y(x) - y_s(x)$	Sight-relative path	m
$d_y = y(x)$	Geometric vertical displacement from the bore origin	m
η	Angular holdover, derived from sight-relative path and range	rad
R_0	Configured sight-zero range	m
R_v	Vital-zone radius, one half of supplied diameter	m
R_{MPBR}	Maximum point-blank range at the lower path boundary	m
x_n, x_f	Near- and far-zero ranges	m

Symbol	Meaning	Canonical unit
x_a	Range of maximum ordinate	m
$q_{\max}$	Maximum sight-relative ordinate	m

A.6. Estimation and uncertainty symbols

Table A.5.: Estimation, uncertainty, and pattern notation.

Symbol	Meaning	Canonical unit
$v_{\text{obs},i}$	Observed velocity at range r_i	m/s
$v_{\text{pred},i}$	Velocity predicted by a trial trajectory	m/s
σ_i	Observation standard uncertainty	m/s
r_i	Normalized fit residual; also range in pattern-only context	dimensionless or m
β	Logarithms of fitted ballistic coefficients	dimensionless
J	Numerical residual Jacobian	dimensionless after residual scaling
λ	Levenberg–Marquardt damping parameter	dimensionless
χ^2	Weighted residual sum of squares	dimensionless
ν	Residual degrees of freedom	count
X_j, Y	An uncertain input and a derived output	output-dependent
σ_{X_j}, σ_Y	Input and propagated output standard uncertainty	matching variable
R	Correlation matrix in Monte Carlo analysis	dimensionless
L	Lower-triangular Cholesky factor satisfying $LL^T = R$	dimensionless
D_{90}	Pattern diameter containing 90 percent probability	m
α	Through-origin D90 growth slope	dimensionless
σ_p	Per-axis standard deviation of the isotropic pattern Gaussian	m
p_T	Probability that one pellet lies in the target region	dimensionless
K	Number of pellet hits in the target	count

B. Constants, defaults, and configured bounds

The tables in this appendix describe the present calculation identity, not universal recommendations. Some values are physical constants, some are numerical defaults, some delimit accepted inputs, and some bound computational work. Keeping those categories separate prevents an iteration cap from being mistaken for a physical domain or a display interval from being mistaken for solver accuracy.

B.1. Physical and atmospheric constants

Table B.1.: Physical and atmospheric constants encoded by the current model.

Quantity	Value	Computational use
Constant gravitational acceleration	9.80665 m/s ²	Default vertical acceleration and standard atmosphere
Earth angular speed	7.2921150×10^{-5} rad/s	Coriolis acceleration
Dry-air gas constant R_d	287.058 J/(kg K)	Ideal-mixture density
Water-vapor gas constant R_v	461.495 J/(kg K)	Ideal-mixture density
Universal gas constant	8.31447 J/(mol K)	Pressure-altitude exponent
Dry-air molar mass	0.0289644 kg/mol	Pressure-altitude exponent
Standard temperature	288.15 K	Pressure/altimeter reference
Standard sea-level pressure	1,013.25 hPa	Pressure-altitude reference
Standard lapse rate	0.0065 K/m	Altitude conversions and optional atmosphere
Sutherland reference viscosity	1.716×10^{-5} Pa s	Viscosity at the reference temperature
Sutherland reference temperature	273.15 K	Viscosity scaling
Sutherland constant	110.333 K	Viscosity scaling
Cramer carbon-dioxide fraction	400 ppm	Speed-of-sound composition term
Somigliana equatorial gravity	9.7803253359 m/s ²	Latitude-dependent gravity
Somigliana numerator coefficient	0.00193185265241	Latitude-dependent gravity
Earth eccentricity-squared term	0.00669437999013	Latitude-dependent gravity
Free-air gravity gradient	-3.086×10^{-6} s ⁻²	Linear height correction
Reference projectile diameter	1 in	G1/G7 BC reference geometry

The free-air gradient multiplies total height $h_f + y$, where h_f is firing altitude and y is the integrated vertical coordinate. The altitude-dependent atmosphere instead uses y relative to its supplied station state. This difference is intentional: gravity needs geographic height, whereas the local pressure evolution begins from the measured station condition.

B.2. Adaptive trajectory defaults

Table B.2.: Production adaptive-solver defaults.

Setting	Value	Meaning
Relative tolerance	5×10^{-10}	Scales with each state-component magnitude
Absolute position tolerance	5×10^{-10} m	Floor for x, y, z error scaling
Absolute velocity tolerance	5×10^{-9} m/s	Floor for v_x, v_y, v_z error scaling
Initial time step	1×10^{-3} s	First Dormand–Prince trial
Minimum time step	1×10^{-8} s	Lower adaptation bound
Maximum time step	0.05 s	Upper adaptation bound
Maximum flight time	120 s	Work and physical-horizon termination
Maximum attempted steps	2,000,000	Work bound
Minimum forward velocity	1 m/s	Low-forward-speed termination
Retained internal range interval	$\min(\Delta x_{\text{out}}, 0.25 \text{ m})$	Resolution available to later analysis
Range-root iterations	60	Bisection used inside dense localization
Safety factor	0.9	Adaptive step controller
Accepted growth range	0.2 to 5	Bounds accepted-step factor
Rejected shrink range	0.1 to 0.5	Bounds rejected-step factor

The retained interval is not the integrator step. The integrator chooses time steps from local error. After acceptance, dense interpolation can emit several exact-range retained states from one time interval or no retained state if the interval does not cross the next range coordinate. Public response downsampling is yet another layer. Those three grids (adaptive time, retained range, and presented range) are separate, and keeping them distinct is what makes the retained trajectory interpretable.

At the minimum time step, a finite trial is accepted even if its normalized embedded error remains above one. That rule permits progress through a difficult region while recording the large error diagnostic. It does not declare that the error target was satisfied. Non-finite states or derivatives still terminate rather than being accepted.

B.3. Nested numerical procedures

Table B.3.: Bounds and stopping scales for repeated-solve procedures.

Procedure	Present value	Role
Zero initial half-bracket	0.05 rad	Search around sight inclination
Zero absolute angular cap	0.4 rad	Bracket-expansion limit
Zero bisection iterations	50	Bore-elevation solve
Zero vertical error tolerance	1×10^{-8} m	Early convergence test
Scenario-MPBR initial bracket	$\theta_s - 0.01$ to $\theta_s + 0.05$	Search for upper-zone tangency
Scenario-MPBR absolute angular cap	0.4 rad	Bracket-expansion limit
Scenario-MPBR bisections	45	Upper-bound root
Scenario-MPBR path tolerance	1×10^{-8} m	Early convergence test

Procedure	Present value	Role
Calibration BC bounds	0.005 to 2.0	Bounds in physical BC space
Calibration log finite difference	10^{-4}	Numerical Jacobian step
Initial LM damping	10^{-3}	First regularization level
LM damping decrease	divide by 3	Accepted update
LM damping increase	multiply by 10	Rejected update
LM damping floor	10^{-12}	Lower numerical bound
LM failure threshold	10^{14}	Inability to obtain descent
Default LM iterations	30	Fit work bound
Damping trials per iteration	8	Candidate update bound
Monte Carlo samples	100 to 10,000	Accepted configured range
Default Monte Carlo seed	1,113,017,667	Decimal form of hexadecimal 0x42574d43
Circular pattern subintervals	1024	Even Simpson partition count

Calibration convergence also tests gradient magnitude, parameter-step magnitude, and relative objective improvement. They are described in Chapter 18; the table records the stable scales that determine the present implementation. The Monte Carlo seed makes repeated requests with identical inputs reproducible. It is not intended as a cryptographic source of randomness.

B.4. Encoded atmosphere and drag domains

Table B.4.: Selected declared or enforced model domains.

Model	Domain	Behavior or status
Ideal-mixture density comparison domain	15 °C to 27 °C, 60,000 Pa to 110,000 Pa	Outside values can still be calculated; provenance status leaves the checked domain
Cramer speed-of-sound comparison domain	0 °C to 30 °C, 75,000 Pa to 102,000 Pa	Formula remains evaluable subject to physical validation; declared check domain is reported
Sutherland viscosity comparison domain	−60 °C to 60 °C	Outside declared check range is distinguished from arithmetic failure
Pressure-altitude conversion	−1,000 m to 11,000 m	Altitude input is clamped before the tropospheric relation
Tabulated Mach- C_D	Supplied first through last Mach knots	Linear interpolation within; endpoint value is held beyond either end
Sphere compressibility correction	Encoded Mach interval of the correlation	Mach argument is clamped for the correction term
Reference BC schedule	Supplied velocity thresholds	First matching minimum-velocity band; lowest band continues below its threshold

For a tabulated direct-drag curve, endpoint holding is clamping, not extrapolation. For a banded BC schedule, using the lowest band below its threshold is a schedule continuation. For an at-

mosphere formula evaluated outside its declared comparison range, the code can still produce a finite number while reporting that its registered validity domain was exceeded. These cases are numerically distinct and the result status preserves that distinction.

C. Built-in projectile definitions

The built-in catalogue supplies complete calculation definitions, not merely names for interface rows. Each entry combines a projectile mass, muzzle velocity, drag representation, and the geometry required by optional analyses. A trajectory always integrates one projectile. Payload count affects presentation totals and, for the built-in buckshot entry, the later binomial pattern calculation.

C.1. Reference-BC projectiles

Table C.1.: Built-in reference-ballistic-coefficient loads.

Load	Mass (kg)	v_0 (m/s)	Curve	BC	Geometry
White Blackout	0.028000000000	575.000	G1	0.054624716532086	none
BlackShock	0.032000000000	455.000	G1	0.0709673760860212	none
Winchester X123RS15	0.028349523125	536.448	G1	0.0680000000000000	none
Hornady A-MAX 168 gr	0.010886216880	822.960	G1	0.4750000000000000	1.24 in long, 0.308 in diameter
Federal SP 150 gr	0.009719836500	859.536	G1	0.312368144835017	1.13 in long, 0.308 in diameter

The values above are stored in SI except for bullet geometry, whose inches are explicit because the Miller stability relation is conventionally parameterized in grains, inches, calibers, and feet per second. The trajectory model itself does not need length or actual diameter for G1/G7 reference drag. Those dimensions become relevant only to gyroscopic stability and empirical spin drift.

The first three entries intentionally have no projectile geometry. Their exterior trajectory remains computable, but a spin-drift request cannot manufacture length or diameter and therefore reports missing geometry. This is an example of typed partial capability: absence in one optional empirical layer does not invalidate the six-state point-mass trajectory.

C.2. Derived 00-buck sphere

The built-in 00-buck entry is a direct physical sphere model:

$$v_0 = 403.86 \text{ m/s}, \quad d = 0.008382 \text{ m}, \quad \rho_m = 11,340 \text{ kg/m}^3, \quad N = 9.$$

Its one-pellet volume and mass are

$$V_s = \frac{\pi d^3}{6} = \frac{\pi (0.008382)^3}{6} \approx 3.0835 \times 10^{-7} \text{ m}^3,$$

$$m_s = \rho_m V_s \approx 0.00349667 \text{ kg}.$$

The exact stored derived mass is 0.0034966699909749735 kg. The nominal payload mass displayed for nine identical pellets is therefore approximately

$$m_{\text{payload}} = 9m_s \approx 0.0314700 \text{ kg}.$$

That multiplication is never substituted back into the sphere drag equation. If it were, area would remain one-pellet area while inertia became nine-pellet inertia, creating a fictitious object with radically incorrect deceleration. I preserve the one-pellet physical model and use N only where a total or pellet-count distribution is requested.

C.2.1. Initial pellet observables

At the stored muzzle velocity, the per-pellet kinetic energy and momentum are

$$E_{k,0} = \frac{1}{2} (0.0034966699909749735) (403.86)^2 \approx 285.2 \text{ J},$$

$$p_0 = (0.0034966699909749735) (403.86) \approx 1.412 \text{ kg m/s}.$$

Nine-pellet presentation totals are nine times these values. They summarize a nominal shell payload but do not include pellet-to-pellet velocity variation, aerodynamic interaction, or pattern position.

C.3. Custom projectile invariants

A custom definition selects exactly one mutually exclusive aerodynamic representation:

1. a G1 or G7 reference curve with either one constant BC or an ordered velocity-band schedule;
2. a direct tabulated Mach- C_D curve with reference diameter and at least the required knot structure; or
3. a sphere with diameter and material density, from which mass and area can be derived consistently.

The mutual exclusion matters mathematically. A BC does not multiply a direct physical C_D , and a sphere correlation does not fall back to a G1 curve at a chosen Mach. Drag dispatch resolves the representation once and the derivative evaluator follows that model's own path.

The common invariants are finite positive mass and muzzle velocity, a nonempty identity for transport and display, and model-specific positive parameters. Mach knots must be strictly increasing and their coefficients finite and positive. BC bands are selected by velocity thresholds rather than interpolated between thresholds. Geometry, if present, must provide finite positive length and diameter. Twist rate belongs to the firearm configuration rather than to the projectile because the same projectile definition may be evaluated with different launch configurations.

D. Status meanings and partial results

Numerical software must distinguish a finite result from the reason a calculation ended. I therefore carry explicit statuses through the core and protocol. They avoid turning absence into an arbitrary zero and prevent a horizon-limited value from being presented as though a physical event had been located.

D.1. Trajectory termination

Table D.1.: Trajectory termination states.

Status	Meaning
requested_distance	Dense localization reached the requested downrange horizon. This is the ordinary complete termination.
ground_impact	Ground detection was enabled and the trajectory crossed $y = 0$ after beginning above it; the crossing was localized within the accepted time interval.
minimum_forward_speed	Forward component v_x fell to or below the configured 1 m/s floor.
maximum_time	Elapsed time reached the configured 120 s bound before the requested range.
maximum_steps	Attempted integration steps reached 2,000,000 before another terminal event.
horizontal_reversal	v_x crossed from positive to nonpositive; the reversal was localized. A range-indexed continuation is no longer single-valued in the required direction.
non_finite_state	A state, derivative, trial endpoint, or normalized error became non-finite, so integration stopped rather than propagating invalid arithmetic.

The order of checks is computationally relevant when more than one condition becomes true in one interval. The solver first protects global work and time bounds, then considers the requested range, localized horizontal reversal, localized ground impact, minimum forward speed, and maximum time according to its accepted-step flow. The status is therefore the reason selected by the implemented termination ordering, not a post hoc guess from the last sample.

D.2. Sight solutions and trajectory events

Table D.2.: Zeroing, MPBR, and event-analysis statuses.

Status	Meaning
complete	The required root or event set was resolved within its requested horizon and numerical procedure.

Status	Meaning
range_unavailable	The zero range could not be sampled from the available trajectory, commonly because integration terminated earlier.
no_solution	A valid sign-changing angular bracket could not be established or the requested geometry did not yield the defined root.
invalid_geometry	Sight or vital-zone geometry violates the prerequisites of the problem, such as sight height exceeding the MPBR radius.
horizon_limited	A valid partial event sequence exists, but the configured distance ended before the later boundary or event needed for completion.
baseline_unavailable	An analysis pass had no complete nominal trajectory on which to operate.
not_applicable	The event has no meaning for the initial state or model; for example, a trajectory beginning at or below Mach 1 has no later decelerating supersonic-range crossing to locate.

Near and far zero are properties of crossings actually found. They are not inferred solely from the configured zero range. A zeroed path can touch, cross, or fail to provide two distinct retained crossings under unusual geometry. Similarly, maximum ordinate is found from a path-slope condition, not merely by selecting the largest public graph point.

D.3. Drag and spin-drift validity

Table D.3.: Aerodynamic-domain and spin-drift statuses.

Status	Meaning
within_domain	All observed Mach/Reynolds values remained in the model's declared domain.
extrapolated	At least part of the trajectory required behavior outside a declared domain, including a schedule or correlation continuation identified as extrapolation.
not_declared	The active representation has no encoded domain against which the observed trajectory can be classified.
available	Required rifle twist and projectile geometry exist and the stability result lies in the empirical domain used by the spin formula.
missing_geometry	Projectile length or diameter is absent.
invalid_stability	Inputs make the Miller calculation non-finite or non-positive.
unstable	Calculated gyroscopic stability is below the encoded stability threshold.
outside_empirical_domain	Stability is finite but not in the interval for which the implemented empirical drift relation is reported as available.

Status	Meaning
<code>spin not_applicable</code>	The selected configuration does not call for the rifle spin-drift model.

An aerodynamic status describes the relation between the path and an encoded correlation domain; it does not alter a finite trajectory after the fact. The spin result is separate from integrated z. An unavailable spin correction therefore leaves wind and optional Coriolis response intact.

D.4. Estimation, uncertainty, and pattern status

Table D.4.: Statuses for repeated-solve and empirical analyses.

Status	Meaning
<code>calibration converged</code>	At least one encoded convergence criterion was satisfied with a valid parameter vector.
<code>maximum_iterations</code>	The bounded LM loop ended before a convergence test succeeded; the best current finite fit can still carry diagnostics.
<code>calibration insufficient_information</code>	Observation count, roles, parameterization, or degrees of freedom are inadequate for the requested inference.
<code>solver_failure</code>	Repeated trajectory predictions could not supply the residual vector required by the optimizer.
<code>uncertainty complete</code>	Every requested active perturbation or Monte Carlo sample requirement completed as defined.
<code>uncertainty partial</code>	Some requested components completed and others did not; contributions and counts identify the available subset.
<code>no_inputs</code>	No uncertainty standard deviations were active.
<code>uncertainty baseline_unavailable</code>	The nominal scenario could not supply the outputs to perturb.
<code>validated_in_domain</code>	Pattern target range lies within the calibration range and the fit has enough information.
<code>pattern extrapolated</code>	Target range exceeds the maximum range represented by the calibration observations.
<code>pattern insufficient_information</code>	The D90 fit or requested probability lacks the required observations or degrees of freedom.

Typed status is especially important for uncertainty. Assigning zero standard deviation to an unavailable perturbation would falsely claim certainty. A partial result instead reports which variables completed and combines only their valid contributions, leaving the missing work visible.

E. Algorithm reference

The main body presents the central routines as excerpts of the actual C++ core. This chapter collects them in one place: Table E.1 points to each listing and names the source function, so the algorithms can be read together without hunting through the chapters. The one algorithm not shown in the body, the buckshot pattern fit, is given here in full.

Table E.1.: Where each core algorithm is shown as real source.

Algorithm	Listing	Core function
Physical derivative evaluation	25.1	<code>time_derivative</code>
Adaptive Dormand–Prince loop	11.1	<code>integrate_trajectory</code>
Bore-elevation zero solution	13.1	<code>integrate_zeroed_trajectory</code>
Ballistic-coefficient calibration	18.1	<code>calibrate_reference_ballistic_coefficient</code>
Correlated Monte Carlo wrapper	20.1	<code>propagate_monte_carlo_uncertainty</code>
Buckshot pattern fit	E.1	<code>analyze_buckshot_pattern</code>

E.1. Derivative, solver, and zeroing

The physical derivative in Listing 25.1 performs the most frequently repeated calculation: it maps one six-component state and the environment to a state derivative. The reference-BC path returns a scalar retardation after applying the active coefficient, while direct tabulated and sphere paths first compute C_D and then use density, physical reference area, mass, and v_a^2 . The common vector operation is the final projection opposite v_a , which is why a crosswind changes both scalar airspeed and the direction of aerodynamic acceleration.

The adaptive loop in Listing 11.1 advances that derivative in time. The fifth-order endpoint is the accepted solution; the fourth-order combination supplies a same-stage comparison rather than a second full solve, and the exponent $-1/5$ in the step controller corresponds to control from the embedded error estimate. The stage coefficients themselves are the Butcher tableau of Chapter 11.

The zero solution in Listing 13.1 wraps the solver in a bracket-and-bisect root find on bore elevation. Ground termination is disabled for zero trials because a below-sight trajectory can cross the bore-height plane before the requested zero range even though the elevation root exists. The bore angle it returns then drives the final scenario trajectory under the requested termination configuration.

E.2. Calibration and Monte Carlo

Calibration, Listing 18.1, is a bounded Levenberg–Marquardt fit in log-BC space. Every residual evaluation runs the trajectory to an observation range and extracts predicted velocity; holdout observations are scored afterward and never contribute rows to the Jacobian, the gradient, or the normal matrix.

The Monte Carlo wrapper, Listing 20.1, draws a correlated normal vector through a Cholesky

factor, clamps each physical input to its accepted domain, and re-zeroes and integrates a complete scenario per draw. The drag perturbation preserves one latent interpretation, that positive δ_D means more drag: a direct C_D grows with $\exp(\delta_D)$, whereas BC, an inverse measure of reference deceleration, shrinks with $\exp(-\delta_D)$.

E.3. Buckshot pattern fit

The pattern analysis is the one core routine with no counterpart in the body, because it never calls the trajectory solver. Listing E.1 shows its two numerical steps: the weighted through-origin slope estimate of Equation (21.4), and the log-domain binomial pellet-count distribution of Equation (23.5).

Listing E.1: The slope estimate and pellet-count distribution from `analyze_buckshot_pattern`, condensed. The two pieces are independent: the first fits the pattern spread, the second turns a one-pellet probability into a count distribution.

```
// Weighted through-origin slope: alpha = sum(w r D) / sum(w r^2), weight w = N / s^2.
double weighted_range_diameter = 0.0;
double weighted_range_squared = 0.0;
for (const auto& observation : input.observations)
{
    if (observation.role != ObservationRole::calibration) continue;
    const auto weight = static_cast<double>(observation.shell_count) /
        (observation.standard_uncertainty_m * observation.standard_uncertainty_m);
    weighted_range_diameter += weight * observation.range_m * observation.diameter_90_m;
    weighted_range_squared += weight * observation.range_m * observation.range_m;
}
const auto angular_diameter = weighted_range_diameter / weighted_range_squared;

// Pellet-count distribution: binomial evaluated in the log domain, then normalized.
std::vector<double> result(count + 1);
for (std::size_t hits = 0; hits <= count; ++hits)
{
    const auto log_probability =
        std::lgamma(count + 1.0) - std::lgamma(hits + 1.0) - std::lgamma(count - hits + 1.0)
        + hits * std::log(probability) + (count - hits) * std::log(1 - probability);
    result[hits] = std::exp(log_probability);
}
const auto total = std::accumulate(result.begin(), result.end(), 0.0);
for (auto& item : result) item /= total; // remove accumulated roundoff
```

The through-origin regression has only one numerical predictor, range. Choke class, pellet deformation, average velocity, and velocity spread travel with the analysis as conditioning metadata; they do not enter the slope. The count step uses `log(1 - probability)` so the $(1 - p)$ factor stays accurate when p is small, and it assigns all mass to $K = 0$ or $K = N$ at the degenerate $p \leq 0$ and $p \geq 1$ limits.

F. End-to-end native calculation example

This example follows the built-in Hornady A-MAX 168 gr definition through an actual native protocol fixture. It is not a hand-selected table intended to replace the derivations. Its purpose is to connect the quantities in those derivations to one coherent run of the present executable.

F.1. Scenario and projectile

The calculation request uses a 100 m display distance and a 500 m solution horizon. The rifle sight height is 0.04 m, the sight zero is 100 m, target inclination is zero, and the vital-zone diameter is 0.20 m. The atmosphere is homogeneous:

$$T_C = 15\text{ }^\circ\text{C}, \quad P = 1,013.25\text{ hPa}, \quad H = 50\%, \quad w_H = w_C = 0.$$

The rifle twist is 10 in with right-hand direction. First-order uncertainty activates two inputs: rifle muzzle-velocity standard deviation 3 m/s and relative drag standard deviation 0.02. All other standard deviations in the fixture are zero.

The projectile definition supplies

$$m = 0.01088621688\text{ kg}, \quad v_0 = 822.96\text{ m/s}, \quad BC_{G1} = 0.475,$$

with length 1.24 in and diameter 0.308 in. The G1 representation means the trajectory derivative will evaluate the active G1 reference retardation rather than a direct physical area and C_D for this projectile.

F.2. Atmospheric state

From the supplied thermodynamic inputs, the native result reports

$$\begin{aligned} \rho &= 1.22108938980701\text{ kg/m}^3, \\ a &= 340.875627046343\text{ m/s}, \\ \mu &= 1.78928586035969 \times 10^{-5}\text{ Pa s}. \end{aligned}$$

All three values are inside their corresponding declared comparison domains. The initial Mach number is consequently

$$M_0 = \frac{822.96}{340.875627046343} = 2.41425298467618.$$

Because the wind is zero, ground speed and airspeed begin equal and remain equal in this homogeneous calm example. That equality is a property of this scenario, not an identity in the model.

The initial G1 reference coefficient reported with the public point is 0.536213756472562. The stored projectile BC scales the associated reference retardation. It is therefore wrong to substitute 0.5362 and the projectile's 0.308 in area into the direct physical-drag formula: that would mix two different aerodynamic representations.

F.3. Zero solution and initial state

The zeroing loop varies bore elevation until the integrated height at 100 m equals the horizontal sight-line height 0.04 m. It returns

$$\theta_b = 0.00116295702755451 \text{ rad}$$

with vertical residual

$$\varepsilon_0 = 5.35560838038629 \times 10^{-9} \text{ m.}$$

For zero lateral azimuth, the six-component muzzle state is approximately

$$s(0) = (0, 0, 0, 822.9594434861, 0.9570668997, 0).$$

The small upward velocity is not a separate “zero correction.” It is the vertical component of the muzzle velocity after rotating the bore axis through θ_b . Gravity and drag immediately begin changing it when the solver evaluates the first stage.

At the muzzle, sight-relative path is

$$q(0) = y(0) - y_s(0) = 0 - 0.04 = -0.04 \text{ m.}$$

This explains why the trajectory has a near crossing before the configured 100 m far crossing. The projectile begins below the sight line, rises through it, reaches a small positive maximum ordinate, and falls back through it at the configured zero.

F.4. Adaptive integration record

The final 100 m zeroed trajectory required 97 attempted Dormand–Prince steps: 47 were accepted and 50 rejected. The smallest accepted step was

$$h_{\min, \text{accepted}} = 0.000274633838784758 \text{ s,}$$

while the largest accepted and final steps were both 0.05 s. The maximum recorded normalized error was 141.672505970865. That maximum includes rejected trials, so it does not mean a state with that error norm was ordinarily accepted. A rejected trial is diagnostic evidence that the controller reduced its time step and tried again.

The public output points at 50 and 100 m are exact-range samples obtained from the retained trajectory. They need not coincide with Dormand–Prince endpoints. The sample operation interpolates the state and then recomputes its derived speed, energy, momentum, Mach, and path consistently at that location.

Table F.1.: Selected exact-range values from the native A-MAX fixture.

x (m)	t (s)	v_g (m/s)	M	E_k (J)	q (m)
0	0.000000000	822.9600	2.414253	3686.4168	-0.0400000000
50	0.061947278	791.6360	2.322360	3411.1287	-0.0004274119
100	0.126366605	761.0101	2.232515	3152.3022	5.3556×10^{-9}

The corresponding momentum magnitudes are 8.9589210436 kg m/s, 8.6179216285 kg m/s, and 8.2845211050 kg m/s. Recomputing the 100 m energy provides a direct consistency check:

$$\frac{1}{2}(0.01088621688)(761.010109967768)^2 = 3,152.3021586 \text{ J.}$$

F.5. Drop, path, and holdover

The zeroed result makes the distinction between geometric drop and path concrete. At 100 m, the reported signed drop is

$$d(100) = -0.0400000053556084 \text{ m},$$

which means that the upward-positive state coordinate is

$$y(100) = -d(100) = 0.0400000053556084 \text{ m}.$$

The horizontal sight line is 0.04 m above the bore origin, so the sight-relative construction encoded by the program produces a path residual of only

$$q(100) = 5.35560838038629 \times 10^{-9} \text{ m}.$$

Its angular holdover is correspondingly near zero,

$$\eta(100) = -5.35560838038629 \times 10^{-11} \text{ rad}.$$

At 50 m, path remains $-0.000427411878 \text{ m}$, giving a positive holdover magnitude $8.548237565 \times 10^{-6} \text{ rad}$ under the implemented sign convention.

The calm trajectory has zero integrated lateral displacement. The spin module separately reports rightward empirical displacements of approximately 0.00069225 m at 50 m and 0.00255182 m at 100 m. These numbers are not added to the public integrated z field. They remain separately attributable to the Miller/Litz empirical layer.

F.6. Localized trajectory events

The event-analysis pass integrates to the 500 m solution horizon even though the displayed table ends at 100 m. It localizes the following sight-geometry results:

$$\begin{aligned} x_n &= 51.0635265844 \text{ m}, \\ x_f &= 100.0000132623 \text{ m}, \\ x_a &= 75.6890770556 \text{ m}, \\ q_{\max} &= 0.00487703622815 \text{ m}, \\ x_{\text{ground}} &= 148.479656034 \text{ m}. \end{aligned}$$

The far-zero localization differs slightly from the configured range because event detection and zero root finding have different interpolation paths and tolerances. Both are consistent at the micrometre-to-millimetre scales relevant to their algorithms: the direct zero residual is about five nanometres, while the event range differs by about thirteen micrometres.

No Mach 1.2, 1.0, or 0.8 crossing appears before the available event horizon for this trajectory, and supersonic range is reported as horizon-limited rather than set equal to 500 m. That status carries more information than a fabricated terminal event: the projectile is still supersonic at the analysis horizon, so the crossing lies beyond what was solved.

F.7. MPBR as a separate optimized geometry

The 0.20 m vital-zone diameter corresponds to radius

$$R_v = 0.10 \text{ m}.$$

The MPBR routine does not reuse the configured 100 m zero as though it were already optimal. It varies bore elevation to find a path whose maximum ordinate reaches $+R_v$, then follows that optimized trajectory to its later $-R_v$ crossing. For this fixture it reports an optimized zero range

$$R_{0,\text{MPBR}} = 233.023764460634 \text{ m}$$

and

$$R_{\text{MPBR}} = 274.059087420167 \text{ m}.$$

Those values describe a different bore-angle problem from the ordinary 100 m zero used in Table F.1. The result structure carries both without conflating them.

F.8. First-order uncertainty at the output range

The two active perturbations complete. At 100 m, the propagated standard deviations include

$$\begin{aligned}\sigma_v &= 3.11221473689 \text{ m/s}, \\ \sigma_E &= 25.7832083356 \text{ J}, \\ \sigma_p &= 0.03388024460 \text{ kg m/s}, \\ \sigma_t &= 0.0004790464735 \text{ s}, \\ \sigma_q &= 0.000568314436 \text{ m}, \\ \sigma_\eta &= 5.6831443592 \times 10^{-6} \text{ rad}.\end{aligned}$$

These are local independent-input results around the nominal scenario. The drag and muzzle-velocity contributions are combined in quadrature; the method does not sample nonlinearity, impose a probability distribution beyond the local standard-deviation interpretation, or include model-form error.

At the muzzle, only muzzle velocity can influence energy and momentum in this fixture. The energy sensitivity is

$$\frac{\partial E}{\partial v} = mv = (0.01088621688)(822.96) = 8.9589210436 \text{ kg m/s},$$

so its contribution is

$$\sigma_{E,v} = \left| \frac{\partial E}{\partial v} \right| \sigma_v = (8.9589210436)(3) = 26.8767631307 \text{ J},$$

exactly matching the reported muzzle energy standard deviation to displayed precision. This identity is a useful check on the finite-difference uncertainty path because the underlying muzzle energy formula is analytically differentiable.

F.9. What the example demonstrates

One request activates several distinct numerical layers:

1. atmosphere formulas turn measured conditions into density, sound speed, and viscosity;
2. zeroing repeatedly invokes the trajectory solver until a sight-line residual vanishes;
3. the final adaptive solve advances the six-state ODE and retains exact-range states;
4. an analysis pass localizes zeros, maximum ordinate, ground intersection, and Mach-related status;

5. a separate MPBR root search solves a different bore-angle problem;
6. the spin model adds a separate empirical displacement without changing the point-mass state; and
7. first-order uncertainty repeats perturbed solves and combines local sensitivities.

The table visible in the desktop is therefore the final projection of a larger calculation graph. The fact that these layers share a projectile and atmosphere does not make them one algorithm; each has its own unknown, convergence condition, status, and interpretation.

F.10. Reproducibility boundary

These numerical values belong to the checked fixture inputs stated in Section F.1. Reusing only the projectile name is insufficient: atmosphere, horizon, sight geometry, solver settings, and optional analysis configuration are all part of the calculation, and a result is reproducible only when the complete scenario is specified.

Within the same implementation and inputs, the deterministic trajectory, root searches, and first-order perturbations reproduce the values above to the reported precision. Cross-platform elementary functions can differ in their last bits, so validation uses stated tolerances rather than requiring byte-identical floating-point text. A later model identity is allowed to change results when an equation, source table, constant, or numerical rule changes; such a change should be accompanied by updated registered evidence rather than silently inheriting these numbers.

G. Dimensional and limiting checks

Dimensional analysis cannot establish that a model is physically sufficient, but it catches many transcription and implementation errors before numerical testing. Limiting cases provide a second class of checks: the equations should reduce sensibly when one influence is removed.

G.1. Direct-drag dimensions

The direct vector acceleration is

$$a_D = -\frac{\rho C_D A_p}{2m} v_a v_a.$$

Since C_D is dimensionless,

$$\begin{aligned} \left[\frac{\rho A_p}{m} v_a v_a \right] &= \frac{(\text{kg/m}^3)(\text{m}^2)}{\text{kg}} (\text{m/s})(\text{m/s}) \\ &= \text{m/s}^2. \end{aligned}$$

The scalar factor v_a and vector v_a supply the expected squared-speed magnitude while retaining direction. Omitting either factor would produce the wrong velocity dependence or wrong dimensions.

If $v_a \rightarrow 0$, drag magnitude tends to zero quadratically. The vector normalization implicit in “opposite air velocity” must be guarded, however, because v_a/v_a is undefined at exactly zero. The implementation handles vanishing airspeed before dividing.

G.2. Vacuum and no-wind limits

In the formal limit $\rho \rightarrow 0$ with Coriolis disabled and constant gravity,

$$\dot{v}_x = 0, \quad \dot{v}_y = -g, \quad \dot{v}_z = 0.$$

The analytical solution becomes

$$\begin{aligned} x(t) &= v_{x,0}t, \\ y(t) &= v_{y,0}t - \frac{1}{2}gt^2, \\ z(t) &= v_{z,0}t. \end{aligned}$$

This is a useful conceptual baseline even though the accepted atmosphere validator does not represent a true vacuum scenario. It shows which curvature is gravity-driven and which deceleration is aerodynamic.

When wind vanishes, $v_a = v$, so airspeed and ground speed coincide. When wind is nonzero, energy remains $mv_g^2/2$ even though drag uses v_a . A pure constant crosswind with $v_z = 0$ initially gives $v_{a,z} = -w_C$, and the negative multiplier in the drag vector yields positive lateral acceleration. The projectile is accelerated with the moving air mass, consistent with the rightward-positive convention.

G.3. Sight geometry limits

At $x = 0$, path is always $-h_s$ because the projectile starts at the bore origin. With $h_s = 0$ and horizontal sight inclination, path and vertical coordinate coincide. With gravity and drag removed,

choosing bore elevation equal to sight inclination makes the two straight lines parallel and produces no finite isolated zero unless they begin coincident. This explains why a sign-changing zero bracket is a property of the complete curved trajectory, not a guaranteed consequence of entering a range.

For MPBR, the geometry requires $h_s \leq R_v$. If the sight begins farther above the bore than the allowed radius, $q(0) = -h_s$ already lies below $-R_v$. The path cannot satisfy the formulation's requirement of remaining inside the vertical band from its origin, so the implementation reports invalid geometry rather than running an angle search with no valid premise.

G.4. Uncertainty limits

If every input standard deviation is zero, Equation (19.2) yields zero output variance, but the implementation reports `no_inputs` to distinguish “nothing requested” from an actively evaluated collection of exact-zero sensitivities. If one active uncertainty tends to zero while its derivative remains finite, its variance contribution tends to zero quadratically.

For a diagonal Monte Carlo correlation matrix $R = I$, its Cholesky factor is also I , so $q = Lz = z$ and the sampled standardized variables are independent. For a two-variable block

$$R_2 = \begin{bmatrix} 1 & \rho_{12} \\ \rho_{12} & 1 \end{bmatrix},$$

one valid lower factor is

$$L_2 = \begin{bmatrix} 1 & 0 \\ \rho_{12} & \sqrt{1 - \rho_{12}^2} \end{bmatrix}.$$

Then $q_1 = z_1$ and $q_2 = \rho_{12}z_1 + \sqrt{1 - \rho_{12}^2}z_2$, giving covariance $E[q_1q_2] = \rho_{12}$. This small case makes the seven-variable matrix operation transparent.

G.5. Pattern-probability limits

For a target probability $p_T = 0$, the binomial distribution puts all mass at $K = 0$. For $p_T = 1$, it puts all mass at $K = N$. The implementation handles these endpoints separately because direct evaluation of $k \log p_T + (N - k) \log(1 - p_T)$ would encounter $\log 0$. Between the endpoints, `log1p(-p)` retains numerical accuracy when p is small.

For a circular target radius $R \rightarrow 0$, the Gaussian area probability tends to zero. As $R \rightarrow \infty$, it tends to one. Substituting $R = D_{90}/2$ and $\sigma_p = D_{90}/(2 \times 2.145966026289347)$ gives

$$1 - \exp \left[-\frac{(2.145966026289347)^2}{2} \right] = 0.9,$$

which verifies the D90 conversion constant by construction.

H. Calculation source map

The file map below is a compact route from a mathematical question to its current implementation. It deliberately omits packaging, installer, and window-management detail. Line numbers would become stale with ordinary edits, so the stable landmarks are files, public types, and function names.

H.1. Core public model

Table H.1.: Public calculation types in `src/Ballistics.Core/include/ballistics.hpp`.

Type or function family	Calculation responsibility
<code>Vec3, State</code>	Three-component geometry and the six-component position/velocity state.
<code>ReferenceBcDrag</code>	G1/G7 curve identity plus constant or velocity-banded coefficient definition.
<code>TabulatedDrag</code>	Physical reference diameter and ordered Mach- C_D knots.
<code>SphereDrag</code>	Sphere diameter and material density used for mass, area, Reynolds number, and correlation selection.
<code>DragDefinition</code>	Mutually exclusive aerodynamic representation joining the three preceding paths.
<code>ProjectileDefinition</code>	Physical mass, optional bullet geometry, display identity, and active drag representation.
<code>AmmunitionLoad</code>	Muzzle velocity and payload count, separated from projectile physics.
<code>FirearmConfiguration</code>	Firearm group and optional twist rate for the empirical spin layer.
<code>Atmosphere, Environment</code>	Firing-point thermodynamic state, altitude behavior, wind layers, gravity, and Earth-rotation settings.
<code>TrajectoryOptions</code>	Solver mode, tolerances, step bounds, time/work bounds, retained interval, and termination configuration.
<code>TrajectorySample</code>	Range/time state interpreted with speed, airspeed, drop, energy, momentum, and aerodynamic diagnostics.
<code>Trajectory</code>	Ordered retained samples, covered distance, termination, diagnostics, and exact-range sampling operation.
<code>ShotScenario / geometry types</code>	Sight height, zero range, target inclination, bore-angle context, and environment.
<code>TrajectoryEvents</code>	Optional localized zero, ordinate, Mach, supersonic, and ground events with statuses.
<code>Calibration types</code>	Observations, roles, fixed thresholds, fitted values, covariance-derived intervals, residuals, and diagnostics.
<code>Uncertainty types</code>	Active variables, finite-difference or Monte Carlo settings, per-output intervals, contributions, and completion counts.

Type or function family	Calculation responsibility
Pattern types	D90 observations, target geometry, fit result, target probability, and pellet-count distribution.

The header is an interface for the calculation library; it is not where the numerical formulas are evaluated. Its importance is conceptual: it defines which physical facts are representable and which kinds of absence or partial completion can cross the core boundary.

H.2. Deterministic flight sources

Table H.2.: Deterministic physical-model and solver landmarks.

Source	Landmark	Role
<code>environment.cpp</code>	<code>Atmosphere::create</code>	Saturation pressure, moist ideal-mixture density, Cramer sound speed, Sutherland viscosity, finite validation, and domain flags.
<code>environment.cpp</code>	<code>Environment::atmosphere_at</code>	Homogeneous or lapse-rate local atmosphere evaluated at state height.
<code>environment.cpp</code>	<code>Environment::wind_at</code>	First-containing-layer selection and clamped linear interpolation by height or downrange.
<code>environment.cpp</code>	<code>gravity_at_height</code>	Constant gravity or Somigliana latitude gravity with free-air height correction.
<code>environment.cpp</code>	<code>coriolis_acceleration</code>	Shot-frame to east/north/up transformation, rotating-Earth acceleration, and inverse transformation.
<code>ballistics.cpp</code>	<code>g1_retardation</code>	Piecewise G1 reference retardation using the active descending velocity band.
<code>ballistics.cpp</code>	<code>g7_drag_coefficient</code>	McCoy G7 knot table and linear interpolation.
<code>ballistics.cpp</code>	<code>effective_ballistic_coefficient</code>	Constant coefficient or discrete velocity-band selection.
<code>ballistics.cpp</code>	<code>tabulated_drag_coefficient</code>	Linear Mach interpolation with nearest-endpoint holding outside the knots.
<code>ballistics.cpp</code>	<code>sphere_drag_vs_reynolds</code>	Morrison smooth-sphere base correlation.
<code>ballistics.cpp</code>	<code>bezier_y_from_x</code>	Safeguarded inversion used in Collins transonic correction curves.

Source	Landmark	Role
ballistics.cpp	sphere_drag_coefficient	Reynolds base coefficient plus Mach-dependent compressibility correction.
ballistics.cpp	time_derivative	Air-relative velocity and combined kinematic, drag, gravity, and optional Coriolis derivative.
ballistics.cpp	dormand_prince_54	Seven embedded stages and fifth-/fourth-order endpoint combinations.
ballistics.cpp	error_norm	Maximum component error normalized by absolute and relative scale.
ballistics.cpp	Hermite helpers	Continuous endpoint position/velocity interpolation and bisection in time.
ballistics.cpp	integrate_trajectory	Adaptive accept/reject loop, retained exact-range grid, and terminal event ordering.
ballistics.cpp	Trajectory::sample_at	Exact retained-grid lookup/interpolation and derived sample reconstruction.

H.3. Repeated-solve and analysis sources

Table H.3.: Analysis, estimation, and uncertainty landmarks.

Source	Landmark	Role
trajectory_analysis.cpp	integrate_shot_scenario	Bore-angle bracket expansion, bisection zero, and final zeroed trajectory.
trajectory_analysis.cpp	compute_scenario_mpbr	Upper-vital-bound angular root and later lower-bound range crossing.
trajectory_analysis.cpp	analyze_trajectory_events	Near/far zero, maximum ordinate, Mach thresholds, supersonic range, and ground intersection.
calibration.cpp	calibrate_reference_ballistic_coefficient	Log-BC residual evaluation, central Jacobian, bounded LM updates, diagnostics, and approximate intervals.
uncertainty.cpp	propagate_trajectory_uncertainty	Nominal solve, bounded paired perturbations, zero-range special semantics, and independent RSS propagation.

Source	Landmark	Role
<code>uncertainty.cpp</code>	<code>compute_spin_drift</code>	Miller stability, empirical-domain classification, Litz time relation, and twist-direction sign.
<code>monte_carlo.cpp</code>	random generator / Box–Muller helpers	Deterministic uniform sequence and paired standard normal values.
<code>monte_carlo.cpp</code>	Cholesky helpers	Correlation validation, positive-semidefinite factorization, and correlated normal vector construction.
<code>monte_carlo.cpp</code>	<code>propagate_monte_carlo_uncertainty</code>	Physical transforms, per-sample re-zeroing, complete scenario solve, empirical intervals, and split diagnostics.
<code>buckshot_pattern.cpp</code>	<code>analyze_buckshot_pattern</code>	Weighted D90 slope, residual statistics, target-region probability, Simpson integration, and stable binomial distribution.
<code>projectiles.cpp</code>	built-in catalogue / validation	Current physical definitions, sphere mass derivation, and custom-load invariants.

These files form layers around one central deterministic operation. Trajectory analysis consumes a trajectory and localizes geometry. Calibration changes aerodynamic parameters and creates many trajectories. First-order uncertainty changes one input at a time. Monte Carlo changes a vector of inputs at once. Pattern analysis is the exception: it operates on observed spread data and target geometry rather than on the flight state.

H.4. Native request and result path

Table H.4.: Calculation-oriented native boundary.

Source	Role
<code>src/Ballistics.Cli/protocol.hpp</code>	Typed request vocabulary and protocol-facing parse/result declarations.
<code>src/Ballistics.Cli/protocol.cpp</code>	Bounded parsing, units already normalized to SI, mutually exclusive custom-drag construction, validation issue collection, and optional-analysis settings.
<code>src/Ballistics.Cli/response_writer.cpp</code>	Atmosphere/environment assembly, per-load MPBR and zeroed solve, trajectory-event pass, validity classification, optional uncertainty/pattern work, point downsampling, and structured serialization.

Source	Role
<code>src/Ballistics.Cli/main.cpp</code>	Small standard-input/standard-output process boundary and structured top-level failure behavior.
<code>validation/manifest.json</code>	Registered source locators, normalized datasets, comparison classifications, and declared boundaries for calculation evidence.
<code>build/validation/VALIDATION_SUMMARY.md</code>	Current generated pass/fail and residual summary for the compiled native model.

The response writer is orchestration rather than an alternate physical solver. Its responsibility is to place the calculation layers in their required order and preserve their statuses. The mathematical authority remains in the core functions mapped above.

I. Reference sources

I use the following sources for numerical methods, atmospheric formulations, reference drag data, empirical corrections, and registered comparison tables. The repository's `validation/manifest.json` records retrieval dates, normalizations, checksums, and claim boundaries for the datasets that are transcribed into validation inputs. Manufacturer tables are comparison material for named built-ins; agreement with such a table is not automatically independent empirical validation.

I.1. Bibliography

- [1] O. Cramer, "The variation of the specific heat ratio and the speed of sound in air with temperature, pressure, humidity, and CO₂ concentration," *Journal of the Acoustical Society of America*, vol. 93, no. 5, 1993.
- [2] A. Picard, R. S. Davis, M. Glaser, and K. Fujii, "Revised formula for the density of moist air (CIPM-2007)," *Metrologia*, 2008. Registered comparison copy: <https://www.nist.gov/system/files/documents/calibrations/CIPM-2007.pdf>.
- [3] Ames Research Staff, *Equations, Tables, and Charts for Compressible Flow*, NACA Report 1135, National Advisory Committee for Aeronautics, 1953. <https://ntrs.nasa.gov/citations/19930091059>.
- [4] R. M. Gavioso, M. Astrua, M. Zucco, and M. Pisani, "Speed of sound in humid air: Accurate thermodynamic model and experimental validation," *Journal of Physical and Chemical Reference Data*, vol. 54, 043101, 2025. <https://doi.org/10.1063/5.0294663>.
- [5] J. R. Dormand and P. J. Prince, "A family of embedded Runge–Kutta formulae," *Journal of Computational and Applied Mathematics*, vol. 6, no. 1, pp. 19–26, 1980.
- [6] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., Springer, 1993.
- [7] D. Yates, *GNU Ballistics Library G1 Drag-Function Coefficients*, version 0.201 alpha. Registered transcription locator: <https://sourceforge.net/projects/ballisticslib/files/GNU%20Ballistics%20Library/0.201%20alpha/>.
- [8] R. L. McCoy, *Modern Exterior Ballistics: The Launch and Flight Dynamics of Symmetric Projectiles*, Schiffer Publishing, 1999. The registered G7 numerical transcription is cross-located at <https://jbmballistics.com/ballistics/downloads/text/mcg7.txt>.
- [9] py-ballisticcalc contributors, "TableG7," version 2.2.10. Independent validation transcription: https://raw.githubusercontent.com/o-murphy/py-ballisticcalc/v2.2.10/py_ballisticcalc/drag_tables.py.
- [10] F. A. Morrison, "Data correlation for drag coefficient for sphere," smooth-sphere Reynolds correlation summarized with the implemented combined model at <https://arc.id.au/CannonballDrag.html>.
- [11] Transonic sphere-drag correlation source registered for the Collins correction, *Journal of Fluid Mechanics*, 1979, <https://doi.org/10.1017/S0022112079002597>.
- [12] D. Miller, "A new rule for estimating rifling twist: An aid to choosing bullets and rifles," *Precision Shooting*, 2005.
- [13] B. Litz, *Applied Ballistics for Long Range Shooting*, Applied Ballistics, empirical gyroscopic stability and spin-drift treatment.

- [14] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Quarterly of Applied Mathematics*, vol. 2, pp. 164–168, 1944.
- [15] D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *Journal of the Society for Industrial and Applied Mathematics*, vol. 11, no. 2, pp. 431–441, 1963.
- [16] G. E. P. Box and M. E. Muller, "A note on the generation of random normal deviates," *Annals of Mathematical Statistics*, vol. 29, no. 2, pp. 610–611, 1958.
- [17] Hornady Manufacturing, 2022 *Standard Ballistics Chart: BLACK .308 Win 168 gr A-MAX, item 80971*, 2022. <https://static.hornady.media/presscenter/docs/1410998060-2022-Standard-Ballistics-Chart.pdf>.
- [18] Federal Premium Ammunition, *Federal Catalog 2022: Power-Shok 308A Rifle Ballistics*, 2022. https://www.federalpremium.com/on/demandware.static/-/Library-Sites-VistaFederalSharedLibrary/default/vd5051f299f79146229507233809e6144584db865/contentDocuments/catalog/Federal-Catalog-2022_website_sm.pdf.
- [19] Olin Winchester, 2026 *Online Catalog: X123RS15 Shotgun Ballistics*, 2026. <https://online.flippingbook.com/view/721179895/42/>.
- [20] Olin Winchester, *Law Enforcement 2019 Catalog: Super-X Buckshot XB1200*, 2019. <https://online.flippingbook.com/view/977082/13/>.
- [21] Olin Winchester, *Winchester Ammunition Academy: Shotgun Ammo 101*, 2022. <https://winchester.com/-/media/Project/Consumer/PDFs/Winchester-Ammunition-Academy-Shotgun-Ammo-101.ashx>.

I.2. How the references enter the program

The references serve different roles. Cramer supplies the polynomial implemented for speed of sound, while CIPM-2007 and the Gavioso data supply comparisons against different or more complete formulations. NACA Report 1135 supplies viscosity values generated from Sutherland's equation. Dormand and Prince supply the embedded integration pair. McCoy and the registered G1 transcription supply reference-projectile drag data. Morrison and Collins define the combined sphere correlation. Miller and Litz underlie an optional empirical correction rather than the integrated point-mass equations.

Manufacturer catalogues supply named ammunition values and finite tables. For Hornady and Winchester X123RS15, the stored BC is published with the named entry and the native checks compare the resulting trajectory with the tabulated velocities. Other built-in effective coefficients are fitted to their source series and therefore cannot use agreement with the same rows as independent validation. The verification chapter keeps this distinction explicit.

J. Closing model statement

BALLISTICS WORKBENCH turns a declared exterior-flight model into a deterministic numerical trajectory, then builds geometric, inferential, and stochastic analyses around that trajectory. Its strongest guarantee is specificity: the state, forces, atmospheric formulas, drag representation, numerical tolerances, root definitions, probability assumptions, and termination statuses are all explicit.

I have kept the calculation layers separable so that their meanings remain visible. Direct drag is not reference BC. Integrated lateral motion is not empirical spin drift. An accepted ODE step is not a graph point. A graph point is not an event root. A zero solution is not an MPBR solution. A local linearized uncertainty is not a Monte Carlo interval. A fitted coefficient is not automatically a holdout result. Those distinctions are the conceptual structure of the program as much as its equations are.

The equations and algorithms described here are the ones used to transform a supplied physical scenario into the reported result. Read together, they define the model completely: the forces, the numerical methods that integrate them, and the estimation procedures that surround the deterministic solve.